Chương 5

Các mô hình chuỗi-sang-chuỗi

> 天下万物之理，无独必有对。
>
> *Theo lẽ của Trời Đất và muôn vật, không có gì tồn tại đơn độc mà ắt phải có cái đối với nó.*
>
> –《近思录》 *Cận tư lục* (Suy ngẫm về những điều gần gũi), Chu Hy (1130–1200) và Lã Tổ Khiêm (1137–1181), bản dịch tiếng Anh của Chang [1967]

Trong thế giới ngôn ngữ, mọi thứ thường đi theo cặp. Có câu hỏi thì có câu trả lời; có một văn bản tiếng Trung thì có bản dịch tiếng Anh; có một câu thì có cây phân tích cú pháp của nó. Nhiều hệ thống NLP được thiết kế để mô hình hóa sự tương ứng giữa những cặp như vậy, tức là một phần tử được lấy làm đầu vào và phần tử kia làm đầu ra. Các bài toán này có thể được biểu diễn dưới một dạng mà ta đã gặp nhiều lần như sau:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}). \tag{5.1}$$

Chương này tập trung vào một họ bài toán đặc biệt, trong đó cả $\mathbf{x}$ và $\mathbf{y}$ đều là chuỗi từ, gọi là các bài toán **chuỗi-sang-chuỗi** (*sequence-to-sequence*) (hay *seq2seq*). Khác với các bài toán phân loại, nơi đầu ra $\hat{\mathbf{y}}$ được chọn từ một tập cố định các lớp, bài toán chuỗi-sang-chuỗi đòi hỏi mô hình sinh ra đầu ra từ một không gian đầu ra lớn theo cấp số mũ. Việc tìm $\hat{\mathbf{y}}$ trong trường hợp này phức tạp hơn đáng kể so với phân loại, vì nó cần những mô hình mạnh hơn để biểu diễn $\Pr(\mathbf{y}|\mathbf{x})$ và những thuật toán tìm kiếm hiệu quả hơn để giải Phương trình (5.1).

Chương này trước hết giới thiệu kiến trúc bộ mã hóa (*encoder*)–bộ giải mã (*decoder*) nổi tiếng cho việc mô hình hóa chuỗi-sang-chuỗi, rồi trình bày cơ chế chú ý (*attention mechanism*) như một mở rộng quan trọng của kiến trúc này. Các mô hình này đặt nền móng cho nhiều cách tiếp cận tiên tiến nhất được bàn đến trong các chương sau. Ngoài ra, chương này sẽ thảo luận bài toán tìm kiếm (giải mã (*decoding*)), vốn đóng vai trò quan trọng trong sinh chuỗi và các nhiệm vụ liên quan.

Chú giải của người dịch: Không gian đầu ra "lớn theo cấp số mũ" là thế nào?

Giả sử bộ từ vựng chỉ có $|V| = 10\,000$ từ và câu đích dài $n = 10$ từ. Mỗi vị trí có 10 000 lựa chọn, nên số câu có thể có là $10\,000^{10} = 10^{40}$ — nhiều hơn rất xa số giây kể từ Vụ Nổ Lớn (khoảng $4 \cdot 10^{17}$). Ta không thể liệt kê hết để chọn câu tốt nhất, vì vậy cần các thuật toán tìm kiếm thông minh (Mục 5.4). So với bài toán phân loại cảm xúc chỉ có 2–3 lớp, đây là một khác biệt khổng lồ.

5.1 Các bài toán chuỗi-sang-chuỗi

Trong suốt chương này, chúng ta dùng dịch máy (*machine translation*) làm ví dụ minh họa, vì hiện nay đây là một trong những nhiệm vụ chuỗi-sang-chuỗi phổ biến nhất. Ta dùng $\mathbf{x} = x_1 \dots x_m$ để ký hiệu một chuỗi từ trong một ngôn ngữ (gọi là chuỗi phía nguồn (*source-side sequence*) hay *chuỗi nguồn*), và dùng $\mathbf{y} = y_1 \dots y_n$ để ký hiệu một chuỗi từ trong một ngôn ngữ khác (gọi là chuỗi phía đích (*target-side sequence*) hay *chuỗi đích*). Với ký hiệu này, Phương trình (5.1) có thể được viết lại thành:

$$\begin{aligned} \hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{y_1 \dots y_n} \Pr(y_1 \dots y_n | x_1 \dots x_m). \end{aligned} \tag{5.2}$$

Như đã bàn ở Chương 1 và trong [Brown và cộng sự, 1993], cách phát biểu này kéo theo ba vấn đề cơ bản.

- **Mô hình hóa.** Trước hết, ta cần xác định dạng của $\Pr(\mathbf{y}|\mathbf{x})$. Trong chương này, chúng ta sẽ thấy $\Pr(\mathbf{y}|\mathbf{x})$ có thể được mô hình hóa bằng một mạng nơ-ron duy nhất dựa trên kiến trúc bộ mã hóa–bộ giải mã

và cơ chế chú ý. Lưu ý rằng đôi khi ta chỉ cần một mô hình để phân biệt chuỗi đích "tốt" với chuỗi đích "tồi". Khi đó, không nhất thiết đòi hỏi mô hình phải mang tính xác suất, và ta có thể dùng một hàm phân biệt (*discriminant function*) thay thế.

- **Huấn luyện.** Tiếp theo, ta cần học các tham số của mô hình $\Pr(\mathbf{y}|\mathbf{x})$ từ một tập dữ liệu huấn luyện. Vì $\Pr(\mathbf{y}|\mathbf{x})$ được hiện thực bằng một mạng nơ-ron, ta có thể huấn luyện nó theo cách tiêu chuẩn: tối ưu một hàm mất mát (*loss function*) bằng hạ gradient (*gradient descent*). Xem Chương 3 về các cách tiếp cận phổ biến để huấn luyện mạng nơ-ron. Trong chương này và các chương sau, chúng ta cũng sẽ bàn về các kỹ thuật được thiết kế riêng cho từng nhiệm vụ cụ thể.

- **Tìm kiếm (hay Giải mã).** Khi đã học được mô hình, ta sẽ tìm $\hat{\mathbf{y}}$ bằng cách tìm kiếm chuỗi đích làm cực đại $\Pr(\mathbf{y}|\mathbf{x})$. Điều này đặt ra một thách thức về tính toán, vì số chuỗi ứng viên tăng theo độ dài chuỗi tối đa và kích thước bộ từ vựng (*vocabulary*). Ở Mục 5.4, chúng ta sẽ bàn về các phương pháp tìm kiếm hiệu quả cho bài toán chuỗi-sang-chuỗi, đặc biệt là cho dịch máy.

Nhiều bài toán NLP có dạng như Phương trình (5.2) có thể được phát biểu thành bài toán chuỗi-sang-chuỗi. Bảng 5.1 liệt kê các ví dụ phổ biến về bài toán chuỗi-sang-chuỗi lấy từ các tài liệu nghiên cứu. Khi phía đích là văn bản, các bài toán như vậy có thể được xếp chung vào nhóm bài toán sinh văn bản (*text generation*), dù một hệ thống sinh văn bản tổng quát không đòi hỏi phía nguồn phải có dạng chuỗi. Ngoài xử lý ngôn ngữ và tiếng nói, bài toán chuỗi-sang-chuỗi có thể được tổng quát hóa cho những trường hợp mà đầu vào và/hoặc đầu ra của hệ thống vốn không có dạng chuỗi. Ví dụ, các hệ thống sinh văn bản từ ảnh (hay chú thích ảnh (*image captioning*)) và sinh ảnh từ văn bản đều phải xử lý ảnh, vốn thường được biểu diễn dưới dạng dữ liệu 2 chiều. Bằng cách biểu diễn ảnh thành chuỗi theo một cách nào đó (chẳng hạn chuỗi các *mảnh ảnh — patches*), các mô hình chuỗi-sang-chuỗi có thể được áp dụng trực tiếp cho các nhiệm vụ này.

Trong lịch sử, hầu hết các hệ thống cho những nhiệm vụ này được phát triển khá độc lập với nhau, dẫn đến các kiến trúc, đặc trưng và phương pháp huấn luyện khác nhau cho từng nhiệm vụ. Tuy nhiên, như sẽ thấy

Bảng 5.1: Các ví dụ về bài toán chuỗi-sang-chuỗi.

Nhiệm vụ	Nguồn	Đích
Dịch máy	Văn bản ở một ngôn ngữ	Văn bản ở ngôn ngữ khác
Hỏi đáp	Câu hỏi	Câu trả lời
Hệ thống hội thoại	Ngữ cảnh hội thoại	Câu đáp
Tóm tắt	Văn bản	Bản tóm tắt
Đơn giản hóa văn bản	Văn bản	Văn bản đã đơn giản hóa
Chuyển đổi phong cách văn bản	Văn bản theo một phong cách	Văn bản theo phong cách khác
Sửa lỗi ngữ pháp	Văn bản có lỗi	Văn bản đã sửa
Nhận dạng tiếng nói	Tiếng nói	Văn bản
Tổng hợp tiếng nói	Văn bản	Tiếng nói
Dịch tiếng nói	Tiếng nói ở một ngôn ngữ	Tiếng nói hoặc văn bản ở ngôn ngữ khác

trong chương này, khi ta biểu diễn các mô hình này bằng mạng nơ-ron và huấn luyện chúng theo kiểu đầu-cuối (*end-to-end*), dường như xuất hiện một khuôn mẫu "phổ quát" cho tất cả các bài toán này. Đây là một thay đổi lớn đối với cộng đồng AI, vì nhiều lĩnh vực nghiên cứu hội tụ lại với nhau và các hệ thống có thể được dùng chung giữa các lĩnh vực. Ta có thể hiểu sâu hơn về bản chất chung của rất nhiều bài toán khác nhau, dù trong thực tế vẫn có nhiều cân nhắc riêng cho từng nhiệm vụ. Trong các mục tiếp theo, chúng ta sẽ bàn về một số điểm chung giữa các mô hình chuỗi-sang-chuỗi.

Chú giải của người dịch: Huấn luyện "đầu-cuối"

Trước đây, một hệ thống dịch máy thường gồm nhiều mô-đun được xây dựng và tinh chỉnh riêng rẽ (tách từ, gióng hàng từ, bảng cụm từ, mô hình sắp xếp lại…). Huấn luyện *đầu-cuối* nghĩa là cả hệ thống là một mạng nơ-ron duy nhất, nhận đầu vào thô và trả ra đầu ra cuối cùng; mọi tham số được điều chỉnh đồng thời theo cùng một hàm mất mát. Nhờ vậy, cùng một "công thức" có thể dùng cho dịch máy, tóm tắt hay nhận dạng tiếng nói — chỉ cần thay dữ liệu huấn luyện.

5.2 Kiến trúc bộ mã hóa–bộ giải mã

Trong mục này, chúng ta bàn về kiến trúc bộ mã hóa–bộ giải mã và một mô hình dịch máy nơ-ron đơn giản xây dựng trên kiến trúc này.

5.2.1 Mã hóa và giải mã

Từ góc nhìn học có giám sát (*supervised learning*), lý tưởng nhất là ta muốn học một mô hình từ một số cặp chuỗi sao cho bất kỳ chuỗi phía nguồn nào cũng có thể được ánh xạ sang chuỗi phía đích tương ứng. Tuy nhiên, việc học các ánh xạ giữa những chuỗi biến rời rạc thường đòi hỏi xử lý dữ liệu nhiều chiều, và không tránh khỏi lời nguyền số chiều (*curse of dimensionality*).

Một cách tiếp cận để học ánh xạ như vậy là chia bài toán thành các bài toán con đơn giản hơn. Ta giả sử có một biểu diễn (*representation*) ít chiều được dùng chung bởi $\mathbf{x}$ và $\mathbf{y}$, ký hiệu là $\mathbf{H}$. Khi đó, ánh xạ $\mathbf{x} \rightarrow \mathbf{y}$ có thể được thực hiện bằng cách ánh xạ $\mathbf{x}$ sang $\mathbf{H}$ rồi từ đó sang $\mathbf{y}$. Một cách hình thức, cho một chuỗi phía nguồn $\mathbf{x}$, ta ánh xạ nó sang biểu diễn $\mathbf{H}$ bằng một hệ thống mã hóa (gọi là *bộ mã hóa*)

$$\mathbf{H} = \text{Encode}(\mathbf{x}). \tag{5.3}$$

Sau đó, ta ánh xạ $\mathbf{H}$ sang chuỗi phía đích $\mathbf{y}$ bằng một hệ thống giải mã (gọi là *bộ giải mã*)[1]

$$\mathbf{y} = \text{Decode}(\mathbf{H}). \tag{5.4}$$

Kiến trúc này, còn gọi là kiến trúc bộ mã hóa–bộ giải mã (*encoder-decoder architecture*), được dùng rộng rãi trong các hệ thống chuỗi-sang-chuỗi hiện đại (xem minh họa ở Hình 5.1). Dễ thấy dạng của Phương trình (5.3) giống với các mô hình chuỗi đã nhắc đến ở Chương 4, nên có nhiều mô hình mã hóa để lựa chọn, chẳng hạn LSTM hai chiều. Mục tiêu

[1] Điều quan trọng là phải phân biệt khái niệm giải mã (hay bộ giải mã) dùng trong các hệ thống chuỗi-sang-chuỗi truyền thống với khái niệm dùng trong kiến trúc bộ mã hóa–bộ giải mã. Hai khái niệm này thường bị nhầm lẫn, dù bản chất của chúng khác nhau. Trong nhiều hệ thống dịch máy hay nhận dạng tiếng nói, giải mã có cùng nghĩa với dịch hay phiên âm, tức là ta khôi phục $\mathbf{y}$ tối ưu từ $\mathbf{x}$. Như đã chỉ ra ở Phương trình (5.2), quá trình này bao gồm việc tìm kiếm trên mọi ứng viên $\mathbf{y}$. Do đó, cách dùng truyền thống của từ giải mã trong các hệ thống này là để chỉ một quá trình tìm kiếm (tức phép toán arg max trong Phương trình (5.2)) [Koehn, 2010]. Ngược lại, trong kiến trúc bộ mã hóa–bộ giải mã, giải mã có nghĩa là quá trình khôi phục chuỗi phía đích $\mathbf{y}$ từ biểu diễn trung gian $\mathbf{H}$. Nó hoàn toàn thuộc về mô hình hóa chứ không phải tìm kiếm. Cũng cần lưu ý rằng, dù thuật ngữ giải mã (hay bộ giải mã) được dùng theo những cách khác nhau, nó đều có thể được hiểu là quá trình ánh xạ một thông điệp đã mã hóa trở lại thông điệp gốc trong một hệ thống truyền thông như định nghĩa trong lý thuyết thông tin [Shannon, 1948b]. Theo nghĩa này, các quá trình giải mã trong những hệ thống đó đều làm đúng như tên gọi của nó: chuyển một thứ gì đó về dạng ban đầu.

của bộ giải mã là sinh ra chuỗi phía đích "tốt nhất" khi biết biểu diễn của chuỗi phía nguồn. Giống như các mô hình phân loại, việc dự đoán được thực hiện bằng cách trước hết tạo ra một phân phối trên mọi chuỗi có thể, rồi chọn chuỗi có xác suất lớn nhất. Như vậy, ta có thể định nghĩa lại $\text{Decode}(\cdot)$ như một hàm xác suất

$$\begin{aligned}\Pr(\cdot|\mathbf{H}) &= \text{Decode}(\mathbf{H}) \\ &= \text{Decode}(\text{Encode}(\mathbf{x})).\end{aligned} \tag{5.5}$$

Nói cách khác, cho một chuỗi phía đích $\mathbf{y}$ và một chuỗi phía nguồn $\mathbf{x}$, bộ giải mã gán cho $\mathbf{y}$ một xác suất

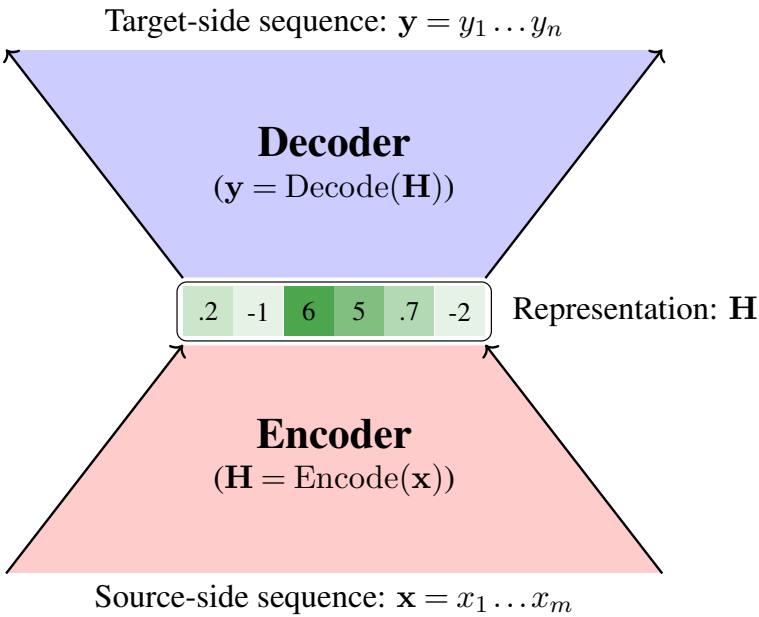
$$\Pr(\mathbf{y}|\mathbf{x}) = \Pr(\mathbf{y}|\mathbf{H}). \tag{5.6}$$

Sau đó, chuỗi tối ưu $\hat{\mathbf{y}}$ thu được bằng cách thực hiện $\arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x})$ như trong Phương trình (5.2). Trong nhiều hệ thống dựa trên kiến trúc bộ mã hóa–bộ giải mã, cả $\text{Encode}(\cdot)$ và $\text{Decode}(\cdot)$ đều là các mô hình được xây dựng từ mạng nơ-ron. Vì vậy, ta có thể coi mô hình chuỗi-sang-chuỗi là một mạng nơ-ron duy nhất và huấn luyện nó như thường lệ, miễn là toàn bộ mô hình là một tổ hợp nào đó của $\text{Encode}(\cdot)$ và $\text{Decode}(\cdot)$.

Chú giải của người dịch: Hình dung bộ mã hóa–bộ giải mã

Hãy tưởng tượng một phiên dịch viên: trước hết họ nghe hết câu tiếng Anh và "hiểu ý" trong đầu (đó là bước *mã hóa*, ý hiểu chính là $\mathbf{H}$), rồi diễn đạt ý đó thành câu tiếng Việt (bước *giải mã*). Trong máy, "ý hiểu" $\mathbf{H}$ là một dãy số thực, chẳng hạn vectơ $[0.2, -1, 6, 5, 0.7, -2]$ trong Hình 5.1. Ký hiệu $\Pr(\mathbf{y}|\mathbf{x})$ đọc là "xác suất của $\mathbf{y}$ khi đã biết $\mathbf{x}$" (xác suất có điều kiện): mô hình cho biết câu dịch $\mathbf{y}$ "hợp lý" đến mức nào đối với câu nguồn $\mathbf{x}$.

Để áp dụng kiến trúc bộ mã hóa–bộ giải mã vào một nhiệm vụ thực tế, ta cần đưa ra một số lựa chọn thiết kế, chẳng hạn dạng của $\mathbf{H}$, $\text{Encode}(\cdot)$ và $\text{Decode}(\cdot)$. Lấy một ví dụ rất đơn giản: xét nhiệm vụ tái tạo lại một từ đầu vào. Ta có thể định nghĩa $\text{Encode}(\cdot)$ là một mạng nơ-ron truyền thẳng nhận vào một từ (ở dạng biểu diễn one-hot (*one-hot*)) và trả ra một vectơ từ. Theo cách này, $\mathbf{H}$ là một biểu diễn phân tán (*distributed representation*) của từ đó. Sau đó, ta định nghĩa $\text{Decode}(\cdot)$ là một mạng nơ-ron truyền thẳng khác, nhận vectơ từ và sinh ra một phân phối trên bộ từ vựng. Khi huấn luyện, ta mong muốn học được một hệ thống gán xác suất lớn nhất



Hình 5.1: Kiến trúc bộ mã hóa–bộ giải mã. Trong trường hợp bài toán chuỗi-sang-chuỗi, nó biến đổi một chuỗi phía nguồn $\mathbf{x} = x_1 \dots x_m$ thành một chuỗi phía đích $\mathbf{y} = y_1 \dots y_n$. Quy trình này gồm hai bước: trước hết $\mathbf{x}$ được mã hóa thành một biểu diễn $\mathbf{H}$, rồi biểu diễn này được giải mã thành $\mathbf{y}$. (Nhãn trong hình: *Source-side sequence* = chuỗi phía nguồn, *Target-side sequence* = chuỗi phía đích, *Representation* = biểu diễn.)

cho chính từ đầu vào. Như đã bàn ở Chương 2, ta có thể gọi hệ thống này là một bộ tự mã hóa (*auto-encoder*), vốn là một trường hợp đặc biệt của kiến trúc bộ mã hóa–bộ giải mã.

5.2.2 Ví dụ: Dịch máy nơ-ron

Tiếp theo, chúng ta minh họa việc áp dụng kiến trúc bộ mã hóa–bộ giải mã qua một ví dụ cụ thể: **dịch máy nơ-ron** (*neural machine translation*) (**NMT**). Ta xét một mô hình NMT nổi tiếng dùng RNN hoặc các biến thể của nó cho cả bộ mã hóa lẫn bộ giải mã [Cho và cộng sự, 2014; Sutskever và cộng sự, 2014]. Bộ mã hóa của mô hình NMT này là một mô hình chuỗi tiêu chuẩn dựa trên RNN. Vì mô hình chuỗi dựa trên RNN đã được bàn chi tiết ở Chương 4, ở đây chúng ta chỉ điểm lại ngắn gọn. Giả sử bộ từ vựng phía nguồn là V_x và mỗi từ phía nguồn x_j được biểu diễn bằng một vectơ one-hot trong $\mathbb{R}^{|V_x|}$. Khi đó, x_j được biến đổi thành một vectơ h_s chiều (hay nhúng từ (*word embedding*))

$$\mathbf{x}_j^e = \text{Embed}_\text{s}(x_j), \tag{5.7}$$

trong đó $\text{Embed}_\text{s}(\cdot)$ là hàm nhúng từ. Chi tiết hơn về các mô hình nhúng từ có thể xem ở Chương 3.

Mô hình RNN nhận chuỗi các vectơ nhúng từ $\mathbf{x}_1^e \dots \mathbf{x}_m^e$ và sinh ra một chuỗi các vectơ trạng thái RNN $\mathbf{h}_1 \dots \mathbf{h}_m$. Một vectơ trạng thái RNN $\mathbf{h}_j \in \mathbb{R}^{d_h}$ được định nghĩa là

$$\mathbf{h}_j = \text{RNN}(\mathbf{h}_{j-1}, \mathbf{x}_j^e). \tag{5.8}$$

Ở đây, $\text{RNN}(\cdot)$ là một đơn vị RNN tóm tắt thông tin tính đến vị trí j bằng cách kết hợp trạng thái trước đó $\mathbf{h}_{j-1}$ và đầu vào hiện tại $\mathbf{x}_j^e$. Khi đó, trạng thái cuối cùng $\mathbf{h}_m$ có thể được xem là một biểu diễn nén của chuỗi đầu vào $x_1 \dots x_m$, và ta dùng $\mathbf{h}_m$ làm đầu ra của bộ mã hóa, viết là

$$\mathbf{h}_m = \text{Encode}(x_1 \dots x_m). \tag{5.9}$$

Hình 5.2 (a–b) minh họa quá trình mã hóa. Lưu ý rằng mô hình mô tả ở trên chỉ gồm một RNN một tầng. Trong các hệ thống thực tế, khung này có thể dễ dàng được mở rộng để có nhiều tầng và các đơn vị hồi quy mạnh hơn (chẳng hạn các đơn vị LSTM).

Chú giải của người dịch: RNN "đọc" câu nguồn như thế nào?

Hãy hình dung RNN như một người đọc từng từ một và ghi chú vào một cuốn sổ nhỏ có kích thước cố định (d_h ô số). Ở mỗi bước j, người đó nhìn ghi chú cũ $\mathbf{h}_{j-1}$ và từ mới $\mathbf{x}_j^e$, rồi viết lại ghi chú mới $\mathbf{h}_j$. Ví dụ với câu "I love cats" ($m = 3$): $\mathbf{h}_1$ tóm tắt "I", $\mathbf{h}_2$ tóm tắt "I love", $\mathbf{h}_3$ tóm tắt cả câu — và $\mathbf{h}_3$ chính là $\mathbf{H}$ được chuyển sang bộ giải mã. Ký hiệu $\mathbf{h}_j \in \mathbb{R}^{d_h}$ nghĩa là $\mathbf{h}_j$ là một vectơ gồm d_h số thực.

Bộ giải mã của mô hình NMT là một mô hình ngôn ngữ (*language model*) tiêu chuẩn dựa trên RNN, tức là nó dự đoán từ tiếp theo y_{i+1} khi biết mọi từ trước đó $y_1 \dots y_i$. Để đưa thông tin phía nguồn vào bản dịch, một cách đơn giản là coi $\mathbf{h}_m$ là trạng thái khởi đầu của RNN phía đích. Gọi $\mathbf{y}_0^e \in \mathbb{R}^{d_s}$ là vectơ từ của ký hiệu bắt đầu $\langle \text{SOS} \rangle$ (ký hiệu là y_0). Trạng thái RNN khởi đầu tương ứng được cho bởi

$$\mathbf{s}_0 = \text{RNN}(\mathbf{h}_m, \mathbf{y}_0^e). \tag{5.10}$$

Ở đây, $\text{RNN}(\cdot)$ có cùng dạng với đơn vị hồi quy dùng trong bộ mã hóa, nhưng với các tham số khác.

Với $i > 0$, vectơ trạng thái $\mathbf{s}_i \in \mathbb{R}^{d_s}$ được định nghĩa là

$$\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{y}_i^e). \tag{5.11}$$

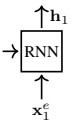
Sau đó, $\mathbf{s}_i$ được đưa vào một tầng Softmax để tạo ra một phân phối trên bộ từ vựng phía đích V_y. Đầu ra của tầng Softmax được cho bởi

$$\begin{aligned} \Pr(\cdot | y_1 \dots y_i, x_1 \dots x_m) &= \Pr(\cdot | \mathbf{s}_i) \\ &= \text{Softmax}(\mathbf{s}_i \mathbf{U}_y + \mathbf{b}_y), \end{aligned} \tag{5.12}$$

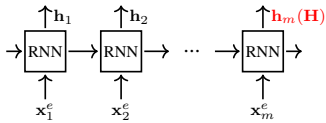
trong đó $\mathbf{U}_y \in \mathbb{R}^{d_s \times |V_y|}$ và $\mathbf{b}_y \in \mathbb{R}^{|V_y|}$ là các tham số của tầng Softmax. $\Pr(y_{i+1} | y_1 \dots y_i, x_1 \dots x_m)$ có thể được xem là xác suất dự đoán từ y_{i+1} với điều kiện đã biết cả các từ đã dịch $y_1 \dots y_i$ và chuỗi phía nguồn $x_1 \dots x_m$. Xem Hình 5.2 (c–d) để thấy minh họa về việc dự đoán từ của một bộ giải mã.

Dựa trên mô hình dự đoán từ này, xác suất đồng thời thường dùng trong các tài liệu về NMT được phát biểu như sau:

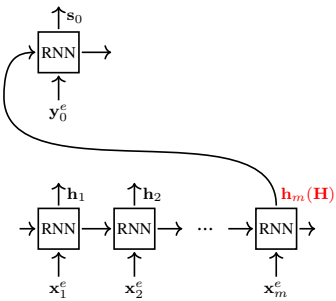
$$\Pr(\mathbf{y} | \mathbf{x}) = \Pr(y_0 y_1 \dots y_n | x_1 \dots x_m)$$



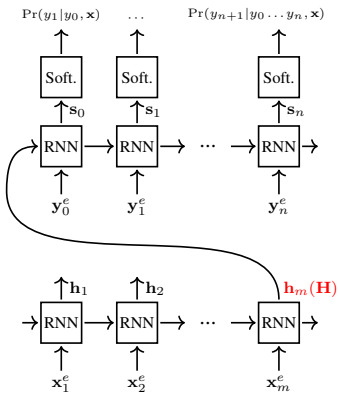
(a) The encoding process starts



(b) The encoder represents $\mathbf{x}$ as $\mathbf{H}$



(c) The decoder takes the representation of $\mathbf{x}$



(d) The decoder predicts the target-side words

Hình 5.2: Các bước mã hóa và giải mã của một hệ thống NMT dựa trên RNN. Bộ mã hóa là một RNN tiêu chuẩn. Quá trình mã hóa bắt đầu từ từ đầu tiên của phía nguồn và kết thúc ở từ cuối cùng của phía nguồn. Trạng thái cuối của RNN được lấy làm biểu diễn của toàn bộ chuỗi phía nguồn (tức $\mathbf{H} = \mathbf{h}_m$). Bộ giải mã là một RNN khác. Ở bước đầu tiên, nó nhận $\mathbf{H}$ từ bộ mã hóa. Sau khi biểu diễn $(\mathbf{y}_0^e \dots \mathbf{y}_i^e, \mathbf{H})$ thành $\mathbf{s}_i$ tại vị trí i, một tầng Softmax được áp dụng để dự đoán từ tiếp theo y_{i+1}. (Các tiểu hình: (a) quá trình mã hóa bắt đầu; (b) bộ mã hóa biểu diễn $\mathbf{x}$ thành $\mathbf{H}$; (c) bộ giải mã nhận biểu diễn của $\mathbf{x}$; (d) bộ giải mã dự đoán các từ phía đích. "Soft." là tầng Softmax.)

$$= \Pr(y_0|x_1 \ldots x_m) \Pr(y_1 \ldots y_n|y_0, x_1 \ldots x_m)$$
$$= \prod_{i=0}^{n-1} \Pr(y_{i+1}|y_0 \ldots y_i, x_1 \ldots x_m). \quad (5.13)$$

Đôi khi, phương trình này cũng được viết dưới một dạng tương đương:

$$\Pr(\mathbf{y}|\mathbf{x}) = \prod_{i=1}^{n} \Pr(y_i|y_0 \ldots y_{i-1}, x_1 \ldots x_m). \quad (5.14)$$

Ở đây, ta giả sử $\mathbf{y}$ luôn bắt đầu bằng y_0 (tức $\langle\text{SOS}\rangle$), sao cho $\Pr(y_0|x_1 \ldots x_m) = 1$. Trong nhiều hệ thống thực tế, người ta cũng thường giả sử $\mathbf{y}$ kết thúc bằng một ký hiệu đặc biệt $\langle\text{EOS}\rangle$. Do đó, ta có thể sửa phương trình này để bao gồm $\langle\text{SOS}\rangle$ và $\langle\text{EOS}\rangle$ ở cả phía nguồn lẫn phía đích như sau:

$$\begin{aligned}\Pr(y_0\mathbf{y}y_{n+1}|x_0\mathbf{x}x_{m+1}) &= \Pr(y_0y_1 \ldots y_ny_{n+1}|x_0x_1 \ldots x_mx_{m+1}) \\ &= \Pr(y_0|x_0 \ldots x_{m+1}) \cdot \\ &\quad \Pr(y_1 \ldots y_ny_{n+1}|y_0, x_0 \ldots x_{m+1}) \\ &= \prod_{i=0}^{n} \Pr(y_{i+1}|y_0 \ldots y_i, x_0 \ldots x_{m+1}), \quad (5.15)\end{aligned}$$

trong đó $x_0 = y_0 = \langle\text{SOS}\rangle$, $x_{m+1} = y_{n+1} = \langle\text{EOS}\rangle$, và $\Pr(y_0|x_0x_1 \ldots x_mx_{m+1}) = 1$.

Chú giải của người dịch: Tách xác suất của cả câu thành tích các xác suất từng từ

Phương trình (5.13) dùng *quy tắc nhân xác suất*: $\Pr(A, B) = \Pr(A) \cdot \Pr(B|A)$, áp dụng lặp lại. Ví dụ dịch "I love cats" thành "tôi yêu mèo" ($n = 3$), mô hình tính $\Pr(\text{tôi}|\langle\text{SOS}\rangle, \mathbf{x}) \cdot \Pr(\text{yêu}|\langle\text{SOS}\rangle \text{ tôi}, \mathbf{x}) \cdot \Pr(\text{mèo}|\langle\text{SOS}\rangle \text{ tôi yêu}, \mathbf{x})$. Nếu ba xác suất này lần lượt là 0.6, 0.5, 0.8 thì $\Pr(\mathbf{y}|\mathbf{x}) = 0.6 \cdot 0.5 \cdot 0.8 = 0.24$. Ký hiệu $\prod$ là tích (giống $\sum$ là tổng). Ký hiệu $\langle\text{EOS}\rangle$ (*end of sequence*) giúp mô hình "biết" khi nào dừng sinh từ, còn $\langle\text{SOS}\rangle$ (*start of sequence*) là tín hiệu bắt đầu.

Vì $\Pr(\mathbf{y}|\mathbf{x})$ có thể được hiện thực bằng một mạng nơ-ron, việc huấn luyện mô hình này khá đơn giản. Như đã mô tả ở Chương 4, các mô hình ngôn ngữ dựa trên RNN được huấn luyện bằng mất mát entropy chéo

(*cross-entropy*) và hạ gradient. NMT dùng chính phương pháp này để huấn luyện các tham số mô hình. Khi có được mô hình đã tối ưu, ta có thể dùng nó để dịch các câu mới. Việc tìm bản dịch tốt nhất cho một câu phía nguồn bất kỳ là một bài toán tìm kiếm tiêu chuẩn, mà chúng ta sẽ bàn ở Mục 5.4.

5.3 Cơ chế chú ý

Mô hình NMT bàn ở mục trước dựa trên một biểu diễn có độ dài cố định của chuỗi phía nguồn. Mặc dù mô hình này dễ hiện thực, trong nhiều ứng dụng thực tế nó thường không đủ tốt, vì một vectơ có độ dài cố định không đủ để biểu diễn một chuỗi có độ dài thay đổi, đặc biệt khi chuỗi dài. Do đó, ta cần một cơ chế nào đó để gắn kết bộ mã hóa và bộ giải mã một cách chi tiết hơn. Trong mục này, chúng ta bàn về cơ chế chú ý, nhờ đó mô hình có thể học, cho mỗi từ trong chuỗi phía đích, một biểu diễn thích ứng tập trung nhiều hơn vào những phần quan trọng của chuỗi phía nguồn.

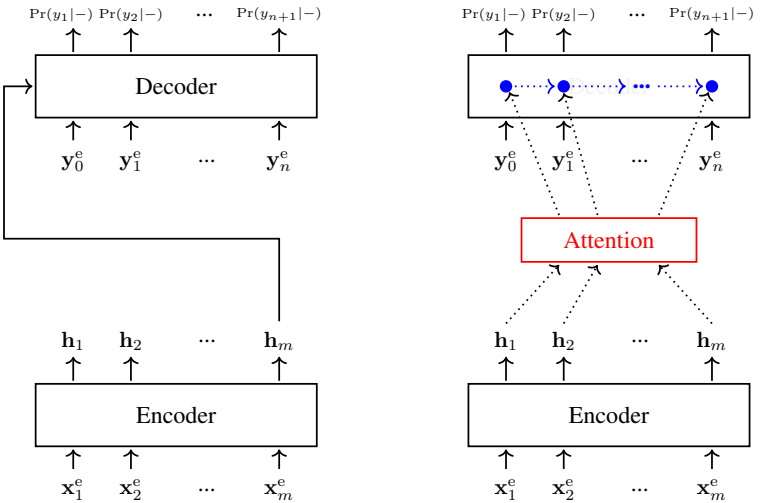
Ý tưởng này liên hệ chặt chẽ với các mô hình về sự chú ý trong tâm lý học, vì bản thân việc dịch là một quá trình nhận thức [Sternberg, 1996; Neisser, 2014]. Ý tưởng then chốt đằng sau loại mô hình này rất trực quan: sự chú ý thường tập trung vào những phần cụ thể của đầu vào. Trực giác này tạo nền tảng cho nhiều mô hình chuỗi-sang-chuỗi tiên tiến nhất, và cơ chế chú ý đã trở thành tiêu chuẩn trên thực tế trong mô hình hóa chuỗi-sang-chuỗi.

5.3.1 Một mô hình cơ bản

Nhớ lại rằng trong mô hình NMT ở mục trước, bộ mã hóa biểu diễn một chuỗi từ phía nguồn thành $\mathbf{h}_1, \ldots, \mathbf{h}_m$, còn bộ giải mã biểu diễn một chuỗi từ phía đích thành $\mathbf{s}_1, \ldots, \mathbf{s}_n$. Cơ chế chú ý giải quyết câu hỏi: làm thế nào học được một biểu diễn từ $\mathbf{h}_1, \ldots, \mathbf{h}_m$ sao cho biểu diễn này mã hóa tốt chuỗi phía nguồn đối với một trạng thái truy vấn $\mathbf{s}_i$ cho trước[2]. Từ góc độ xử lý thông tin, chừng nào ta bỏ qua ý nghĩa của $\mathbf{h}_1, \ldots, \mathbf{h}_m$ và $\mathbf{s}_i$ trong NMT, cơ chế chú ý có thể được coi là một cơ chế tổng quát để xử lý thông

[2]Theo quy ước trong dịch máy [Brown và cộng sự, 1993], ta dùng j để chỉ một vị trí trong chuỗi phía nguồn, và dùng i để chỉ một vị trí trong chuỗi phía đích.

tin đầu vào $\mathbf{h}_1, \ldots, \mathbf{h}_m$ bằng cách xem xét mỗi $\mathbf{h}_j$ liên quan đến trạng thái truy vấn $\mathbf{s}_i$ như thế nào. Hình 5.3 so sánh các kiến trúc NMT có và không có cơ chế chú ý.



(a) An NMT system without attention

(b) An NMT system with attention

Hình 5.3: Các kiến trúc NMT không có (trái) và có (phải) mô hình chú ý. Khi không dùng mô hình chú ý, một biểu diễn có độ dài cố định được dùng để sinh toàn bộ chuỗi phía đích. Ngược lại, khi dùng mô hình chú ý, một biểu diễn mới được tính riêng cho từng trạng thái phía đích, nhờ đó bộ giải mã có thể học cách tập trung vào các phần khác nhau của chuỗi phía nguồn để dự đoán một từ phía đích. ((a) Hệ thống NMT không có cơ chế chú ý; (b) hệ thống NMT có cơ chế chú ý.)

Một cách hình thức hơn, một mô hình chú ý tạo ra một tổ hợp có trọng số của $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$ có dạng

$$\mathbf{c}_i = \sum_{j=1}^{m} \alpha_{i,j} \cdot \mathbf{h}_j, \tag{5.16}$$

trong đó $\alpha_{i,j}$ là trọng số chú ý (*attention weight*), mô tả mô hình nên dựa vào $\mathbf{h}_j$ đến mức nào khi tính $\mathbf{c}_i$ cho $\mathbf{s}_i$. Đôi khi $\mathbf{c}_i$ còn được gọi là vectơ ngữ cảnh (*context vector*).

Một cách phổ biến để tính trọng số chú ý là chuẩn hóa các điểm gióng hàng (*alignment score*) theo dạng sau

$$\alpha_{i,j} = \frac{\exp(a(\mathbf{s}_i, \mathbf{h}_j))}{\sum_{j'=1}^{m} \exp(a(\mathbf{s}_i, \mathbf{h}_{j'}))}. \tag{5.17}$$

Ở đây, điểm gióng hàng $a(\mathbf{s}_i, \mathbf{h}_j)$ đo mức độ liên quan mạnh đến đâu giữa $\mathbf{h}_j$ và $\mathbf{s}_i$.

Chú giải của người dịch: Trọng số chú ý: một phép "bình quân có trọng số"

Phương trình (5.17) chính là hàm Softmax áp dụng lên các điểm $a(\mathbf{s}_i, \mathbf{h}_1), \ldots, a(\mathbf{s}_i, \mathbf{h}_m)$: mọi $\alpha_{i,j}$ đều dương và có tổng bằng 1. Ví dụ, với $m = 3$ và các điểm lần lượt là 2, 0, 0: $e^2 \approx 7.39$, $e^0 = 1$, nên $\alpha_{i,\cdot} \approx (7.39/9.39,\ 1/9.39,\ 1/9.39) \approx (0.79,\ 0.11,\ 0.11)$. Phương trình (5.16) khi đó giống cách tính *điểm trung bình môn có hệ số*: $\mathbf{c}_i \approx 0.79\,\mathbf{h}_1 + 0.11\,\mathbf{h}_2 + 0.11\,\mathbf{h}_3$ — vectơ ngữ cảnh chủ yếu "lấy" thông tin từ từ nguồn thứ nhất. Khi dịch từ "mèo", mô hình sẽ dồn trọng số vào vị trí của "cats".

Nói chung, $a(\mathbf{s}_i, \mathbf{h}_j)$ có thể được định nghĩa theo nhiều cách khác nhau [Graves và cộng sự, 2014; Bahdanau và cộng sự, 2014; Luong và cộng sự, 2015]. Một danh sách đầy đủ các hàm này có thể tìm thấy trong các bài tổng quan về chủ đề này [Chaudhari và cộng sự, 2021]. Dưới đây chúng ta giới thiệu một số hàm phổ biến.

- **Chú ý tích vô hướng** (*Dot-product Attention*). Một trong những phương pháp đơn giản nhất là đo độ tương đồng giữa $\mathbf{h}_j$ và $\mathbf{s}_i$. Như vậy, ta có thể tính tích vô hướng (*dot-product*) của hai vectơ như sau

$$\begin{aligned} a(\mathbf{s}_i, \mathbf{h}_j) &= \mathbf{s}_i \mathbf{h}_j^\top \\ &= \sum_{k=1}^{d_h} s_i(k) \cdot h_j(k). \end{aligned} \tag{5.18}$$

  Một biến thể của mô hình này, gọi là **chú ý tích vô hướng có co giãn** (*scaled dot-product attention*), nhân vế phải của Phương trình (5.18) với hệ số co giãn $\frac{1}{\beta}$ như sau

$$a(\mathbf{s}_i, \mathbf{h}_j) = \frac{\mathbf{s}_i \mathbf{h}_j^\top}{\beta}, \tag{5.19}$$

trong đó β là một siêu tham số. Chúng ta sẽ thấy một ví dụ về mô hình này ở phần sau của mục này.

- **Chú ý cosin** (*Cosine Attention*). Một độ đo tương đồng phổ biến khác trong đại số vectơ là cosin của góc giữa hai vectơ, cho bởi

$$\begin{aligned} a(\mathbf{s}_i, \mathbf{h}_j) &= \cos(\mathbf{s}_i, \mathbf{h}_j) \\ &= \frac{\mathbf{s}_i \mathbf{h}_j^\top}{\|\mathbf{s}_i\|_2 \cdot \|\mathbf{h}_j\|_2}, \end{aligned} \tag{5.20}$$

trong đó $\|\mathbf{a}\|_2 = (\mathbf{a} \cdot \mathbf{a})^{\frac{1}{2}}$ là chuẩn Euclid (*Euclidean norm*) của vectơ $\mathbf{a}$.

- **Chú ý tích vô hướng có trọng số** (*Weighted Dot-product Attention*). Mô hình chú ý này thực hiện một ánh xạ tuyến tính lên các vectơ đầu vào trước khi thực hiện phép tích vô hướng, cho bởi

$$a(\mathbf{s}_i, \mathbf{h}_j) = \mathbf{s}_i \mathbf{W}_\mathrm{a} \mathbf{h}_j^\top, \tag{5.21}$$

trong đó $\mathbf{W}_\mathrm{a} \in \mathbb{R}^{d_h \times d_h}$ là ma trận tham số của ánh xạ tuyến tính. Cả cách tiếp cận này và cách chú ý tích vô hướng đều còn được gọi là **chú ý nhân** (*multiplicative attention*) [Ruder, 2017].

- **Chú ý cộng** (*Additive Attention*). Trong chú ý cộng, các phần tử của hai vectơ được cộng lại theo một cách nào đó. Một dạng được dùng rộng rãi được đưa ra bởi Bahdanau và cộng sự [2014]

$$a(\mathbf{s}_i, \mathbf{h}_j) = \mathbf{v}_\mathrm{a}^\top \mathrm{TanH}(\mathbf{s}_i \mathbf{W}_\mathrm{s} + \mathbf{h}_j \mathbf{W}_\mathrm{h}), \tag{5.22}$$

trong đó $\mathbf{W}_\mathrm{h}, \mathbf{W}_\mathrm{s} \in \mathbb{R}^{d_h \times d_a}$ và $\mathbf{v}_\mathrm{a} \in \mathbb{R}^{d_a}$ là các tham số. $\mathrm{TanH}(\mathbf{s}_i \mathbf{W}_\mathrm{s} + \mathbf{h}_j \mathbf{W}_\mathrm{h})$ tạo ra một vectơ d_a chiều, trong đó mỗi phần tử là một tổng có trọng số đã được biến đổi của các phần tử của $\mathbf{h}_j$ và $\mathbf{s}_i$. Tiếp theo là một phép tích vô hướng với một vectơ trọng số khác $\mathbf{v}_\mathrm{a}$.

Chú giải của người dịch: Tích vô hướng và cosin — liên hệ với hình học lớp 10

Ở lớp 10 ta có $\vec{u} \cdot \vec{v} = |\vec{u}|\,|\vec{v}| \cos(\vec{u}, \vec{v})$, nên $\cos(\vec{u}, \vec{v}) = \dfrac{\vec{u} \cdot \vec{v}}{|\vec{u}|\,|\vec{v}|}$ — đó chính là Phương trình (5.20), chỉ khác là vectơ có d_h tọa độ thay vì 2. Ví dụ $\mathbf{s}_i = [1, 2]$,

$\mathbf{h}_j = [3, 1]$: tích vô hướng $= 1 \cdot 3 + 2 \cdot 1 = 5$, $\|\mathbf{s}_i\|_2 = \sqrt{5}$, $\|\mathbf{h}_j\|_2 = \sqrt{10}$, nên cosin $= 5/\sqrt{50} \approx 0.71$. Hai vectơ càng "cùng hướng" thì điểm càng lớn, tức $\mathbf{h}_j$ càng liên quan đến $\mathbf{s}_i$. Cách viết $\mathbf{s}_i\mathbf{h}_j^\top$ là tích của vectơ hàng $\mathbf{s}_i$ với vectơ cột $\mathbf{h}_j^\top$ (chuyển vị của $\mathbf{h}_j$), cho ra đúng tổng $\sum_k s_i(k)h_j(k)$. Hàm TanH là hàm tang hyperbolic, "nén" mọi số thực về khoảng $(-1, 1)$.

Giờ hãy quay lại Phương trình (5.16–5.17) và suy nghĩ lại về vai trò của các trọng số chú ý. Phương trình (5.17) định nghĩa một cách không chính thức một "phân phối" trên $\mathbf{h}_1 \dots \mathbf{h}_m$, viết là

$$\Pr(\mathbf{h}_j|\mathbf{s}_i) = \alpha_{i,j}. \tag{5.23}$$

Nếu ta coi $\mathbf{h}$ là một biến ngẫu nhiên nhận giá trị trong $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$, thì $\alpha_{i,j}$ có thể được xem là xác suất để $\mathbf{h} = \mathbf{h}_j$ với điều kiện $\mathbf{s}_i$, và Phương trình (5.16) có thể được viết lại thành

$$\begin{aligned}\mathbf{c}_i &= \sum_{j=1}^{m} \Pr(\mathbf{h}_j|\mathbf{s}_i) \cdot \mathbf{h}_j \\ &= \mathbb{E}_{\mathbf{h}\sim\Pr(\mathbf{h}|\mathbf{s}_i)}(\mathbf{h}).\end{aligned} \tag{5.24}$$

Nói cách khác, $\mathbf{c}_i$ có thể được xem là một *biểu diễn kỳ vọng* của chuỗi phía nguồn khi biết trạng thái phía đích $\mathbf{s}_i$, tức là kỳ vọng (*expectation*) của $\mathbf{h}$ trên tập $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ theo phân phối $\Pr(\mathbf{h}_j|\mathbf{s}_i)$. Điều này cho ta một khung tổng quát để mô tả cách bộ giải mã nhận thông tin từ bộ mã hóa: bộ giải mã là một *bên nhận*, quyết định nhận bao nhiêu thông tin từ mỗi *bên gửi*. Chẳng hạn, trong mô hình NMT ở mục trước, chỉ có một bên gửi là $\mathbf{h}_m$, nên bên nhận nhận toàn bộ thông tin mà bên gửi gửi đến. Ngược lại, trong mô hình NMT được trang bị cơ chế chú ý, có m bên gửi $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ và bên nhận nhận thông tin theo một phân phối mức ưu tiên trên các bên gửi.

Chú giải của người dịch: Kỳ vọng — "giá trị trung bình theo xác suất"

Ký hiệu $\mathbb{E}_{\mathbf{h}\sim\Pr(\mathbf{h}|\mathbf{s}_i)}(\mathbf{h})$ đọc là "kỳ vọng của $\mathbf{h}$ khi $\mathbf{h}$ được rút ngẫu nhiên theo phân phối $\Pr(\mathbf{h}|\mathbf{s}_i)$". Với biến ngẫu nhiên rời rạc, kỳ vọng là tổng "giá trị × xác suất". Ví dụ quen thuộc: tung một con xúc xắc cân đối, kỳ vọng số chấm là $1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + \dots + 6 \cdot \frac{1}{6} = 3.5$. Ở đây "giá trị" là các vectơ $\mathbf{h}_j$ và "xác suất" là $\alpha_{i,j}$, nên kỳ vọng chính là tổng có trọng số ở Phương trình (5.16).

Việc đưa mô hình chú ý vào quá trình dự đoán từ khá đơn giản. Để tận dụng cả thông tin phía nguồn và phía đích ở mỗi bước giải mã, ta sửa nhẹ định nghĩa của $\mathbf{s}_i$ để bao gồm vectơ ngữ cảnh tương ứng với trạng thái trước đó $\mathbf{s}_{i-1}$, như sau:

$$\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{c}_{i-1}, \mathbf{y}_i^e). \tag{5.25}$$

So với mô hình ở Phương trình (5.11), mô hình ở Phương trình (5.25) nhận thêm $\mathbf{c}_{i-1}$ làm đầu vào. Do đó, mô hình này xét cả biểu diễn của các từ phía đích $y_1 \dots y_{i-1}$ (được mã hóa trong $\mathbf{s}_{i-1}$ và $\mathbf{y}_i^e$) lẫn biểu diễn của toàn bộ chuỗi phía nguồn $x_1 \dots x_m$ (được mã hóa trong $\mathbf{c}_{i-1}$). Khi đó, phân phối của các từ đích tại vị trí i có thể được lấy điều kiện theo $\mathbf{s}_i$ như thường lệ:

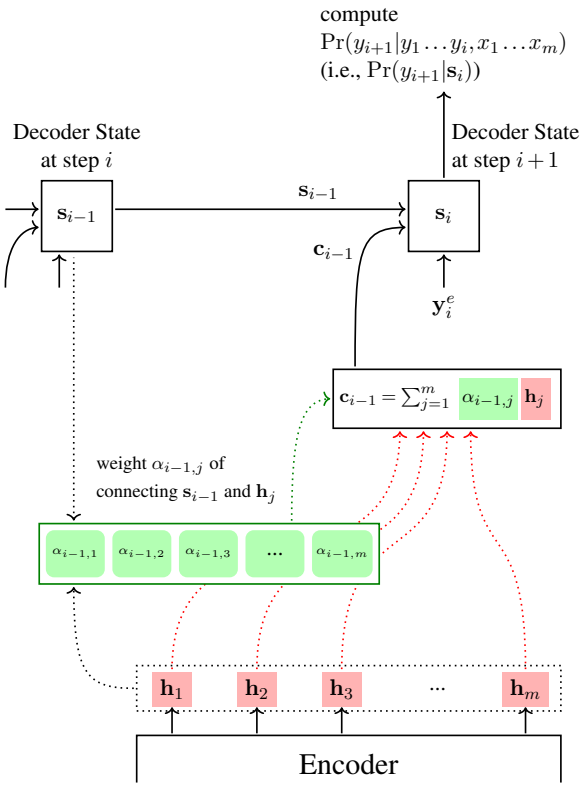
$$\Pr(\cdot|y_1, \dots, y_i, x_1, \dots, x_m) = \Pr(\cdot|\mathbf{s}_i), \tag{5.26}$$

trong đó $\Pr(\cdot|\mathbf{s}_i)$ thường là một tầng Softmax. Quá trình này được minh họa ở Hình 5.4.

Như vậy, ta đã có một mô hình để tính $\Pr(y_{i+1}|y_1 \dots y_i, x_1 \dots x_m)$. Các bước chính của mô hình này được tóm tắt như sau

1. Mã hóa chuỗi phía nguồn thành $\mathbf{h}_1 \dots \mathbf{h}_m$, trong đó $\mathbf{h}_j = \text{RNN}(\mathbf{h}_{j-1}, \mathbf{x}_j^e)$.
2. Lặp lại quy trình sau từ $i = 1$ đến $n - 1$.
    a. Tính điểm gióng hàng $a(\mathbf{s}_{i-1}, \mathbf{h}_j)$ cho mỗi j.
    b. Tính các trọng số chú ý $\{\alpha_{i-1,1}, \dots, \alpha_{i-1,m}\}$, trong đó $\alpha_{i-1,j} = \frac{\exp(a(\mathbf{s}_{i-1}, \mathbf{h}_j))}{\sum_{j'=1}^{m} \exp(a(\mathbf{s}_{i-1}, \mathbf{h}_{j'}))}$.
    c. Tính vectơ ngữ cảnh $\mathbf{c}_{i-1} = \sum_{j=1}^{m} \alpha_{i-1,j} \cdot \mathbf{h}_j$.
    d. Tính trạng thái phía đích $\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{c}_{i-1}, \mathbf{y}_i^e)$.
    e. Tính phân phối của các từ phía đích $\Pr(\cdot|\mathbf{s}_i)$.
    f. Tính $\Pr(y_{i+1}|y_1 \dots y_i, x_1 \dots x_m) = \Pr(y_{i+1}|\mathbf{s}_i)$ cho một từ y_{i+1} cho trước (như khi huấn luyện), hoặc chọn từ có khả năng nhất $\hat{y}_{i+1} = \arg\max_{y_{i+1}} \Pr(y_{i+1}|y_1 \dots y_i, x_1 \dots x_m)$ (như khi kiểm tra).

Trong các hệ thống thực tế, mô hình cơ bản này có thể được sửa đổi để dự đoán các từ phía đích tốt hơn. Chẳng hạn, ta có thể thêm các tầng hợp



Hình 5.4: Một mô hình chú ý cho NMT. Giả sử đến thời điểm này ta đã có các biểu diễn $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ và trạng thái bộ giải mã $\mathbf{s}_{i-1}$. Ta muốn tính trạng thái bộ giải mã ở bước tiếp theo. Để làm vậy, trước hết ta tính các trọng số chú ý bằng cách chuẩn hóa các điểm chú ý giữa $\mathbf{s}_{i-1}$ và $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$, rồi tính vectơ ngữ cảnh $\mathbf{c}_{i-1}$ bằng cách lấy tổng trên $\{\mathbf{h}_1, \dots, \mathbf{h}_m\}$ với các trọng số chú ý đó. Một trạng thái bộ giải mã mới $\mathbf{s}_i$ được tạo ra từ vectơ ngữ cảnh $\mathbf{c}_{i-1}$, trạng thái trước $\mathbf{s}_{i-1}$ và biểu diễn từ $\mathbf{y}_i^e$. $\mathbf{s}_i$ sẽ được dùng làm điều kiện để dự đoán phân phối các từ ở bước $i+1$. (Nhãn trong hình: *Decoder State at step* i = trạng thái bộ giải mã ở bước i; *weight* $\alpha_{i-1,j}$ *of connecting* $\mathbf{s}_{i-1}$ *and* $\mathbf{h}_j$ = trọng số $\alpha_{i-1,j}$ nối $\mathbf{s}_{i-1}$ với $\mathbf{h}_j$; *compute* = tính.)

nhất (*fusion layer*) để kết hợp $\mathbf{s}_i$, $\mathbf{c}_{i-1}$ và $\mathbf{y}_i^e$ trước tầng Softmax, nhờ đó có một mô hình sâu hơn để dự đoán [Bahdanau và cộng sự, 2014]. Một cách tiếp cận thường dùng khác là xếp chồng nhiều tầng RNN ở phía đích. Khi đó, ta có thể thực hiện chú ý ở mỗi tầng của chồng [Wu và cộng sự, 2016] hoặc chỉ ở tầng trên cùng [Luong và cộng sự, 2015]. Xem Mục 5.3.4 để biết thêm về các cách tiếp cận chú ý nhiều tầng.

5.3.2 Chú ý QKV

Vì cơ chế chú ý là một cách tiếp cận mạnh mẽ đến vậy, nhiều biến thể của nó đã được phát triển. Có lẽ cách tiếp cận được dùng rộng rãi nhất là phát biểu lại bài toán chú ý thành bài toán khớp một truy vấn (*query*) với một tập các cặp khóa (*key*)–giá trị (*value*). Nó đặt nền móng cho mô hình chuỗi nổi tiếng — Transformer [Vaswani và cộng sự, 2017].

Ở đây ta giả sử có một số cặp khóa–giá trị $\{(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)\}$ và một truy vấn $\mathbf{q}$. Mục tiêu của mô hình chú ý truy vấn–khóa–giá trị (*query-key-value attention*) (hay *chú ý QKV*) là tổng hợp các giá trị bằng cách xét sự tương ứng giữa truy vấn và các khóa. Đây là một bài toán tìm kiếm tiêu chuẩn trong các hệ cơ sở dữ liệu, trong đó thông tin được trả về ở dạng gốc hoặc một dạng mới khi nó khớp với truy vấn. Trong chú ý QKV, kết quả tìm kiếm không phải là một giá trị đơn lẻ trong $\{\mathbf{v}_1, \ldots, \mathbf{v}_m\}$ mà là một tổ hợp có trọng số của các giá trị này. Đây là khác biệt then chốt giữa mô hình chú ý này và các mô hình tìm kiếm thông thường.

Một cách hình thức, kết quả của chú ý QKV được định nghĩa là

$$\mathbf{c} = \sum_{j=1}^{m} \alpha_j \mathbf{v}_j, \tag{5.27}$$

trong đó

$$\alpha_j = \frac{\exp(\mathbf{q}\mathbf{k}_j^\top / \beta)}{\sum_{j'=1}^{m} \exp(\mathbf{q}\mathbf{k}_{j'}^\top / \beta)} \tag{5.28}$$

là trọng số chú ý. Hóa ra mô hình trên có đúng cùng dạng tổng quát với mô hình mô tả ở tiểu mục trước, và $\mathbf{c}$ đơn giản có thể được xem là một vectơ ngữ cảnh.

Chú giải của người dịch: Truy vấn, khóa, giá trị — ví dụ từ thư viện

Hãy tưởng tượng bạn đến thư viện với một *câu hỏi* (truy vấn $\mathbf{q}$). Mỗi cuốn sách có một *nhãn gáy* mô tả nội dung (khóa $\mathbf{k}_j$) và *nội dung bên trong* (giá trị $\mathbf{v}_j$). Một hệ cơ sở dữ liệu thông thường chỉ trả về đúng cuốn sách có nhãn khớp nhất. Chú ý QKV thì "mềm" hơn: nó so câu hỏi với mọi nhãn (bằng tích vô hướng $\mathbf{q}\mathbf{k}_j^\top$), đổi điểm thành tỉ lệ α_j bằng Softmax, rồi trộn nội dung các cuốn sách theo tỉ lệ đó. Ví dụ nếu $\alpha = (0.7, 0.2, 0.1)$ thì $\mathbf{c} = 0.7\mathbf{v}_1 + 0.2\mathbf{v}_2 + 0.1\mathbf{v}_3$.

Mặc dù dạng cơ bản của chú ý QKV không có gì mới, nó có thể xử lý nhiều bài toán khác nhau bằng cách gán cho $\mathbf{q}$, $\mathbf{k}_j$ và $\mathbf{v}_j$ những ý nghĩa thích hợp. Ở đây ta xét một trường hợp tổng quát hơn, trong đó có n truy vấn $\{\mathbf{q}_1, \ldots, \mathbf{q}_n\}$ và n vectơ đầu ra $\{\mathbf{c}_1, \ldots, \mathbf{c}_n\}$. Để đơn giản hóa ký hiệu, ta dùng $\mathbf{Q}$ để ký hiệu ma trận có hàng thứ i là $\mathbf{q}_i$:

$$\mathbf{Q} = \begin{bmatrix} \mathbf{q}_1 \\ \vdots \\ \mathbf{q}_n \end{bmatrix}. \tag{5.29}$$

Tương tự, ta có thể định nghĩa $\mathbf{K} = \begin{bmatrix} \mathbf{k}_1 \\ \vdots \\ \mathbf{k}_m \end{bmatrix}$, $\mathbf{V} = \begin{bmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_m \end{bmatrix}$ và $\mathbf{C} = \begin{bmatrix} \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_n \end{bmatrix}$. Khi đó, mô hình chú ý có thể được phát biểu thành

$$\mathbf{C} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\beta}\right)\mathbf{V}. \tag{5.30}$$

Hình 5.5 minh họa phương trình này. Lưu ý rằng Softmax$(\frac{\mathbf{Q}\mathbf{K}^\top}{\beta})$ là hàm Softmax theo từng hàng, tạo ra một ma trận các trọng số chú ý

$$\text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\beta}\right) = \begin{bmatrix} \alpha_{1,1} & \cdots & \alpha_{1,m} \\ \vdots & & \vdots \\ \alpha_{n,1} & \cdots & \alpha_{n,m} \end{bmatrix}, \tag{5.31}$$

trong đó một vectơ hàng $\begin{bmatrix} \alpha_{i,1} & \cdots & \alpha_{i,m} \end{bmatrix}$ biểu diễn một phân phối trên chỉ số của các giá trị. Ta có thể khai triển Phương trình (5.30) để dễ hiểu

mô hình hơn

$$\begin{aligned}\mathbf{C} &= \begin{bmatrix}\mathbf{c}_1\\ \vdots \\ \mathbf{c}_n\end{bmatrix}\\ &= \begin{bmatrix}\sum_{j=1}^{m}\alpha_{1,j}\mathbf{v}_j\\ \vdots \\ \sum_{j=1}^{m}\alpha_{n,j}\mathbf{v}_j\end{bmatrix}\\ &= \begin{bmatrix}\alpha_{1,1} & \dots & \alpha_{1,m}\\ \vdots & & \vdots \\ \alpha_{n,1} & \dots & \alpha_{n,m}\end{bmatrix}\begin{bmatrix}\mathbf{v}_1\\ \vdots \\ \mathbf{v}_m\end{bmatrix}.\end{aligned} \tag{5.32}$$

Chú giải của người dịch: Phép nhân ma trận trong Phương trình (5.30–5.32)

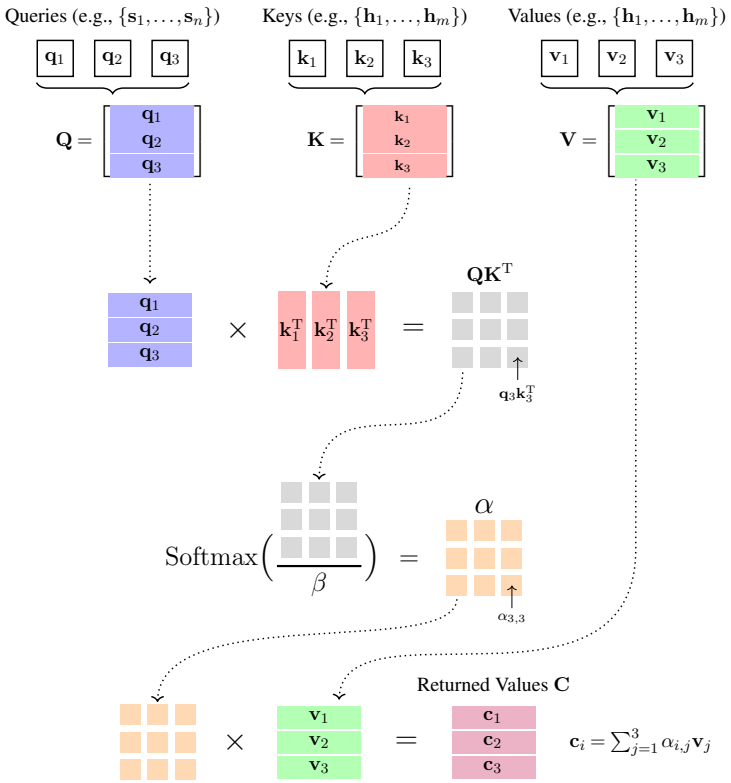
Tích hai ma trận **AB**: phần tử ở hàng i, cột j của kết quả bằng tích vô hướng của hàng i của **A** với cột j của **B**. Vì vậy phần tử (i, j) của $\mathbf{QK}^\top$ đúng bằng $\mathbf{q}_i\mathbf{k}_j^\top$ — điểm của truy vấn i với khóa j. Ví dụ nhỏ với vectơ 2 chiều: $\mathbf{Q} = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$, $\mathbf{K} = \begin{bmatrix}2 & 0\\ 0 & 1\end{bmatrix}$ thì $\mathbf{QK}^\top = \begin{bmatrix}2 & 0\\ 0 & 1\end{bmatrix}$. Sau khi áp Softmax cho *từng hàng* (mỗi hàng có tổng 1), nhân với **V** cho ra hàng i là $\sum_j \alpha_{i,j}\mathbf{v}_j$ — tức là tính n vectơ ngữ cảnh *cùng một lúc* chỉ bằng vài phép nhân ma trận, rất hợp với GPU.

Trong mô hình hóa chuỗi-sang-chuỗi, **Q**, **K** và **V** có thể được định nghĩa theo nhiều cách khác nhau. Để mô tả sự tương ứng giữa chuỗi phía nguồn và chuỗi phía đích, một cách tiếp cận, gọi là **chú ý bộ mã hóa–bộ giải mã** (*encoder-decoder attention*), đơn giản là giả sử

$$\mathbf{Q} = \begin{bmatrix}\mathbf{s}_1\\ \vdots \\ \mathbf{s}_n\end{bmatrix}, \tag{5.33}$$

và

$$\mathbf{K} = \mathbf{V} = \begin{bmatrix}\mathbf{h}_1\\ \vdots \\ \mathbf{h}_m\end{bmatrix}. \tag{5.34}$$



Hình 5.5: Mô hình chú ý QKV cho các lô truy vấn ($\mathbf{Q}$), khóa ($\mathbf{K}$) và giá trị ($\mathbf{V}$). Hình cho thấy một cách hiện thực trực tiếp công thức $\mathbf{C} = \mathrm{Softmax}(\frac{\mathbf{QK}^\top}{\beta})\mathbf{V}$. $\mathrm{Softmax}(\frac{\mathbf{QK}^\top}{\beta})$ tính các trọng số chú ý bằng cách chuẩn hóa tích vô hướng có co giãn của $\mathbf{Q}$ và $\mathbf{K}^\top$. Kết quả là một ma trận α, trong đó mỗi vectơ hàng biểu diễn các trọng số trên các giá trị khác nhau. Nhân α với $\mathbf{V}$, ta thu được một chuỗi giá trị mới, mỗi giá trị là một tổng có trọng số của các giá trị ban đầu. (Nhãn trong hình: *Queries/Keys/Values* = truy vấn/khóa/giá trị; *Returned Values* = các giá trị trả về.)

Trong trường hợp này, $\mathbf{C}$ là một chuỗi các biểu diễn mới của chuỗi phía nguồn khi biết các biểu diễn của chuỗi phía đích. Giống như mô hình mô tả ở tiểu mục trước, mỗi $\mathbf{c}_i \in \mathbf{C}$ có thể được dùng để dự đoán từ y_{i+1}.

Ngoài việc áp dụng mô hình này cho các bài toán chuỗi-sang-chuỗi, một loại cách tiếp cận khác là coi nó như một mô hình chuỗi, tức là ta dùng chú ý QKV để biểu diễn một chuỗi trong một ngôn ngữ. Trong trường hợp này, chú ý QKV còn được gọi là **tự chú ý** (*self-attention*), vốn là nền tảng của mô hình Transformer nổi tiếng [Vaswani và cộng sự, 2017]. Chẳng hạn, xét chuỗi trạng thái $\mathbf{h}_1 \dots \mathbf{h}_m$. Mô hình tự chú ý giả sử rằng

$$\mathbf{Q} = \mathbf{K} = \mathbf{V} = \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_m \end{bmatrix}. \tag{5.35}$$

Khi đó, đầu ra của mô hình là một chuỗi biểu diễn $\mathbf{c}_1 \dots \mathbf{c}_m$. $\mathbf{c}_j$ là một biểu diễn nắm bắt mối tương quan giữa $\mathbf{h}_j$ và mọi phần tử của chuỗi đầu vào. Chúng ta sẽ thấy thảo luận chi tiết hơn về mô hình này ở Chương 6.

Chú giải của người dịch: Chú ý bộ mã hóa–bộ giải mã và tự chú ý khác nhau ở đâu?

Cùng một công thức $\text{Softmax}(\mathbf{Q}\mathbf{K}^\top/\beta)\mathbf{V}$, chỉ khác ở chỗ "ai hỏi ai". Trong *chú ý bộ mã hóa–bộ giải mã*, các từ của câu *đích* đặt câu hỏi ($\mathbf{Q}$ là các $\mathbf{s}_i$) và tìm câu trả lời trong câu *nguồn* ($\mathbf{K} = \mathbf{V}$ là các $\mathbf{h}_j$). Trong *tự chú ý*, các từ của *cùng một câu* hỏi lẫn nhau. Ví dụ trong câu "Con mèo đuổi con chuột vì *nó* đói", tự chú ý có thể giúp biểu diễn của "nó" lấy nhiều thông tin từ "con mèo".

5.3.3 Chú ý đa đầu

Chú ý đa đầu (*multi-head attention*) là một mở rộng tự nhiên của các mô hình trên. Ý tưởng then chốt là thực hiện chú ý đồng thời trong các không gian con khác nhau của không gian biểu diễn, thay vì trong một không gian biểu diễn duy nhất. Để minh họa, xét một mô hình chú ý tiêu chuẩn nhận các chuỗi trạng thái phía nguồn và phía đích rồi trả ra một chuỗi trạng thái mới, viết là

$$\mathbf{c}_1 \dots \mathbf{c}_n = \text{Att}(\mathbf{h}_1 \dots \mathbf{h}_m, \mathbf{s}_1 \dots \mathbf{s}_n), \tag{5.36}$$

trong đó $\mathbf{h}_j, \mathbf{s}_i, \mathbf{c}_i \in \mathbb{R}^{d_h}$, và $\text{Att}(\cdot)$ là hàm chú ý. Ta có thể ánh xạ $\mathbf{h}_j$ thành τ vectơ $\{\mathbf{h}_j^{[1]}, \ldots, \mathbf{h}_j^{[\tau]}\}$ bằng các phép biến đổi tuyến tính sau

$$\mathbf{h}_j^{[1]} = \mathbf{h}_j \mathbf{W}_h^{[1]}, \tag{5.37}$$

$$\vdots$$

$$\mathbf{h}_j^{[\tau]} = \mathbf{h}_j \mathbf{W}_h^{[\tau]}, \tag{5.38}$$

trong đó $\mathbf{h}_j^{[1]}, \ldots, \mathbf{h}_j^{[\tau]} \in \mathbb{R}^{\frac{d_h}{\tau}}$, và $\mathbf{W}_h^{[1]}, \ldots, \mathbf{W}_h^{[\tau]} \in \mathbb{R}^{d_h \times \frac{d_h}{\tau}}$.

Tương tự, ta có thể ánh xạ $\mathbf{s}_i$ thành τ vectơ $\{\mathbf{s}_i^{[1]}, \ldots, \mathbf{s}_i^{[\tau]}\}$. Sau đó ta định nghĩa τ **không gian con đặc trưng** (*feature subspace*), trong đó hàm chú ý được thực hiện độc lập. Với không gian con đặc trưng thứ k, ta có

$$\mathbf{c}_1^{[k]} \ldots \mathbf{c}_n^{[k]} = \text{Att}(\mathbf{h}_1^{[k]} \ldots \mathbf{h}_m^{[k]}, \mathbf{s}_1^{[k]} \ldots \mathbf{s}_n^{[k]}). \tag{5.39}$$

Đầu ra của mô hình là một chuỗi các vectơ d_h chiều, mỗi vectơ có được bằng cách ghép nối các vectơ sinh ra trong tất cả các không gian con đặc trưng này, rồi thực hiện một phép biến đổi tuyến tính. Quy trình này được cho bởi

$$\mathbf{c}_1 = [\mathbf{c}_1^{[1]}, \ldots, \mathbf{c}_1^{[\tau]}]\mathbf{W}_c, \tag{5.40}$$

$$\ldots$$

$$\mathbf{c}_n = [\mathbf{c}_n^{[1]}, \ldots, \mathbf{c}_n^{[\tau]}]\mathbf{W}_c, \tag{5.41}$$

trong đó $\mathbf{W}_c \in \mathbb{R}^{d_h \times d_h}$.

Chú giải của người dịch: Ví dụ kích thước trong chú ý đa đầu

Giả sử $d_h = 6$ và $\tau = 3$ đầu. Mỗi $\mathbf{h}_j$ (6 số) được chiếu thành 3 vectơ nhỏ, mỗi vectơ $6/3 = 2$ số (nhân với ma trận $\mathbf{W}_h^{[k]}$ cỡ 6×2). Mỗi "đầu" thực hiện chú ý riêng trên các vectơ 2 chiều của mình và cho ra $\mathbf{c}_i^{[k]}$ (2 số). Ghép 3 kết quả lại được vectơ $2+2+2 = 6$ số, rồi nhân với $\mathbf{W}_c$ cỡ 6×6 để ra $\mathbf{c}_i$. Giống như một bài văn được 3 giám khảo chấm, mỗi người chú trọng một tiêu chí (ngữ pháp, nội dung, bố cục), rồi tổng hợp lại.

Theo ký hiệu dùng ở tiểu mục trước, ta có thể biểu diễn một chuỗi vectơ thành một ma trận, chẳng hạn $\mathbf{H} = \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_m \end{bmatrix} \in \mathbb{R}^{m \times d_h}$, $\mathbf{S} = \begin{bmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_n \end{bmatrix} \in$

$\mathbb{R}^{n\times d_h}$ và $\mathbf{C} = \begin{bmatrix}\mathbf{c}_1\\ \vdots \\ \mathbf{c}_n\end{bmatrix} \in \mathbb{R}^{n\times d_h}$. Với ký hiệu này, ta viết lại Phương trình (5.36) thành

$$\mathbf{C} = \text{Att}(\mathbf{H}, \mathbf{S}). \tag{5.42}$$

Để đưa ra định nghĩa hình thức của chú ý đa đầu, trước hết ta giới thiệu các hàm tách (*split*) và gộp (*merge*). Hàm tách chia mỗi vectơ hàng của một ma trận thành một số vectơ con, tạo ra một tensor 3 chiều. Ví dụ, tách một ma trận $\mathbf{A}$ cỡ $m \times d_h$ với τ sẽ tạo ra một tensor cỡ $\tau \times m \times \frac{d_h}{\tau}$[3]

$$\mathbf{A}_{\text{heads}} = \text{Split}(\mathbf{A}, \tau). \tag{5.43}$$

Hàm gộp thực hiện phép toán ngược của hàm tách. Cho một tensor cỡ $\tau \times n \times \frac{d_h}{\tau}$ (chẳng hạn $\mathbf{A}_{\text{heads}}$), nó gộp mỗi nhóm gồm τ mảng con $\frac{d_h}{\tau}$ chiều theo dạng

$$\mathbf{A}_{\text{merge}} = \text{Merge}(\mathbf{A}_{\text{heads}}, \tau). \tag{5.44}$$

Như vậy, dạng của chú ý đa đầu được cho bởi

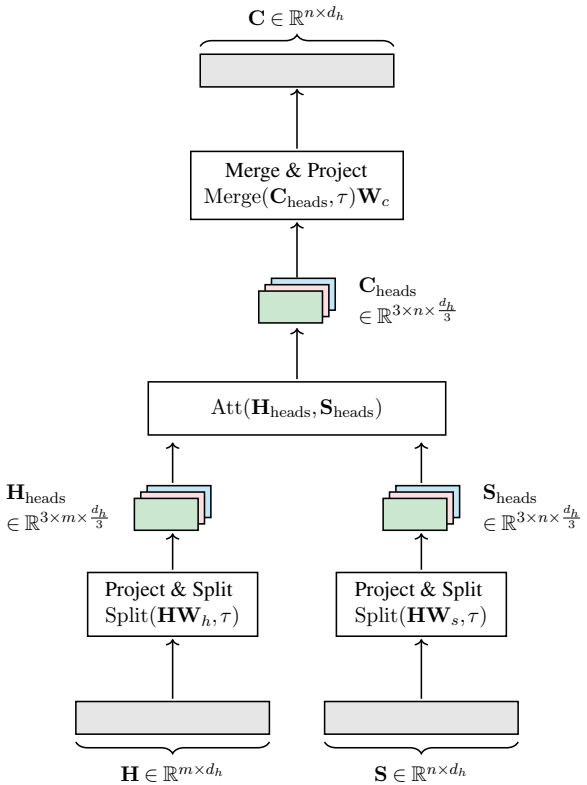
$$\begin{aligned}\mathbf{C} &= \mathbf{C}_{\text{merge}}\mathbf{W}_c \\ &= \text{Merge}(\mathbf{C}_{\text{heads}}, \tau)\mathbf{W}_c \\ &= \text{Merge}(\text{Att}(\mathbf{H}_{\text{heads}}, \mathbf{S}_{\text{heads}}), \tau)\mathbf{W}_c \qquad (5.45)\\ \mathbf{H}_{\text{heads}} &= \text{Split}(\mathbf{H}\mathbf{W}_h, \tau) \qquad (5.46)\\ \mathbf{S}_{\text{heads}} &= \text{Split}(\mathbf{S}\mathbf{W}_s, \tau), \qquad (5.47)\end{aligned}$$

trong đó $\mathbf{W}_h, \mathbf{W}_s \in \mathbb{R}^{d_h\times d_h}$ là các tham số. $\text{Split}(\mathbf{H}\mathbf{W}_h, \tau)$ hiện thực các phép chiếu ở Phương trình (5.37–5.38) cho mọi $\mathbf{h}_j$. Tương tự, $\text{Split}(\mathbf{S}\mathbf{W}_s, \tau)$ cũng có cách hiểu như vậy. Lưu ý rằng ở đây $\text{Att}(\cdot)$ được mở rộng để xử lý đầu vào đa đầu. Xem minh họa mô hình này ở Hình 5.6.

Chú ý đa đầu là một cách tiếp cận rất tổng quát, có thể mở rộng cho nhiều mô hình. Lấy một ví dụ đơn giản về sự mở rộng này, xét mô hình chú ý QKV đã bàn ở tiểu mục trước. Gọi $\text{Att}_{\text{QKV}}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ là hàm chú ý, và $\mathbf{Q} \in \mathbb{R}^{d_k}, \mathbf{K} \in \mathbb{R}^{d_k}, \mathbf{V} \in \mathbb{R}^{d_v}$ là các đầu vào. Mô hình chú ý QKV đa đầu được cho bởi

$$\mathbf{C} = \text{Merge}(\text{Att}_{\text{QKV}}(\mathbf{Q}_{\text{heads}}, \mathbf{K}_{\text{heads}}, \mathbf{V}_{\text{heads}}))\mathbf{W}_c, \tag{5.48}$$

[3]Một tensor cỡ $a \times b \times c$ có thể được xem là một mảng gồm a ma trận, mỗi ma trận có kích thước $b \times c$.



Hình 5.6: Một mô hình chú ý với $\tau = 3$ đầu. Trước hết, ta biến đổi các ma trận đầu vào thành biểu diễn đa đầu, tức là các tensor 3 chiều $\mathbf{H}_{\text{heads}} \in \mathbb{R}^{3 \times m \times \frac{d_h}{3}}$ và $\mathbf{S}_{\text{heads}} \in \mathbb{R}^{3 \times n \times \frac{d_h}{3}}$. Các tensor này sau đó được đưa vào một mô hình chú ý. Đầu ra của mô hình này là một tensor $\mathbf{C}_{\text{heads}} \in \mathbb{R}^{3 \times n \times \frac{d_h}{3}}$. Sau đó ta gộp các đầu của $\mathbf{C}_{\text{heads}}$, rồi thực hiện một phép biến đổi tuyến tính. Cuối cùng, ta thu được n vectơ kích thước d_h, biểu diễn bằng một ma trận $n \times d_h$. (Nhãn trong hình: *Project & Split* = chiếu và tách; *Merge & Project* = gộp và chiếu.)

$$\mathbf{Q}_{\text{heads}} = \text{Split}(\mathbf{Q}\mathbf{W}_q, \tau), \tag{5.49}$$

$$\mathbf{K}_{\text{heads}} = \text{Split}(\mathbf{K}\mathbf{W}_k, \tau), \tag{5.50}$$

$$\mathbf{V}_{\text{heads}} = \text{Split}(\mathbf{V}\mathbf{W}_v, \tau), \tag{5.51}$$

trong đó $\mathbf{W}_q \in \mathbb{R}^{d_k \times d_k}$, $\mathbf{W}_k \in \mathbb{R}^{d_k \times d_k}$, $\mathbf{W}_v \in \mathbb{R}^{d_v \times d_v}$, $\mathbf{W}_c \in \mathbb{R}^{d_v \times d_v}$ là các tham số mô hình.

Một ưu điểm của chú ý đa đầu là mỗi không gian con đặc trưng mô tả một góc nhìn khác nhau của sự chú ý (gọi là một **đầu chú ý** (*attention head*), hay gọi tắt là **đầu** — *head*). Do đó, việc ghép nối đầu ra của các đầu này biểu diễn một *tập hợp* (*ensemble*) các mô hình chú ý xử lý các phần khác nhau của dữ liệu. Điều này tương tự như học độc lập một nhóm mô hình rồi kết hợp chúng để tạo thành một mô hình mạnh hơn. Những cách tiếp cận như vậy đã được chứng minh là hữu ích trong nhiều bài toán [Opitz và Maclin, 1999; Zhou, 2012b]. Lưu ý rằng các mô hình chú ý đa đầu bàn ở đây được tham số hóa bởi các phép chiếu tuyến tính trên không gian đầu vào và đầu ra. Việc dùng các phép chiếu tuyến tính này nói chung là có ích khi mô hình trở nên sâu hơn và có thể mô tả những bài toán phức tạp hơn.

Từ góc độ thiết kế kiến trúc, chú ý đa đầu thuộc về một lớp rộng các mạng nơ-ron — những mạng gồm nhiều nhánh chồng tầng cùng xử lý một đầu vào (gọi là **mạng nơ-ron đa nhánh** (*multi-branch neural network*)). Tuy nhiên, khác với các cách tiếp cận thông thường đòi hỏi các kiến trúc mô hình khác nhau cho các nhánh khác nhau, cách tiếp cận chú ý đa đầu dựa trên một mô hình duy nhất cho tất cả các đầu. Nhờ vậy, các hệ thống này rất hiệu quả trong thực tế vì quá trình chú ý có thể chạy song song trên các đầu.

5.3.4 Chú ý nhiều tầng

Cho đến giờ, ta đã xét trường hợp **chú ý một tầng** (*single-layer attention*) — đầu ra của các mô hình chú ý có thể được viết thành một tổ hợp tuyến tính của các biểu diễn phía nguồn. Bây giờ ta mở rộng nó một cách tự nhiên thành **chú ý nhiều tầng** (*multi-layer attention*), trong đó quy trình chú ý một tầng được chạy nhiều lần để tạo thành một mô hình chú ý "sâu hơn".

Để làm điều này, một mạng nơ-ron nhiều tầng được tạo ra ở phía đích.

Kiến trúc mô hình khá đều đặn. Ta xếp chồng một số tầng chú ý, mỗi tầng tương tác với chuỗi phía nguồn và đưa đầu ra của nó cho tầng tiếp theo. Trong một tầng chú ý, ta thực hiện chú ý như thường lệ. Với tầng thứ l trong chồng, bước này nhận chuỗi phía nguồn (ký hiệu là $\mathbf{h}_1 \dots \mathbf{h}_m$) cùng đầu ra của tầng trước (ký hiệu là $\mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}$), và sinh ra một chuỗi vectơ bằng

$$\mathbf{c}_1^l \dots \mathbf{c}_n^l = \text{Att}(\mathbf{h}_1 \dots \mathbf{h}_m, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}), \tag{5.52}$$

trong đó $\text{Att}(\cdot)$ có thể là bất kỳ hàm chú ý nào được mô tả trong chương này.

Sau đó, ta tạo thêm một mạng nơ-ron khác $f(\cdot)$ để tăng khả năng mô hình hóa cho mô hình. Do đó, đầu ra của tầng chú ý được định nghĩa là

$$\mathbf{s}_1^l \dots \mathbf{s}_n^l = f(\mathbf{c}_1^l \dots \mathbf{c}_n^l, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}). \tag{5.53}$$

$f(\cdot)$ có thể được thiết kế theo nhiều cách [Sukhbaatar và cộng sự, 2015; Wu và cộng sự, 2016; Vaswani và cộng sự, 2017]. Một lựa chọn phổ biến là định nghĩa $f(\cdot)$ là một mạng truyền thẳng (*feed-forward network*) với một kết nối tắt (*residual connection*), cho bởi

$$f(\mathbf{c}_1^l \dots \mathbf{c}_n^l, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}) = \text{FFN}(\mathbf{c}_1^l \dots \mathbf{c}_n^l) + \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}. \tag{5.54}$$

Thay các vectơ $\mathbf{c}_1^l \dots \mathbf{c}_n^l$ bằng Phương trình (5.52), đầu ra của tầng l được viết dưới dạng

$$\mathbf{s}_1^l \dots \mathbf{s}_n^l = \text{FFN}(\text{Att}(\mathbf{h}_1 \dots \mathbf{h}_m, \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1})) + \mathbf{s}_1^{l-1} \dots \mathbf{s}_n^{l-1}. \tag{5.55}$$

Giống như các mô hình ở các tiểu mục trước, sẽ thuận tiện hơn nếu dùng ký hiệu gọn bằng cách biểu diễn một chuỗi vectơ thành một ma trận. Như vậy, mô hình này có thể được viết dưới một dạng khác

$$\mathbf{S}^l = \text{FFN}(\text{Att}(\mathbf{H}, \mathbf{S}^{l-1})) + \mathbf{S}^{l-1}. \tag{5.56}$$

Ở đây, $\text{FFN}(\cdot)$ thường là một mạng nơ-ron nhiều tầng với các hàm kích hoạt (*activation function*) phi tuyến. Ánh xạ đồng nhất (*identity mapping*) (tức $+\mathbf{S}^{l-1}$) tạo ra một đường đi trực tiếp từ đầu vào tới đầu ra của tầng, giúp việc huấn luyện một mạng nơ-ron sâu trở nên dễ dàng hơn.

Chú giải của người dịch: Vì sao kết nối tắt giúp huấn luyện mạng sâu?

Trong Phương trình (5.56), đầu ra = đầu vào + một "phần chỉnh sửa" $\text{FFN}(\text{Att}(\cdot))$. Mỗi tầng chỉ cần học *phần thay đổi* so với đầu vào, chứ không phải học lại toàn bộ. Nếu một tầng chưa học được gì hữu ích, phần chỉnh sửa gần bằng 0 và thông tin vẫn đi thẳng qua nhờ $+\mathbf{S}^{l-1}$. Khi lan truyền ngược, gradient cũng có "đường cao tốc" này để đi từ tầng trên cùng xuống các tầng dưới mà không bị suy giảm quá nhiều qua hàng chục tầng.

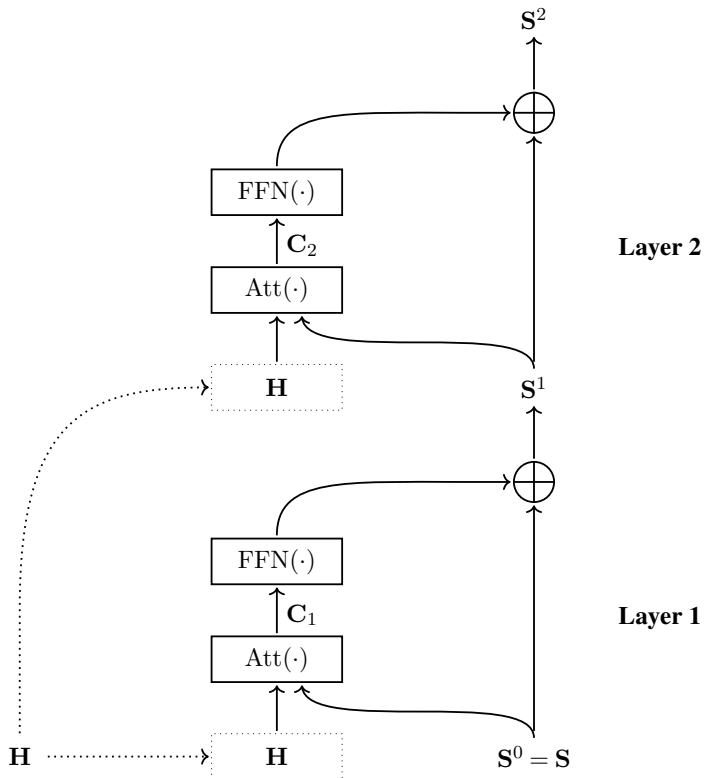
Hình 5.7 cho thấy kiến trúc của mô hình này. Mô hình chú ý bắt đầu với biểu diễn khởi đầu của chuỗi phía đích, tức là $\mathbf{S}^0 = \mathbf{S} = \begin{bmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_n \end{bmatrix}$. Nếu có L tầng chú ý thì đầu ra cuối cùng sẽ là $\mathbf{S}^L$.

5.3.5 Nhận xét

Ở trên, chúng ta đã xét một mô hình chú ý cơ bản và một loạt cải tiến. Các tài liệu về chú ý và các chủ đề liên quan chứa rất nhiều phương pháp mô hình hóa cách một hệ thống tập trung vào các phần khác nhau của đầu vào, cũng như nhiều ứng dụng của các mô hình như vậy. Tiểu mục này bàn về một số vấn đề thú vị.

1. Gióng hàng từ và chú ý

Chú ý có liên quan tới một dòng nghiên cứu lâu đời về gióng hàng (*alignment*) (từ), vốn mô hình hóa sự tương ứng giữa hai nhóm đơn vị ngôn ngữ. Trong NLP, gióng hàng là một khái niệm rất tổng quát, được dùng để chỉ nhiều bài toán khác nhau. Chẳng hạn, hầu hết các hệ thống dịch máy thống kê (*statistical machine translation*) được huấn luyện trên các văn bản song ngữ có gióng hàng từ-với-từ [Koehn và cộng sự, 2003; Chiang, 2005]. Do đó, các mô hình **gióng hàng từ** (*word alignment*) được phát triển để tạo ra các liên kết giữa các từ trong hai câu [Vogel và cộng sự, 1996; Och và Ney, 2003; Taskar và cộng sự, 2005; Dyer và cộng sự, 2013]. Mặc dù đầu ra của các hệ thống này là các biến rời rạc, các mô hình bên dưới thường mang tính xác suất và có thể dùng các biểu diễn liên tục. Vì vậy, sự tương ứng giữa gióng hàng từ và các mô hình chú ý



Hình 5.7: Một mô hình chú ý 2 tầng. Các tầng này nhận cùng các cặp "khóa–giá trị" (tức $\mathbf{H}$) nhưng mỗi tầng nhận một "truy vấn" khác nhau (tức $\mathbf{S}^l$). Mô hình chú ý là tiêu chuẩn: các vectơ ngữ cảnh $\mathbf{C}^l$ được sinh ra bằng cách nhận cả $\mathbf{H}$ và $\mathbf{S}^l$. Một mạng nơ-ron truyền thẳng được xây dựng để biến đổi $\mathbf{C}^l$, sau đó cộng thêm $\mathbf{S}^l$. Như vậy, mô hình này có thể được phát biểu thành $\mathbf{S}^l = \text{FFN}(\text{Att}(\mathbf{H}, \mathbf{S}^{l-1})) + \mathbf{S}^{l-1}$. $\mathbf{S}^l$ sau đó được dùng làm truy vấn ở tầng tiếp theo, tức là tầng $l+1$ nhận $\mathbf{H}$ và $\mathbf{S}^l$ rồi lặp lại quá trình trên. Đầu ra của tầng cuối cùng có thể được xem là một biểu diễn sâu hơn của $\mathbf{H}$ dành cho $\mathbf{S}$. (*Layer* = tầng.)

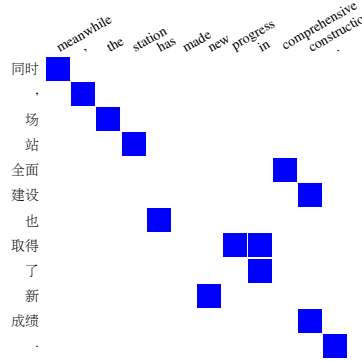
bàn ở đây là rõ ràng, vì cả hai đều gán trọng số cho các cặp từ.

Lưu ý rằng dù gióng hàng từ và chú ý có điểm tương đồng, mục tiêu của chúng khác nhau. Trong hầu hết trường hợp, các mô hình gióng hàng từ được dùng như những hệ thống riêng lẻ để tạo ra kết quả gióng hàng cho các hệ thống phía sau, trong khi các mô hình chú ý thường được coi là thành phần của những hệ thống lớn hơn và không hoạt động độc lập (xem Hình 5.8 để so sánh các mô hình này). Kết quả là, trong thực tế chúng phù hợp với những loại hệ thống chuỗi-sang-chuỗi khác nhau: gióng hàng từ là một bước trong các hệ thống nối tầng (*cascaded system*), còn chú ý biểu diễn các trạng thái trung gian của một mạng nơ-ron.

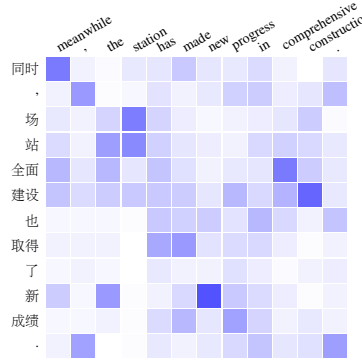
Tuy vậy, gióng hàng từ và chú ý là hai bài toán liên quan, và có thể hỗ trợ nhau trong một số trường hợp. Ví dụ, một cách để xem mô hình chú ý hoạt động ra sao là suy ra gióng hàng từ từ nó rồi đo chất lượng của các gióng hàng này [Tu và cộng sự, 2016; Li và cộng sự, 2019; Garg và cộng sự, 2019]. Ngoài ra, các hệ thống có cơ chế chú ý cũng có thể được định hướng bởi các nguồn gióng hàng từ bên ngoài [Mi và cộng sự, 2016b; Liu và cộng sự, 2016b].

Chú giải của người dịch: Đọc bản đồ nhiệt ở Hình 5.8

Mỗi hàng là một từ tiếng Trung, mỗi cột là một từ tiếng Anh. Ở hình (a), ô xanh đậm nghĩa là "hai từ này là bản dịch của nhau" (ví dụ 同时 — *meanwhile*, 新 — *new*); mỗi ô chỉ có thể là 0 hoặc 1. Ở hình (b), mỗi ô là một trọng số chú ý $\alpha_{i,j}$ trong khoảng $[0, 1]$: màu càng đậm thì mô hình càng "nhìn" vào cặp từ đó. Ta thấy các ô đậm ở (b) phần lớn trùng với các liên kết ở (a), nhưng chú ý "mờ" hơn và trải ra nhiều ô — đúng như bản chất mềm (phân phối xác suất) của nó.



(a) Heat map of word alignments



(b) Heat map of attention weights

Hình 5.8: Bản đồ nhiệt của một mô hình gióng hàng từ và một mô hình chú ý cho một cặp câu tiếng Trung và tiếng Anh. Các bản đồ nhiệt được biểu diễn dưới dạng lưới tô màu, trong đó mỗi ô mô tả sự tương ứng của một cặp từ tiếng Trung và tiếng Anh. Sự tương ứng này được thể hiện trên thang màu từ trắng (trọng số 0) đến xanh lam đậm (trọng số 1). ((a) Bản đồ nhiệt của gióng hàng từ; (b) bản đồ nhiệt của trọng số chú ý. Câu tiếng Anh: "Meanwhile, the station has made new progress in comprehensive construction." — "Trong khi đó, nhà ga đã đạt được tiến bộ mới trong việc xây dựng toàn diện.")

2. Đưa vào tri thức tiên nghiệm

Như đã bàn ở Mục 5.3.1, các mô hình chú ý ngầm định nghĩa một phân phối chú ý (*attention distribution*) trên $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$, được dùng để tính tổng có trọng số của các biểu diễn này. Phân phối này được biểu diễn qua các trọng số gióng hàng và được học như một phần của mô hình. Ngoài việc học phân phối chú ý theo kiểu đầu-cuối, ta cũng có thể định nghĩa nó dựa trên tri thức tiên nghiệm về cách sự chú ý được phân bố trong chuỗi.

Một cách tiếp cận là áp đặt trực tiếp một cấu trúc nào đó lên phân phối. Một ví dụ đơn giản là giới hạn sự chú ý trong một đoạn cục bộ của chuỗi và bỏ qua phần còn lại. Gọi $[\rho_i - D, \rho_i + D]$ là một cửa sổ $(2D+1)$ từ có tâm tại vị trí ρ_i của chuỗi phía nguồn. Ta có thể giới hạn sự chú ý từ $\mathbf{s}_i$ tới $\mathbf{h}_{\rho_i - D} \ldots \mathbf{h}_{\rho_i + D}$, thu được một vectơ ngữ cảnh cục bộ được phát biểu như sau:

$$\mathbf{c}_i = \text{Att}(\mathbf{h}_{\rho_i - D} \ldots \mathbf{h}_{\rho_i + D}, \mathbf{s}_i). \tag{5.57}$$

Cách tiếp cận này còn được gọi là **chú ý cục bộ** (*local attention*). Việc xác định ρ_i tương tự như **bài toán sắp xếp lại trật tự** (*reordering problem*) trong dịch máy. Khi dịch giữa các ngôn ngữ có trật tự từ tương tự nhau, người ta thường giả sử bản dịch là *đơn điệu* và ρ_i tuyến tính theo i [Koehn, 2004], chẳng hạn $\rho_i = \lfloor m\frac{i}{n} \rfloor$ hoặc $\lceil m\frac{i}{n} \rceil$. Một phương pháp khác là dùng một mạng nơ-ron để dự đoán một vị trí tương đối trong chuỗi phía nguồn (ký hiệu là r_i) [Luong và cộng sự, 2015]. Khi đó ρ_i có thể được định nghĩa là $\lfloor mr_i \rfloor$ hoặc $\lceil mr_i \rceil$.

Chú giải của người dịch: Ví dụ về chú ý cục bộ

$\lfloor x \rfloor$ là phần nguyên dưới (làm tròn xuống), $\lceil x \rceil$ là làm tròn lên. Giả sử câu nguồn có $m = 10$ từ, câu đích có $n = 5$ từ. Khi sinh từ đích thứ $i = 2$, ta đoán vị trí tương ứng ở câu nguồn là $\rho_2 = \lfloor 10 \cdot \frac{2}{5} \rfloor = 4$. Với $D = 1$, mô hình chỉ chú ý tới cửa sổ $[3, 5]$, tức $\mathbf{h}_3, \mathbf{h}_4, \mathbf{h}_5$, thay vì cả 10 vectơ. Giả thiết "đơn điệu" hợp lý với các cặp ngôn ngữ có trật tự từ gần nhau (như tiếng Anh–tiếng Pháp).

Trong một hướng nghiên cứu khác, một phân phối mới được xây dựng bằng cách kết hợp phân phối chú ý ban đầu với một phân phối tiên nghiệm (*prior distribution*) nào đó. Phân phối đơn giản nhất loại này có dạng nội suy tuyến tính

$$\widetilde{\Pr}(\mathbf{h}_j|\mathbf{s}_i) = \eta \cdot \Pr(\mathbf{h}_j|\mathbf{s}_i) + (1-\eta) \cdot \text{Prior}, \tag{5.58}$$

trong đó Prior ký hiệu phân phối tiên nghiệm, và η ký hiệu hệ số nội suy. Khi $\eta = 1$, đây là một mô hình chú ý tiêu chuẩn. Ngược lại, khi $\eta = 0$, sự chú ý hoàn toàn phụ thuộc vào phân phối tiên nghiệm [You và cộng sự, 2020].

Phân phối tiên nghiệm có thể được chọn theo nhiều cách. Một lựa chọn đơn giản là giả sử Prior là một phân phối Gauss (*Gaussian distribution*) $\mathcal{N}(\mu, \sigma^2)$. Điều này cho một cách hiểu trực quan rõ ràng: sự chú ý tập trung tại một điểm tiêu điểm trong chuỗi và giảm dần khi khoảng cách tới điểm này tăng lên. Để xác định giá trị trung bình (*mean*) và phương sai (*variance*) của phân phối Gauss, ta có thể dùng cùng kỹ thuật mô tả ở trên, ví dụ xây dựng hai mạng nơ-ron để lần lượt tính giá trị trung bình và phương sai.

Chú giải của người dịch: Phân phối Gauss và phép nội suy

Phân phối Gauss (phân phối chuẩn) $\mathcal{N}(\mu, \sigma^2)$ có đồ thị hình "quả chuông": cao nhất tại μ (tâm) và thấp dần đối xứng hai bên; σ càng nhỏ thì chuông càng hẹp. Dùng nó làm tiên nghiệm nghĩa là "ta tin rằng các từ nguồn gần vị trí μ quan trọng hơn". Phương trình (5.58) trộn hai ý kiến: ví dụ với $\eta = 0.8$, nếu mô hình chú ý cho Pr $= 0.5$ còn tiên nghiệm cho 0.2 thì kết quả là $0.8 \cdot 0.5 + 0.2 \cdot 0.2 = 0.44$. Vì là trung bình có trọng số của hai phân phối nên kết quả vẫn có tổng bằng 1.

Phép nội suy cũng có thể được coi là một bước trung gian trong quá trình tính phân phối chú ý. Ví dụ, xét chú ý QKV đã bàn ở Mục 5.3.2. Phép nội suy có thể được áp dụng lên tích vô hướng truy vấn–khóa [Yang và cộng sự, 2018a; Guo và cộng sự, 2019]. Để làm vậy, ta có thể sửa Phương trình (5.28) thành dạng sau

$$\begin{aligned}\alpha_j &= \text{Softmax}\left(\frac{\mathbf{q}\mathbf{k}_j^\top}{\beta} + \eta\,\text{Prior}\right)\\ &= \text{Softmax}\left(\frac{\mathbf{s}_i\mathbf{h}_j^\top}{\beta} + \eta\,\text{Prior}\right).\end{aligned} \tag{5.59}$$

Vì $\frac{\mathbf{q}\mathbf{k}_j^\top}{\beta}$ (hay $\frac{\mathbf{s}_i\mathbf{h}_j^\top}{\beta}$) không bị giới hạn trong $[0, 1]$, Prior được co giãn lại bởi một siêu tham số η.

Đôi khi, tiên nghiệm xuất hiện khi tri thức về sự chú ý đến từ các nguồn bên ngoài. Như đã bàn ở trên, đưa gióng hàng từ vào mô hình chú

ý là một trong những cách đơn giản nhất để làm điều này. Ý tưởng này có thể được mở rộng để tận dụng nhiều thông tin cấu trúc hơn, như độ sinh sản (*fertility*) và độ bao phủ (*coverage*) [Cohn và cộng sự, 2016; Feng và cộng sự, 2016; Tu và cộng sự, 2016], hoặc các ràng buộc đặc thù cho nhiệm vụ hơn, như giống hàng đơn điệu giữa chuỗi đầu vào và chuỗi đầu ra [Raffel và cộng sự, 2017; Chiu và Raffel, 2018]. Ngoài ra, giống như các hệ thống dịch máy dựa trên cú pháp, các cây phân tích cú pháp có thể được dùng làm đầu vào phụ trợ để định hướng quá trình chú ý. Chẳng hạn, cây phụ thuộc (*dependency tree*) là một nguồn thông tin được dùng rộng rãi trong mô hình hóa sự tương ứng giữa các từ, cho cả bài toán chuỗi-sang-chuỗi [Chen và cộng sự, 2018a] lẫn bài toán mô hình hóa chuỗi [Zhang và cộng sự, 2020c; Nguyen và cộng sự, 2020; Xu và cộng sự, 2021b].

Vì các cơ chế chú ý tiêu chuẩn có thể tốn kém về tính toán trong các ứng dụng quy mô lớn, các nhà nghiên cứu đã phát triển những phương án hiệu quả hơn bằng cách đưa các thiên kiến quy nạp (*inductive bias*) mạnh hơn vào thiết kế mô hình [Tay và cộng sự, 2020b]. Hướng nghiên cứu này thuộc về nhóm rộng hơn các phương pháp NLP hiệu quả. Chương 6 bàn chi tiết về các phương pháp này.

3. Chú ý trong mạng bộ nhớ

Không chỉ được quan tâm nhiều trong các hệ thống chuỗi-sang-chuỗi, cơ chế chú ý còn được dùng rộng rãi trong các mô hình nơ-ron dựa trên bộ nhớ [Sukhbaatar và cộng sự, 2015; Graves và cộng sự, 2014; Kumar và cộng sự, 2016]. Như đã bàn ở Chương 4, một hệ thống bộ nhớ duy trì một tập các mục dữ liệu, cho phép mô hình lưu giữ và truy xuất thông tin. Cho một truy vấn, hệ thống tính một điểm khớp giữa truy vấn và khóa của mỗi mục dữ liệu. Quy trình này còn được gọi là **định địa chỉ** (*addressing*) [Graves và cộng sự, 2014]. Việc định địa chỉ như vậy thường được hiện thực bằng cách trước hết biểu diễn truy vấn và các mục dữ liệu thành vectơ số thực, rồi tính một trọng số dựa trên một độ đo tương đồng giữa chúng. Kết quả truy xuất là một tổng có trọng số của các mục dữ liệu. Cách hình thức hóa này khớp hoàn toàn với mô hình chú ý QKV đã bàn ở Mục 5.3.2.

Với mối liên hệ về khái niệm giữa chú ý và cơ chế bộ nhớ, ít nhất là từ góc độ tính toán, nếu không phải từ góc độ tâm lý học, ta có thể xem chú ý như một quá trình truy xuất thông tin từ một ngân hàng bộ nhớ (tức $\{\mathbf{h}_1, \ldots, \mathbf{h}_m\}$) bằng một truy vấn cho trước (tức $\mathbf{s}_i$). Như vậy, ta có thể

diễn giải một hệ thống chuỗi-sang-chuỗi có cơ chế chú ý như sau. Ở phía nguồn, ta lưu thông tin vào một bộ nhớ được biểu diễn bằng một chuỗi vectơ $\mathbf{h}_1 \dots \mathbf{h}_m$. Sau đó, ở phía đích, ta truy xuất từ bộ nhớ này để khôi phục từng bước thông tin phía nguồn.

4. Vượt ra ngoài bài toán chuỗi-sang-chuỗi

Dù trong mục này chúng ta giới hạn thảo luận ở bài toán biến đổi một chuỗi thành một chuỗi khác, cơ chế chú ý tổng quát có thể được áp dụng cho một phạm vi nhiệm vụ rộng hơn. Như đã nhắc ở Mục 5.3.2 và sẽ bàn ở Chương 6, một biến thể nổi tiếng của cách tiếp cận này là tự chú ý. Trong tự chú ý, mô hình chú ý QKV được dùng như một mô hình chuỗi. Nó cung cấp một khung tổng quát để mô hình hóa các tương tác và phụ thuộc giữa các biến đầu vào, nhờ đó áp dụng được cho nhiều bài toán. Chẳng hạn, ta có thể ghép $\mathbf{h}_1 \dots \mathbf{h}_m$ và $\mathbf{s}_1 \dots \mathbf{s}_n$ thành một chuỗi kết hợp duy nhất $\mathbf{h}_1 \dots \mathbf{h}_m \mathbf{s}_1 \dots \mathbf{s}_n$, rồi áp dụng mô hình cho chuỗi mới này. Bằng cách đó, tự chú ý có thể được mở rộng một cách tự nhiên thành một mô hình chuỗi-sang-chuỗi [Lample và Conneau, 2019; Raffel và cộng sự, 2020]. Cách tiếp cận này hiệu quả ngay cả khi $\mathbf{h}_1 \dots \mathbf{h}_m$ và $\mathbf{s}_1 \dots \mathbf{s}_n$ biểu diễn các phương thức (*modality*) khác nhau. Ví dụ, nếu ta dùng $\mathbf{h}_1 \dots \mathbf{h}_m$ để biểu diễn văn bản và $\mathbf{s}_1 \dots \mathbf{s}_n$ để biểu diễn một ảnh, ta sẽ có một mô hình đa phương thức (*multi-modal model*) hợp nhất biểu diễn văn bản và hình ảnh bằng cách thực hiện tự chú ý trên cả hai [Chen và cộng sự, 2020c].

Một cách tiếp cận khác để học biểu diễn chung của các chuỗi là xây dựng nhiều mô hình chú ý, cho phép mỗi chuỗi chú ý tới chuỗi kia. Một ví dụ của cách tiếp cận này là **đồng chú ý** (*co-attention*), được dùng rộng rãi trong xử lý ngôn ngữ đa phương thức [Lu và cộng sự, 2016]. Trong các nhiệm vụ như hỏi đáp trực quan (*visual question answering*) (VQA), ta muốn xác định những phần nào của ảnh tương ứng với một từ cụ thể trong câu hỏi, và ngược lại, những từ nào trong câu hỏi khớp với một vùng hình ảnh cho trước. Để làm được điều này, ta xây dựng hai mô hình chú ý: một cho chú ý từ ảnh sang văn bản, và một cho chú ý từ văn bản sang ảnh. Đầu ra của cả hai mô hình đóng vai trò biểu diễn chung của ảnh và văn bản, và do đó có thể được dùng làm đặc trưng để dự đoán câu trả lời.

Các mô hình chú ý bàn trong mục này *bất biến* đối với thứ tự của đầu vào. Dù tính bất biến hoán vị (*permutation invariance*) này gây khó khăn cho việc mô hình hóa dữ liệu tuần tự, vấn đề có thể được giải quyết bằng

cách mã hóa tường minh thông tin vị trí vào chính đầu vào (xem Chương 4 và Chương 6). Mặt khác, sự đơn giản của cách phát biểu này khiến các mô hình này áp dụng được rộng rãi: dữ liệu đầu vào không cần có dạng chuỗi. Nhờ đó, cơ chế chú ý có thể được áp dụng trực tiếp cho các cấu trúc dữ liệu phức tạp hơn, chẳng hạn đồ thị [Veličković và cộng sự, 2018; Lee và cộng sự, 2019].

Chú giải của người dịch: Vì sao chú ý "không biết thứ tự"?

Trong Phương trình (5.16), $\mathbf{c}_i = \sum_j \alpha_{i,j}\mathbf{h}_j$ là một *tổng*, mà phép cộng thì giao hoán: nếu ta tráo thứ tự các $\mathbf{h}_j$ (cùng với trọng số của chúng), kết quả không đổi. Như vậy, nếu không có thông tin vị trí, chú ý xem "mèo đuổi chuột" và "chuột đuổi mèo" như cùng một "túi từ". Đó là lý do Transformer phải thêm *mã hóa vị trí* vào đầu vào (Chương 6). Ngược lại, chính tính chất này giúp chú ý dùng được cho dữ liệu không có thứ tự tự nhiên như đồ thị.

5.4 Tìm kiếm

Tìm kiếm (*search*) là một bài toán nền tảng của trí tuệ nhân tạo và đóng vai trò then chốt trong nhiều hệ thống NLP. Ở đây, bài toán tìm kiếm là một thách thức về mặt tính toán, bởi số lượng giả thuyết (*hypothesis*) trong không gian tìm kiếm (*search space*) tăng theo hàm mũ cùng với độ dài của chuỗi và kích thước của bộ từ vựng phía đích. Trong bối cảnh này, tìm kiếm vét cạn (*exhaustive search*) tốn kém đến mức không thể thực hiện được. Vì vậy, các hệ thống thực tế thường dùng các thuật toán tìm kiếm hoặc các phương pháp heuristic (*heuristic*) để bảo đảm có thể tìm được lời giải tối ưu hoặc gần tối ưu trong một khoảng thời gian tính toán chấp nhận được.

Với nhiều ứng dụng chuỗi-sang-chuỗi trong thực tế, bài toán tìm kiếm — thường được gọi là suy luận hay giải mã (*decoding*) — có thể được phát biểu như sau:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}\in\Omega} \text{Score}(\mathbf{x}, \mathbf{y}), \tag{5.60}$$

trong đó $\text{Score}(\mathbf{x}, \mathbf{y})$ là một hàm tính điểm (*scoring function*) đo chất lượng của $\mathbf{y}$ khi biết $\mathbf{x}$.

Cách phát biểu này hơi khác so với Phương trình (5.2). Thứ nhất, ta dùng $\text{Score}(\mathbf{x}, \mathbf{y})$ thay cho $\Pr(\mathbf{y}|\mathbf{x})$ làm hàm mục tiêu. Mặc dù cách tiêu chuẩn để huấn luyện các mô hình chuỗi-sang-chuỗi là cực đại hóa $\Pr(\mathbf{y}|\mathbf{x})$ (hoặc $\Pr(\mathbf{x}, \mathbf{y})$), ở thời điểm suy luận ta thường phải tính đến những vấn đề riêng của từng tác vụ, chẳng hạn thiên lệch độ dài (*length bias*). Do đó, người ta thường đưa thêm các số hạng phạt và thưởng bên cạnh $\Pr(\mathbf{y}|\mathbf{x})$ để định nghĩa mục tiêu tìm kiếm cuối cùng (xem Mục 5.4.1). Khác biệt thứ hai nằm ở chính không gian tìm kiếm: nó bị giới hạn trong một tập con Ω. Trên thực tế, Ω là một không gian tìm kiếm đã được cắt tỉa, chỉ chứa một số tương đối nhỏ các giả thuyết có xác suất cao. Điều này thường đạt được nhờ các kỹ thuật cắt tỉa (*pruning*) và các thuật toán tìm kiếm tiên tiến (xem Mục 5.4.2). Trong mục này, chúng ta sẽ tìm hiểu các giải pháp cho những thách thức trên và bàn về một số cải tiến. Dù các phương pháp này phần lớn được thúc đẩy bởi những phát triển trong dịch máy, các khái niệm là tổng quát và áp dụng rộng rãi cho hầu hết các tác vụ sinh văn bản.

Chú giải của người dịch: Vì sao không thể thử hết mọi câu?

Giả sử bộ từ vựng chỉ có 10 từ và câu đầu ra dài đúng 5 từ. Theo quy tắc nhân trong tổ hợp, số câu có thể có là $10^5 = 100\,000$. Nhưng bộ từ vựng thực tế thường có vài chục nghìn từ, câu dài vài chục từ, nên số câu có thể có lớn hơn cả số nguyên tử trong vũ trụ quan sát được. Vì thế, "tìm kiếm" ở đây là tìm một cách khôn ngoan một câu có điểm cao, chứ không thể liệt kê hết rồi chọn. Ký hiệu $\arg\max_{\mathbf{y}\in\Omega}$ đọc là "giá trị $\mathbf{y}$ trong tập Ω làm cho biểu thức đạt lớn nhất".

5.4.1 Vấn đề độ dài

Nhắc lại từ Mục 5.2.2 rằng xác suất của chuỗi phía đích $\mathbf{y}$ khi biết chuỗi phía nguồn $\mathbf{x}$ có thể viết thành tích các xác suất của từng từ y_i khi biết cả các từ đã sinh trước đó $y_0 \dots y_{i-1}$ và $\mathbf{x}$. Ở đây, ta viết lại Phương trình (5.14) bằng một ký hiệu gọn hơn:

$$\Pr(\mathbf{y}|\mathbf{x}) = \prod_{i=1}^{n} \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}), \tag{5.61}$$

trong đó $\mathbf{y}_{<i}$ ký hiệu chuỗi $y_0 \dots y_{i-1}$. Một cách tương đương, ta có thể biểu diễn bằng log xác suất

$$\log \Pr(\mathbf{y}|\mathbf{x}) \;=\; \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}). \tag{5.62}$$

Dù cách mô hình hóa đơn giản này mang lại những lợi thế rõ ràng cho các ứng dụng NLP thực tế, đáng tiếc là nó gây ra một thiên lệch mạnh về phía các chuỗi đích ngắn hơn. Vì vậy, dùng trực tiếp $\text{Score}(\mathbf{x}, \mathbf{y}) = \Pr(\mathbf{y}|\mathbf{x})$ hoặc $\log \Pr(\mathbf{y}|\mathbf{x})$ làm mục tiêu tìm kiếm là có vấn đề, vì đầu ra rất có thể sẽ là một chuỗi ngắn quá mức (ví dụ, một chuỗi chỉ gồm một hoặc hai từ). Vấn đề này là hệ quả trực tiếp của thiên kiến quy nạp (*inductive bias*) của mô hình. Nhìn từ góc độ học có giám sát, vấn đề này nảy sinh vì kỹ thuật teacher forcing (*teacher forcing*) được dùng trong huấn luyện: mô hình được tiếp xúc với các chuỗi đích chuẩn (*ground-truth*) và bị buộc phải dự đoán đúng chúng. Ngược lại, khi suy luận trên dữ liệu kiểm tra, ta phải khám phá một không gian giả thuyết rất lớn gồm các $\mathbf{y}$ với độ dài khác nhau, và so sánh chúng bằng một hàm tính điểm mà trên thực tế khác với hàm được tối ưu trong huấn luyện.

Chú giải của người dịch: Vì sao xác suất "thích" câu ngắn?

Mỗi xác suất $\Pr(y_i|\mathbf{y}_{<i}, \mathbf{x})$ là một số trong khoảng $(0, 1]$, nên logarit của nó luôn ≤ 0. Mỗi khi thêm một từ, ta lại cộng thêm một số âm (hay nhân thêm một số nhỏ hơn 1). Ví dụ, một câu 2 từ với xác suất mỗi từ là 0.5 có xác suất $0.5 \cdot 0.5 = 0.25$; một câu 6 từ với mỗi từ có xác suất 0.8 (tốt hơn hẳn!) lại chỉ có $0.8^6 \approx 0.26$, gần như bằng câu ngắn. Câu càng dài thì tích càng nhỏ, nên nếu chỉ so sánh xác suất, mô hình có xu hướng chọn câu ngắn cụt lủn dù chất lượng kém hơn.

Vấn đề này có thể được giải quyết bằng một kỹ thuật gọi là **thưởng độ dài** (*length reward*), tức là cộng điểm thưởng cho các chuỗi dài hơn bằng cách thêm một số hạng vào $\text{Score}(\mathbf{x}, \mathbf{y})$ [He và cộng sự, 2016c]. Như đã bàn ở Chương 3, một dạng thưởng độ dài thường dùng là

$$\text{Score}(\mathbf{x}, \mathbf{y}) \;=\; \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot n. \tag{5.63}$$

Ở đây, số hạng thưởng độ dài là độ dài của $\mathbf{y}$ (tức $n = |\mathbf{y}|$), được nhân với trọng số là tham số $\lambda > 0$. Các cải tiến của cách làm này thay n bằng một

độ dài chuỗi được ước lượng nhờ một mô hình dự đoán độ dài. Chẳng hạn, ta có thể chặn trên phần thưởng theo dạng sau [Huang và cộng sự, 2017b; Yang và cộng sự, 2018b]

$$\text{Score}(\mathbf{x}, \mathbf{y}) \;=\; \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot \max(l_p, n), \tag{5.64}$$

trong đó l_p là một độ dài dự đoán, thường được định nghĩa là độ dài của $\mathbf{x}$ nhân với một hệ số, tức là $l_p = \text{scalar}_p \cdot m$.

Nếu thay log xác suất $\log \Pr(\mathbf{y}|\mathbf{x})$ cho bởi Phương trình (5.62) vào Phương trình (5.63), ta thu được

$$\begin{aligned}\text{Score}(\mathbf{x}, \mathbf{y}) &= \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) + \lambda \cdot n \\ &= \sum_{i=1}^{n} \big[\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) + \lambda\big].\end{aligned} \tag{5.65}$$

Như vậy, ta có thể hiểu số hạng thưởng độ dài là một phần thưởng dành cho mỗi từ y_i. Phương pháp này đã được dùng rộng rãi trong các hệ thống **dịch máy thống kê** (*statistical machine translation, SMT*) (**SMT**), trong đó các phần thưởng được xem là đặc trưng của một mô hình log-tuyến tính (*log-linear model*) [Koehn và cộng sự, 2003; Chiang, 2007]. Để tìm giá trị thích hợp cho λ, ta có thể dùng huấn luyện cực tiểu tỉ lệ lỗi (*minimum error rate training*) [Och, 2003] theo truyền thống của SMT, hoặc dùng các phương pháp dựa trên gradient như trong các hệ thống dựa trên mạng nơ-ron [Murray và Chiang, 2018].

Chú giải của người dịch: Thưởng độ dài bằng số cụ thể

Lấy lại ví dụ ở chú giải trên, với log tự nhiên: câu 2 từ có $\log 0.25 \approx -1.39$; câu 6 từ có $\log 0.26 \approx -1.34$. Nếu chọn $\lambda = 0.3$, điểm theo Phương trình (5.63) là $-1.39 + 0.3 \cdot 2 = -0.79$ cho câu ngắn và $-1.34 + 0.3 \cdot 6 = 0.46$ cho câu dài. Nhờ phần thưởng, câu dài và tốt hơn giờ thắng rõ rệt. Phương trình (5.65) cho thấy điều này tương đương với việc cộng thêm λ cho *mỗi* từ được sinh ra.

Cách thứ hai để khiến việc tìm kiếm nghiêng về các chuỗi dài hơn, gọi là **chuẩn hóa độ dài** (*length normalization*), là chia $\log \Pr(\mathbf{y}|\mathbf{x})$ cho một

số hạng hiệu chỉnh độ dài, viết dưới dạng

$$\text{Score}(\mathbf{x}, \mathbf{y}) \;=\; \frac{\log \Pr(\mathbf{y}|\mathbf{x})}{n_{\text{correct}}}. \tag{5.66}$$

Một ví dụ đơn giản của mô hình này là định nghĩa số hạng hiệu chỉnh độ dài chính là độ dài chuỗi [Jean và cộng sự, 2015], như sau

$$\begin{aligned} n_{\text{correct}} &= n \\ &= |\mathbf{y}|. \end{aligned} \tag{5.67}$$

Trong trường hợp này, $\frac{\log \Pr(\mathbf{y}|\mathbf{x})}{n} = \frac{\sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x})}{n}$ có thể được xem là trung bình nhân (*geometric mean*) trên thang log của các xác suất $\{\Pr(y_i|\mathbf{y}_{<i}, \mathbf{x})\}$[4].

Bảng 5.2: Các hàm tính điểm cho thưởng độ dài và chuẩn hóa độ dài. $m = |\mathbf{x}|$, $n = |\mathbf{y}|$ và $l_p = \text{scalar}_p \cdot m$. λ và α là các tham số.

Phương pháp	Dạng của $\text{Score}(\mathbf{x}, \mathbf{y})$
Không thưởng/chuẩn hóa	$\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}\|\mathbf{x})$
Thưởng độ dài	$\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}\|\mathbf{x}) + \lambda \cdot n$
Thưởng độ dài có chặn	$\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}\|\mathbf{x}) + \lambda \cdot \max(l_p, n)$
Chuẩn hóa độ dài (cơ bản)	$\text{Score}(\mathbf{x}, \mathbf{y}) = \frac{\log \Pr(\mathbf{y}\|\mathbf{x})}{n}$
Chuẩn hóa độ dài (GNMT)	$\text{Score}(\mathbf{x}, \mathbf{y}) = \frac{\log \Pr(\mathbf{y}\|\mathbf{x})}{(5+n)^\alpha/(5+1)^\alpha}$

Một ví dụ nổi bật khác là chiến lược chuẩn hóa được dùng trong hệ thống GNMT [Wu và cộng sự, 2016]. Nó dùng một hệ số phạt độ dài được co giãn theo lũy thừa và dịch chuyển:

$$n_{\text{correct}} \;=\; \frac{(5+n)^\alpha}{(5+1)^\alpha}, \tag{5.70}$$

[4] Giả sử $\{a_1, \ldots, a_n\}$ là n biến. Vì

$$\exp\Big(\frac{\sum_{i=1}^{n} \log a_i}{n}\Big) \;=\; \Big(\prod_{i=1}^{n} a_i\Big)^{\frac{1}{n}}, \tag{5.68}$$

nên ta có

$$\frac{\sum_{i=1}^{n} \log a_i}{n} \;=\; \log\Big(\prod_{i=1}^{n} a_i\Big)^{\frac{1}{n}}. \tag{5.69}$$

trong đó số mũ α là một siêu tham số và có thể được xác định bằng thực nghiệm trên một tập tinh chỉnh. Để so sánh các phương pháp khác nhau, Bảng 5.2 liệt kê các hàm tính điểm cho thưởng độ dài và chuẩn hóa độ dài.

Chú giải của người dịch: Chuẩn hóa độ dài: lấy "điểm trung bình mỗi từ"

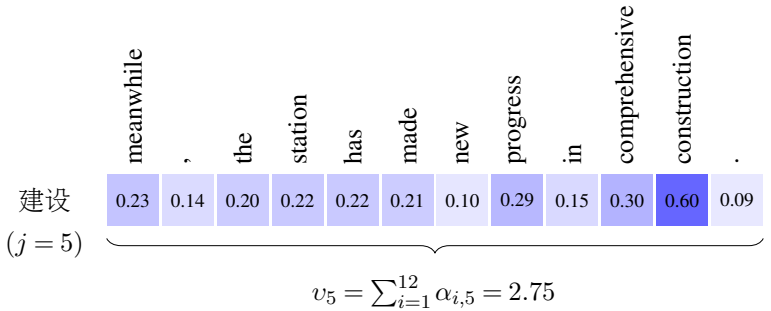
Chia cho n giống như tính điểm trung bình mỗi môn thay vì tổng điểm: một học sinh học nhiều môn hơn không bị thiệt chỉ vì tổng điểm âm cộng dồn nhiều hơn. Với ví dụ trên: câu 2 từ có $-1.39/2 \approx -0.69$, câu 6 từ có $-1.34/6 \approx -0.22$, nên câu dài thắng. Chú thích chân trang kèm Phương trình (5.68–5.69) nhắc lại tính chất logarit $\log a_1 + \cdots + \log a_n = \log(a_1 \cdots a_n)$: trung bình cộng của các log bằng log của trung bình nhân $\sqrt[n]{a_1 \cdots a_n}$. Trong công thức GNMT (5.70), khi $\alpha = 0$ thì $n_{\text{correct}} = 1$ (không chuẩn hóa); khi $\alpha = 1$ thì $n_{\text{correct}} = (5+n)/6$, gần với chia cho độ dài. Vậy α là "núm vặn" điều chỉnh mức độ ưu ái câu dài.

Trong dịch máy, vấn đề độ dài gắn liền với vấn đề **độ phủ** (*coverage*), một chủ đề được nghiên cứu sâu rộng trong SMT. Khi dịch một chuỗi nguồn, điều quan trọng là theo dõi mỗi từ đã được dịch bao nhiêu lần. Ta nói rằng xảy ra **dịch thừa** (*over-translation*) (tạo ra một bản dịch dài sai) nếu các từ nguồn bị dịch nhiều lần, và xảy ra **dịch thiếu** (*under-translation*) (tạo ra một bản dịch bị cụt) nếu các từ nguồn không được dịch đầy đủ. Theo truyền thống, độ phủ của một chuỗi nguồn được mô hình hóa bằng một **vectơ độ phủ** (*coverage vector*) m chiều $\begin{bmatrix} \upsilon_1 & \dots & \upsilon_m \end{bmatrix}$. Mỗi phần tử υ_j cho biết mức độ mà từ nguồn x_j đã được dịch. Trong các hệ thống SMT, υ_j thường là một biến nhị phân: 0 nghĩa là chưa dịch, và 1 nghĩa là đã dịch. Tuy nhiên, các hệ thống NMT tiêu chuẩn không có một cơ chế độ phủ tường minh dạng ký hiệu như vậy. Thay vào đó, chúng dựa vào các trọng số chú ý phân bố trên các từ đích. Vì thế, một cách định lượng độ phủ của từ nguồn x_j là đo xem nó được các từ đích đã sinh ra chú ý tới mạnh đến mức nào. Để làm vậy, ta định nghĩa lại υ_j là một biến liên tục:

$$\upsilon_j = \sum_{i=1}^{n} \alpha_{i,j}. \tag{5.71}$$

Do đó, υ_j có thể được xem là "số lần" x_j được dịch; ví dụ, $\upsilon_j = 0$ nghĩa là x_j hoàn toàn không được dịch, và $\upsilon_j = 1$ nghĩa là x_j chỉ được tính

một lần trong bản dịch. Hãy xét ví dụ trong Hình 5.8. Với từ phía nguồn 建设[5], các trọng số chú ý tương ứng được cho dưới đây.



Ta sẽ nói rằng 建设 được dịch 2.75 lần. Có thể dùng $\{\upsilon_1, \ldots, \upsilon_m\}$ để định nghĩa mức độ chuỗi phía nguồn được phủ trong bản dịch. Một cách đơn giản là xây dựng một điểm độ phủ (*coverage score*) $\text{cp}(\mathbf{x}, \mathbf{y})$ bằng cách kết hợp $\{\upsilon_1, \ldots, \upsilon_m\}$. Chẳng hạn, hệ thống GNMT định nghĩa $\text{cp}(\mathbf{x}, \mathbf{y})$ theo dạng sau

$$\text{cp}(\mathbf{x}, \mathbf{y}) \;=\; \beta \sum_{j=1}^{m} \log(\min(\upsilon_j, 1)), \tag{5.72}$$

trong đó β quyết định trọng số của mô hình độ phủ. Ý tưởng trực giác là: nếu $\upsilon_j \geq 1$, từ x_j được xem là đã được dịch đầy đủ; nếu $\upsilon_j < 1$, x_j chưa được dịch đủ. Kết quả là $\text{cp}(\mathbf{x}, \mathbf{y})$ phạt những giả thuyết có các từ nguồn bị dịch thiếu. Một phiên bản cải tiến của hình phạt này được Li và cộng sự [2018] đề xuất:

$$\text{cp}(\mathbf{x}, \mathbf{y}) \;=\; \beta \sum_{j=1}^{m} \log(\max(\upsilon_j, \gamma)), \tag{5.73}$$

trong đó γ là siêu tham số cắt ngưỡng, cho phép một mức dung sai đối với việc dịch thiếu. Một dạng tương tự được đề xuất trong [Chorowski và Jaitly, 2017]

$$\text{cp}(\mathbf{x}, \mathbf{y}) \;=\; \beta \sum_{j=1}^{m} \mathbb{1}(\upsilon_j > \gamma). \tag{5.74}$$

[5] **[ND]** Tiếng Trung, đọc là *jiànshè*, nghĩa là "xây dựng".

Nó đếm số lần $v_j > \gamma$.

Chú giải của người dịch: Đọc hiểu điểm độ phủ

Trong hình trên, mỗi ô là trọng số chú ý $\alpha_{i,5}$ mà từ đích thứ i dành cho từ nguồn 建设; cộng cả 12 ô được $v_5 = 2.75$ (từ "construction" — "xây dựng" — chú ý mạnh nhất, 0.60). Ở Phương trình (5.72): nếu $v_j \geq 1$ thì $\min(v_j, 1) = 1$ và $\log 1 = 0$, tức không bị phạt; nếu, chẳng hạn, $v_j = 0.1$ thì $\log 0.1 \approx -2.3$, một mức phạt âm khá lớn. Vì vậy, từ nào "bị bỏ quên" sẽ kéo điểm xuống. Ở Phương trình (5.74), $\mathbb{1}(\cdot)$ là *hàm chỉ thị*: bằng 1 nếu điều kiện trong ngoặc đúng, bằng 0 nếu sai; tổng của nó đơn giản là đếm số từ nguồn có $v_j > \gamma$.

$\text{cp}(\mathbf{x}, \mathbf{y})$ có thể dễ dàng được đưa vào quá trình tìm kiếm bằng cách cộng nó vào $\text{Score}(\mathbf{x}, \mathbf{y})$. Ví dụ, hàm tính điểm kiểu GNMT được định nghĩa là

$$\text{Score}(\mathbf{x}, \mathbf{y}) = \frac{\log \Pr(\mathbf{y}|\mathbf{x})}{(5+n)^\alpha/(5+1)^\alpha} + \text{cp}(\mathbf{x}, \mathbf{y}). \tag{5.75}$$

Trong thực tế, sửa đổi $\text{Score}(\mathbf{x}, \mathbf{y})$ không phải là cách duy nhất để giảm nhẹ các vấn đề độ dài và độ phủ trong quá trình tìm kiếm. Một chiến lược khác là thay đổi kiến trúc để mô hình hóa tường minh các hiện tượng này [Tu và cộng sự, 2016; Mi và cộng sự, 2016a; Sankaran và cộng sự, 2016; See và cộng sự, 2017; Malaviya và cộng sự, 2018]. Lưu ý rằng trong một số trường hợp, độ dài của chuỗi đích được chỉ định hoặc dự đoán trước. Khi đó, ta có thể dùng các mô hình phi tự hồi quy (*non-autoregressive model*) [Gu và cộng sự, 2018] hoặc thiết kế các hệ thống sinh văn bản có kiểm soát độ dài cho những ứng dụng cụ thể [Rush và cộng sự, 2015; Kikuchi và cộng sự, 2016].

5.4.2 Cắt tỉa và tìm kiếm chùm

Có nhiều cách để định nghĩa một không gian tìm kiếm. Là một khái niệm chung trong khoa học máy tính, không gian tìm kiếm thường được định nghĩa là tập hợp mọi lời giải có thể có của một bài toán. Với các bài toán chuỗi-sang-chuỗi, ta có thể xem một giả thuyết là một ánh xạ từ một chuỗi phía nguồn $\mathbf{x}$ sang một chuỗi phía đích $\mathbf{y}$, và không gian tìm kiếm là tập

hợp tất cả những giả thuyết như vậy[6].

Ta có thể hiện thực một thuật toán tìm kiếm bằng cách tổ chức các giả thuyết. Nhắc lại rằng trong Phương trình (5.61) và (5.62), ta tính xác suất của **y** khi biết **x** bằng một phép phân tích từ trái sang phải. Một hệ thống tìm kiếm điển hình duy trì một tập các giả thuyết (hoặc giả thuyết bộ phận (*partial hypothesis*)) và mở rộng chúng dần dần[7]. Thủ tục tìm kiếm bắt đầu với một tập giả thuyết ban đầu Z_0 chứa một giả thuyết duy nhất z_0, có phía đích là $y_0 = \langle \text{SOS} \rangle$, ký hiệu bắt đầu của mọi chuỗi đích. Sau đó ta mở rộng tập giả thuyết này qua một loạt bước tìm kiếm. Gọi $Z_0 \dots Z_{n_{\max}}$ là dãy các tập giả thuyết, trong đó $n_{\max}$ là số bước tìm kiếm tối đa. Ở bước i, ta mở rộng mỗi giả thuyết bằng cách nối thêm một từ mới v_k lấy từ bộ từ vựng V_y. Gọi $z.src$ và $z.tgt$ lần lượt là phía nguồn và phía đích của giả thuyết z. Hiển nhiên $z.src = \mathbf{x}$ với mọi z. Cho một giả thuyết $z_{\text{cur}} \in Z_{i-1}$, ta có thể mở rộng nó để sinh ra $|V_y|$ giả thuyết mới, ký hiệu là $\{z^1_{\text{next}}, \dots, z^{|V_y|}_{\text{next}}\}$:

$$\begin{aligned} \{z^1_{\text{next}}, \dots, z^{|V_y|}_{\text{next}}\} &= \text{Extend}(z_{\text{cur}}, V_y) \\ &= \bigcup_{v_k \in V_y} \text{Extend}(z_{\text{cur}}, v_k). \end{aligned} \tag{5.76}$$

Ở đây, $\text{Extend}(z_{\text{cur}}, v_k)$ là hàm mở rộng giả thuyết đầu vào z_{cur} với một từ $v_k \in V_y$. Phía đích của giả thuyết thu được là phép nối $z_{\text{cur}}.tgt$ với v_k, viết là[8]:

$$z^k_{\text{next}}.tgt \;=\; z_{\text{cur}}.tgt \circ v_k. \tag{5.77}$$

Các giả thuyết mới $\{z^1_{\text{next}}, \dots, z^{|V_y|}_{\text{next}}\}$ sau đó được thêm vào Z_i. Hình 5.9 minh họa vài bước đầu của quá trình mở rộng này. Mọi giả thuyết có thể được tổ chức thành một cấu trúc cây, trong đó Z_i tương ứng với tập các nút ở mức i của cây tìm kiếm. Do đó ta có:

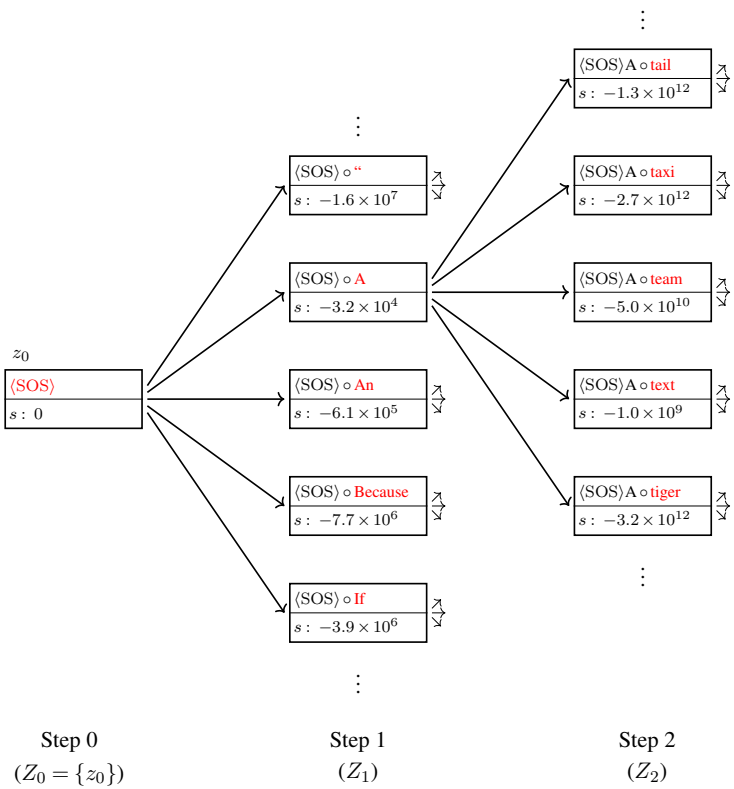
$$|Z_i| \;=\; |V_y| \cdot |Z_{i-1}|. \tag{5.78}$$

[6]Ở đây ta dùng $(\mathbf{x}, \mathbf{y})$ để ký hiệu một giả thuyết. Khi có nhiều ánh xạ từ **x** sang **y**, một giả thuyết có thể được biểu diễn là $(\mathbf{x}, \mathbf{y}, d)$, trong đó d ký hiệu ánh xạ cụ thể. Ví dụ, nếu ta biến đổi **x** thành **y** bằng một văn phạm đồng bộ, có thể có nhiều cách dẫn xuất bằng các luật văn phạm để đạt được điều đó.

[7]Một giả thuyết được xem là bộ phận khi chuỗi phía đích tương ứng không kết thúc bằng $\langle \text{EOS} \rangle$, tức là một chuỗi chưa hoàn chỉnh. Trong mục này, chúng tôi dùng các thuật ngữ *giả thuyết* và *giả thuyết bộ phận* thay thế cho nhau vì dạng của chúng giống hệt nhau.

[8]Ta dùng $a \circ b$ để ký hiệu phép nối hai xâu a và b.

Nói cách khác, kích thước của Z_i tăng theo hàm mũ với số bước, cho ra $|Z_i| = |V_y|^i$.



Hình 5.9: Minh họa việc mở rộng giả thuyết trong 3 bước đầu tiên. Mỗi giả thuyết (bộ phận) được biểu diễn bằng một hộp, trong đó ta ghi chuỗi phía đích tương ứng và điểm của mô hình. Mỗi bước tìm kiếm gắn với một tập giả thuyết Z_i. Ta bắt đầu với giả thuyết $z_0 \in Z_0$ biểu thị ký hiệu bắt đầu ⟨SOS⟩. Ở bước i, ta mở rộng mọi giả thuyết trong Z_{i-1} bằng cách thử nối thêm mọi từ trong bộ từ vựng V_y (xem các từ màu đỏ). Thao tác này cho ra $|V_y| \cdot |Z_{i-1}|$ giả thuyết, tạo thành tập giả thuyết Z_i. Thủ tục mở rộng giả thuyết này thể hiện một thuật toán tìm kiếm theo chiều rộng: ta tạo tất cả các nút (hay trạng thái tìm kiếm) ở độ sâu $i-1$ trước khi chuyển sang độ sâu i. Một cấu trúc cây được tạo ra cùng với thủ tục này, và từ một nút lá của cây có thể lần ngược đường tìm kiếm về tới nút gốc.

Mỗi giả thuyết z gắn với một log xác suất $\log \Pr(z.tgt|z.src)$. Theo

dạng của Phương trình (5.62), xác suất này có thể được tính theo kiểu đệ quy:

$$\log \Pr(z^k_{\text{next}}.tgt|z^k_{\text{next}}.src) = \log \Pr(z_{\text{cur}}.tgt|z_{\text{cur}}.src) + \log \Pr(v_k|z_{\text{cur}}.tgt, z_{\text{cur}}.src). \quad (5.79)$$

Ví dụ, giả sử $z^k_{\text{next}}.tgt = y_0 \dots y_{i+1}$. Dạng của Phương trình (5.79) trở nên rõ ràng từ biểu thức sau

$$\begin{aligned}\underbrace{\log \Pr(y_0 \dots y_{i+1}|\mathbf{x})}_{\log \Pr(z^k_{\text{next}}.tgt|z^k_{\text{next}}.src)} &= \underbrace{\log \Pr(y_0 \dots y_i|\mathbf{x})}_{\log \Pr(z_{\text{cur}}.tgt|z_{\text{cur}}.src)} + \underbrace{\log \Pr(y_{i+1}|y_0 \dots y_i, \mathbf{x})}_{\log \Pr(v_k|z_{\text{cur}}.tgt, z_{\text{cur}}.src)} \\ &= \sum_{k=1}^{i} \log \Pr(y_k|\mathbf{y}_{<k}, \mathbf{x}) + \log \Pr(y_{i+1}|y_0 \dots y_i, \mathbf{x}) \\ &= \sum_{k=1}^{i+1} \log \Pr(y_k|\mathbf{y}_{<k}, \mathbf{x}). \quad (5.80)\end{aligned}$$

Có xác suất này, ta có thể tính $z.score = \text{Score}(z.src, z.tgt)$ như ở Mục 5.4.1. Điểm này cho phép ta xếp hạng và so sánh các giả thuyết khác nhau. Nếu một giả thuyết kết thúc bằng ký hiệu $\langle\text{EOS}\rangle$, nó được xem là hoàn chỉnh và không được mở rộng nữa. Khi đã hoàn chỉnh, nó được đẩy vào một đống cực đại (*max-heap*)[9]. Sau đó ta có thể lấy ra các giả thuyết có điểm cao nhất từ đống này. Thủ tục tìm kiếm thường dừng khi đã tìm được một số lượng giả thuyết hoàn chỉnh định trước, chẳng hạn khi đống đã đầy. Thuật toán tìm kiếm thu được được trình bày dưới đây.

Chú giải của người dịch: Đọc thuật toán và cấu trúc đống

Một *thuật toán* là dãy các bước rõ ràng mà máy tính thực hiện. Ở đây, ba vòng lặp **for** lồng nhau có nghĩa là: với mỗi bước, với mỗi giả thuyết hiện có, thử nối thêm *từng* từ trong bộ từ vựng. Ký hiệu $z.tgt$, $z.score$ đọc là "trường tgt (hay $score$) của đối tượng z", giống như "họ tên" và "điểm" là các trường trong hồ sơ của một học sinh. *Đống cực đại* (max-heap) là một cấu trúc dữ liệu luôn giữ phần tử lớn nhất ở "đỉnh", nhờ đó lấy ra phần tử có điểm cao nhất rất nhanh — giống như một bảng xếp hạng luôn được cập nhật. Phương trình (5.79) cho thấy điểm log của câu mới bằng điểm log của câu cũ cộng

[9]Cho một đống cực đại a, ta dùng a.Pop() để ký hiệu hàm lấy ra và trả về phần tử ở đỉnh của a, và a.PopAll() cho hàm lấy ra tất cả các phần tử.

Thuật toán: Một thuật toán tìm kiếm theo chiều rộng đơn giản

SIMPLESEARCH($\mathbf{x}$)
// Tìm giả thuyết tốt nhất khi biết chuỗi phía nguồn $\mathbf{x}$

Tạo một Heap (đống) có $size_{\text{heap}}$ phần tử
$Z_0 = \{z_0\}$ với $z_0.src = \mathbf{x}$ và $z_0.tgt = y_0$
for mỗi bước $i = 1$ đến n_{max} **do**
    for mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$ **do**
        for mỗi từ $v_k \in V_y$ **do**
            $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, \mathbf{x})$
            if $z_{\text{next}}.tgt$ kết thúc bằng $\langle\text{EOS}\rangle$ **then**
                Thêm z_{next} vào Heap
            else
                Thêm z_{next} vào Z_i
            end if
            if Heap đã đầy và/hoặc thỏa các tiêu chí dừng khác **then**
                Thoát khỏi tất cả các vòng lặp
            end if
        end for
    end for
end for
return Heap.Pop()

Extend(z_{cur}, v_k, src)
// Tạo một giả thuyết mới bằng cách nối một từ mới v_k *vào phía đích của* z_{cur}

Tạo một giả thuyết mới z_{next}
$z_{\text{next}}.src = src$
$z_{\text{next}}.tgt = z_{\text{cur}}.tgt \circ v_k$
$z_{\text{next}}.prob = z_{\text{cur}}.prob + \log \Pr(v_k | z_{\text{cur}}.tgt, z_{\text{cur}}.src)$ ▷ xem Phương trình (5.79)
$z_{\text{next}}.score = \text{Score}(z_{\text{next}}.src, z_{\text{next}}.tgt)$ ▷ xem Mục 5.4.1
return z_{next}

thêm log xác suất của từ mới, nên không cần tính lại từ đầu.

Nếu đống giả thuyết có sức chứa vô hạn ($size_{\text{heap}} = \infty$), thuật toán này thực hiện một phép tìm kiếm vét cạn trên không gian của mọi giả thuyết có độ dài tối đa $n_{\max}$, tạo ra nhiều nhất $1 + |V_y| + |V_y|^2 + \cdots + |V_y|^{n_{\max}} = \frac{|V_y|^{n_{\max}+1}-1}{|V_y|-1}$ giả thuyết hoàn chỉnh. Không gian tìm kiếm này cực kỳ lớn và trên thực tế không thể tính toán nổi[10]. Vì vậy, các hệ thống thực tế dựa rất nhiều vào các kỹ thuật cắt tỉa để làm cho quá trình tìm kiếm khả thi. Trong phần còn lại của mục này, chúng tôi sẽ giới thiệu hai thuật toán tìm kiếm phổ biến sử dụng cắt tỉa để giải mã hiệu quả.

Chú giải của người dịch: Tổng cấp số nhân

Biểu thức $1 + |V_y| + |V_y|^2 + \cdots + |V_y|^{n_{\max}}$ chính là tổng của một cấp số nhân có công bội $q = |V_y|$ (kiến thức lớp 11): $1 + q + \cdots + q^N = \frac{q^{N+1}-1}{q-1}$. Với $q = 20{,}000$ và $N = 20$, số hạng trội là $q^{20} = (2 \times 10^4)^{20} \approx 1.05 \times 10^{86}$, lớn hơn cả ước tính số nguyên tử trong vũ trụ quan sát được (khoảng 10^{80}).

1. Tìm kiếm tham lam

Chiến lược tham lam (*greedy strategy*) là một khái niệm nền tảng trong thiết kế thuật toán. Nó dựa trên heuristic rằng có thể thu được lời giải tối ưu toàn cục bằng cách đưa ra các quyết định tối ưu cục bộ. Dù chiến lược này chỉ bảo đảm tính tối ưu cục bộ, nó vẫn đủ dùng cho nhiều ứng dụng thực tế, và độ phức tạp tính toán thấp là một lợi thế rõ ràng của nó.

Áp dụng chiến lược tham lam cho bài toán tìm kiếm của chúng ta rất đơn giản. Ở mỗi bước mở rộng i, ta chỉ xét một giả thuyết tốt nhất cho đến bước này. Chính xác hơn, với mọi tập Z_i, ta chỉ giữ lại giả thuyết có điểm mô hình cao nhất và loại bỏ phần còn lại. Giả thuyết tốt nhất ở mỗi bước của tìm kiếm tham lam được cho bởi

$$z_{\text{best}} = \underset{z_{\text{next}} \in \text{Extend}(Z_{i-1}, V_y)}{\arg\max} z_{\text{next}}.score. \tag{5.81}$$

Ở đây, hàm $\text{Extend}(Z_{i-1}, V_y)$ có cùng ý nghĩa như trong Phương

[10] Ví dụ, xét kích thước bộ từ vựng là 20,000 ($|V_y| = 20{,}000$) và giới hạn độ dài là 20 ($n_{\max} = 20$). Tổng số giả thuyết $\frac{|V_y|^{n_{\max}+1}-1}{|V_y|-1}$ sẽ xấp xỉ 1.05×10^{86}.

trình (5.76), nhưng tác động trên một tập giả thuyết, tức là

$$\text{Extend}(Z_{i-1}, V_y) \;=\; \bigcup_{z \in Z_{i-1}} \text{Extend}(z, V_y). \tag{5.82}$$

Khi đó, Z_i được định nghĩa là

$$Z_i \;=\; \{z_{\text{best}}\}. \tag{5.83}$$

Dưới đây là một thuật toán tìm kiếm tham lam cho các bài toán chuỗi-sang-chuỗi.

Thuật toán: Một thuật toán tìm kiếm tham lam

GreedySearch($\mathbf{x}$)
// Tìm giả thuyết "tốt nhất" theo cách tham lam

Tạo một giả thuyết z_{best}
$Z_0 = \{z_0\}$ với $z_0.src = \mathbf{x}$ và $z_0.tgt = y_0$
for mỗi bước $i = 1$ đến n_{max} **do**
    $z_{\text{best}}.score = -\infty$
    for mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$ **do**
        for mỗi từ $v_k \in V_y$ **do**
            $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, \mathbf{x})$
            if $z_{\text{best}}.score < z_{\text{next}}.score$ **then**
                $z_{\text{best}} = z_{\text{next}}$
            end if
        end for
    end for
    if $z_{\text{best}}.tgt$ kết thúc bằng ⟨EOS⟩ và/hoặc thỏa các tiêu chí dừng khác **then**
        Thoát khỏi vòng lặp
    end if
    $Z_i = \{z_{\text{best}}\}$
end for
return z_{best}

Ở mỗi bước tìm kiếm, ta chỉ có một giả thuyết đang hoạt động cần mở rộng (tức $|Z_{i-1}| = 1$). Do đó, ta sinh ra $|V_y|$ phần mở rộng, rồi chọn phần mở rộng tốt nhất cho bước tiếp theo. Tổng số lần gọi Extend(z_{cur}, v_k) là $|V_y| \cdot n_{\text{max}}$. Giả sử Extend($z_{\text{cur}}, v_k$) chạy trong thời gian hằng số, thì độ phức tạp thời gian (*time complexity*) tổng thể của thuật toán tăng tuyến tính theo $|V_y|$ và n_{max}.

Chú giải của người dịch: Tham lam: chọn tốt nhất trước mắt

Hãy hình dung leo núi trong sương mù: ở mỗi bước, bạn chỉ nhìn thấy xung quanh và luôn bước theo hướng dốc lên nhất. Cách này nhanh, nhưng có thể đưa bạn lên một đỉnh đồi nhỏ thay vì đỉnh núi cao nhất. Ví dụ: bước 1 chọn "A" (xác suất 0.5) thay vì "The" (0.4); nhưng nếu sau "A" từ tốt nhất chỉ có xác suất 0.3 còn sau "The" có từ với xác suất 0.9, thì cặp "The …" có xác suất $0.4 \cdot 0.9 = 0.36$, lớn hơn $0.5 \cdot 0.3 = 0.15$. Tìm kiếm tham lam đã bỏ lỡ lời giải tốt hơn. Bù lại, với $|V_y| = 20{,}000$ và $n_{\max} = 20$, nó chỉ cần 400,000 lần gọi Extend thay vì khoảng 10^{86}.

2. Tìm kiếm chùm

Tìm kiếm chùm (*beam search*) là một sự mở rộng tự nhiên của thuật toán tìm kiếm tham lam ở trên. Nó cũng dựa trên heuristic tham lam, và vì thế là một loại thuật toán tham lam. Ý tưởng cốt lõi của tìm kiếm chùm là giữ lại một số lượng định trước các giả thuyết hứa hẹn nhất ở mỗi bước, thay vì chỉ một giả thuyết tốt nhất. Một **chùm** (*beam*) là một cấu trúc dữ liệu lưu các giả thuyết tốt nhất mà ta đã sinh ra cho đến lúc đó. Số giả thuyết trong một chùm là một tham số định trước, gọi là **độ rộng chùm** (*beam width*) hay **kích thước chùm** (*beam size*). Ở đây ta có thể đơn giản xem Z_i là một chùm, viết là

$$Z_i \;=\; \{z^1_{\text{best}}, \ldots, z^{size_{\text{beam}}}_{\text{best}}\}, \tag{5.84}$$

trong đó $size_{\text{beam}}$ là kích thước chùm. z^1_{best} là giả thuyết tốt nhất trong tập $\text{Extend}(Z_{i-1}, V_y)$ (xem Phương trình (5.81)), z^2_{best} là giả thuyết tốt thứ hai trong $\text{Extend}(Z_{i-1}, V_y)$, và cứ thế tiếp tục.

Mã giả sau đây mô tả một thuật toán tìm kiếm chùm cho các bài toán chuỗi-sang-chuỗi.

Hàm $\text{UpdateBeam}(\text{Beam}, z_{\text{next}})$ có thể được xem là một trường hợp của cắt tỉa theo biểu đồ tần suất (*histogram pruning*). Lưu ý rằng khung tổng quát này cho ta một cách đơn giản để hiện thực các phương pháp cắt tỉa khác, và ta có thể sửa $\text{UpdateBeam}(\text{Beam}, z_{\text{next}})$ khi cần. Ví dụ, một phương pháp khác gọi là **cắt tỉa theo ngưỡng** (*threshold pruning*) giữ lại những giả thuyết có điểm mô hình chênh lệch so với giả thuyết tốt nhất trong Beam ít hơn một ngưỡng định trước; chẳng hạn, ta loại bỏ z_{next} trong

Thuật toán: Một thuật toán tìm kiếm chùm

BeamSearch($\mathbf{x}$)
// Tìm giả thuyết "tốt nhất" bằng cách xét một số ứng viên tốt nhất ở mỗi bước

Tạo một Heap (đống) có $size_{\text{heap}}$ phần tử
$Z_0 = \{z_0\}$ với $z_0.src = \mathbf{x}$ và $z_0.tgt = y_0$
for mỗi bước $i = 1$ đến n_{max} **do**
    Tạo một đống Beam (chùm) có $size_{\text{beam}}$ phần tử
    for mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$ **do**
        for mỗi từ $v_k \in V_y$ **do**
            $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, \mathbf{x})$
            if $z_{\text{next}}.tgt$ kết thúc bằng $\langle\text{EOS}\rangle$ **then**
                Thêm z_{next} vào Heap
            else
                UpdateBeam(Beam, z_{next})
            end if
            if Heap đã đầy và/hoặc thỏa các tiêu chí dừng khác **then**
                Thoát khỏi tất cả các vòng lặp
            end if
        end for
    end for
    $Z_i = \text{Beam.PopAll}()$
end for
return Heap.Pop()

UpdateBeam(Beam, z_{next})
// Cập nhật Beam *với một giả thuyết mới sinh* z_{next}

Thêm z_{next} vào Beam [a]

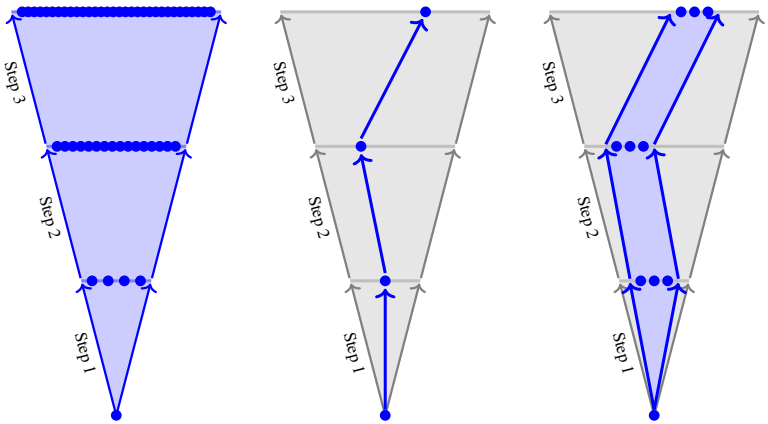
[a] Beam là một đống cực đại có giới hạn, duy trì $size_{\text{beam}}$ giả thuyết đứng đầu. Vì vậy, nếu $z_{\text{next}}.score$ thấp hơn điểm thấp nhất trong đống thì nó bị loại bỏ. Nói cách khác, Beam chỉ lưu $size_{\text{beam}}$ giả thuyết tốt nhất và bỏ qua phần còn lại.

UpdateBeam(Beam, z_{next}) nếu

$$z_{\text{next}}.score \ < \ z_{\text{best}}.score - \theta_{\text{beam}}, \tag{5.85}$$

trong đó z_{best} là giả thuyết tốt nhất trong Beam. Ngoài ra, ta có thể dùng phương pháp ngưỡng tương đối [Freitag và Al-Onaizan, 2017], cho bởi

$$z_{\text{next}}.score \ < \ z_{\text{best}}.score \cdot \theta_{\text{beam}}. \tag{5.86}$$



(a) Exhausted Search (b) (1-best) Greedy Search (c) Beam Search

Hình 5.10: So sánh tìm kiếm vét cạn, tìm kiếm tham lam (1-best) và tìm kiếm chùm. Các quả cầu biểu diễn trạng thái tìm kiếm hay giả thuyết bộ phận. Tìm kiếm vét cạn khám phá mọi trạng thái trong không gian tìm kiếm. Ngược lại, tìm kiếm tham lam chỉ giữ đường đi tốt nhất (1-best) gồm các trạng thái tìm kiếm và cắt bỏ phần còn lại. Tìm kiếm chùm là sự dung hòa giữa hai cách trên và giữ các trạng thái tìm kiếm hứa hẹn nhất trong một chùm ở mỗi bước. (Nhãn trong hình: *Step* = bước; (a) tìm kiếm vét cạn; (b) tìm kiếm tham lam (1-best); (c) tìm kiếm chùm.)

Hình 5.10 so sánh tìm kiếm vét cạn, tìm kiếm tham lam (1-best) và tìm kiếm chùm. Ở một thái cực, lời giải tối ưu được bảo đảm nhưng phải thăm một số trạng thái tìm kiếm lớn theo hàm mũ. Ở thái cực kia, chỉ một số tối thiểu trạng thái được thăm, nhưng lời giải là dưới tối ưu (*sub-optimal*). Còn tìm kiếm chùm dung hòa giữa hai phương pháp. Kích thước chùm lớn hơn nghĩa là tốn công tìm kiếm hơn và có nhiều cơ hội tìm được giả thuyết

tốt hơn, trong khi kích thước chùm nhỏ hơn nghĩa là tìm kiếm nhanh hơn và rủi ro bỏ lỡ lời giải tối ưu cao hơn. Cũng có thể dùng kích thước chùm thay đổi để dung hòa tốt hơn trong quá trình tìm kiếm [Buckman và cộng sự, 2016; Post và Vilar, 2018; Kulikov và cộng sự, 2019].

Chú giải của người dịch: Tìm kiếm chùm qua một ví dụ nhỏ

Giả sử kích thước chùm là 2. Bước 1, trong mọi từ đầu câu, ta giữ 2 từ có điểm cao nhất, ví dụ "A" và "The". Bước 2, nối thêm mọi từ vào *cả hai* (tạo $2 \cdot |V_y|$ ứng viên), rồi lại chỉ giữ 2 ứng viên tốt nhất, chẳng hạn "The cat" và "A cat". Cứ thế tiếp tục. Nhờ giữ nhiều hơn một ứng viên, ví dụ ở chú giải về tìm kiếm tham lam sẽ không bỏ lỡ cặp "The …". Mỗi bước chỉ tốn khoảng $size_{\text{beam}} \cdot |V_y|$ phép mở rộng, nên tổng chi phí vẫn tăng tuyến tính theo độ dài câu. Cắt tỉa theo ngưỡng (5.85) thì khác: thay vì giữ cố định k ứng viên, ta giữ mọi ứng viên có điểm không kém ứng viên tốt nhất quá θ_{beam}.

Một vấn đề quan trọng liên quan đến các thuật toán tìm kiếm này là vấn đề lỗi tìm kiếm (*search error*). Nói chung, lỗi tìm kiếm có thể được định nghĩa theo nhiều cách khác nhau. Ở đây ta nói rằng một lỗi tìm kiếm xảy ra nếu kết quả tìm kiếm không giống với kết quả của tìm kiếm vét cạn. Theo trực giác, giảm thiểu lỗi tìm kiếm sẽ cho kết quả chất lượng cao hơn. Vì vậy, ta có thể kỳ vọng rằng tăng kích thước chùm sẽ luôn cải thiện các chuỗi được sinh ra. Tuy nhiên, giả định này không phải lúc nào cũng đúng. Chẳng hạn, trong các hệ thống dịch máy nơ-ron, giải mã với kích thước chùm quá lớn có thể làm giảm chất lượng bản dịch [Koehn và Knowles, 2017]. Hiện tượng phản trực giác này đã thúc đẩy các nghiên cứu về sự suy giảm hiệu năng gắn với kích thước chùm lớn [Ott và cộng sự, 2018b; Yang và cộng sự, 2018b; Stahlberg và Byrne, 2019].

3. Tiêu chí dừng

Mặc dù độ phức tạp thời gian của các thuật toán đã bàn ở trên bị chặn bởi số bước tìm kiếm tối đa (tức $n_{\max}$), điều quan trọng là thiết kế các tiêu chí dừng (*stopping criteria*) hiệu quả hơn để kết thúc việc tìm kiếm càng sớm càng tốt, đặc biệt với các ứng dụng nhạy cảm với độ trễ (*latency*). Điều này thường đòi hỏi các heuristic để thiết kế thêm những tiêu chí giúp dừng thủ tục tìm kiếm ở một điểm tối ưu. Các tiêu chí dừng thường gặp gồm:

- Nếu đã tạo ra một số lượng giả thuyết hoàn chỉnh cho trước thì ta dừng tìm kiếm. Ví dụ, trong thuật toán tìm kiếm chùm mô tả ở tiểu mục này, chương trình tìm kiếm kết thúc khi ta có $size_{\text{heap}}$ giả thuyết hoàn chỉnh. Một cách khác để hiện thực ý tưởng này là thu hẹp chùm khi số giả thuyết hoàn chỉnh tăng lên. Trong hệ thống của Bahdanau và cộng sự [2014], mỗi khi một giả thuyết hoàn chỉnh mới được tạo ra, kích thước chùm giảm đi 1. Do đó, chương trình tìm kiếm sẽ kết thúc nếu kích thước chùm giảm về 0.

- Nếu mọi giả thuyết ở bước i đều có điểm thấp hơn điểm của giả thuyết hoàn chỉnh tốt nhất trong Heap một khoảng nào đó thì ta dừng tìm kiếm. Giả sử $z_{\text{bestinall}}$ là giả thuyết tốt nhất mà ta đã sinh ra cho đến lúc này (tức $z_{\text{bestinall}} = \text{Heap.Pop}()$). Nếu mọi giả thuyết z_{next} ở bước i thỏa mãn

$$z_{\text{bestinall}}.score - z_{\text{next}}.score \geq \theta_{\text{all}}, \tag{5.87}$$

  thì quá trình tìm kiếm kết thúc. Ở đây, θ_{all} là một siêu tham số thường được xác định bằng thực nghiệm. Một lựa chọn đơn giản là $\theta_{\text{all}} = 0$, được dùng trong một số framework chuỗi-sang-chuỗi phổ biến [Ott và cộng sự, 2019]. Trong một số điều kiện nhất định, chiến lược dừng sớm (*early stopping*) như vậy thậm chí có thể bảo đảm tính tối ưu của việc tìm kiếm [Huang và cộng sự, 2017b; Yang và cộng sự, 2018b].

- Nếu mọi giả thuyết ở bước i đều có điểm thấp hơn điểm của giả thuyết hoàn chỉnh cuối cùng trong Heap một khoảng nào đó thì ta dừng tìm kiếm. Đây là một điều kiện yếu cho việc dừng sớm.

- Nếu các giả thuyết xếp hạng đầu ở bước i đều là giả thuyết hoàn chỉnh thì ta dừng tìm kiếm. Đây là một phiên bản dừng sớm quyết

liệt hơn. Ví dụ, trong hệ thống của Klein và cộng sự [2017], chương trình tìm kiếm kết thúc ở bước i nếu giả thuyết đứng đầu (top-1) là một giả thuyết hoàn chỉnh.

- Nếu ngân sách tính toán (ví dụ, số phép toán dấu phẩy động tối đa hoặc một giới hạn chặt về thời gian thực) đã cạn thì việc tìm kiếm kết thúc. Trong các ứng dụng mà tài nguyên tính toán rất hạn chế và độ trễ là mối quan tâm hàng đầu, tiêu chí này trở nên thiết yếu.

Đôi khi thuật toán tìm kiếm có thể không tìm được giả thuyết hoàn chỉnh nào trước khi chạm giới hạn độ dài $n_{\max}$. Trên thực tế, một chiến lược dự phòng phổ biến trong những trường hợp này là hoàn chỉnh giả thuyết bộ phận có điểm cao nhất bằng cách nối token $\langle \text{EOS} \rangle$ vào cuối của nó.

Lưu ý rằng việc chọn tiêu chí dừng phù hợp phản ánh sự thỏa hiệp giữa hiệu quả tính toán và độ chính xác dự đoán khi suy luận (thường gọi là đánh đổi tốc độ–độ chính xác (*speed-accuracy trade-off*)). Dù dành thêm thời gian tính toán không bảo đảm đầu ra tốt hơn cho một hệ thống chuỗi-sang-chuỗi, việc hiểu kết quả heuristic của ta xấp xỉ lời giải tối ưu sát đến mức nào, bằng cách khám phá một vùng rộng hơn của không gian tìm kiếm, vẫn là điều quan trọng. Các thuật toán tìm kiếm chính xác (*exact search*) bảo đảm được tính tối ưu này sẽ được bàn thêm ở Mục 5.4.4.

Chú giải của người dịch: Vì sao $\theta_{\text{all}} = 0$ có thể đủ an toàn?

Nếu điểm chỉ là log xác suất (không có thưởng độ dài), thì nối thêm một từ luôn cộng thêm một số ≤ 0, nên điểm của một giả thuyết chỉ có thể *giảm* khi nó dài thêm. Do đó, nếu mọi giả thuyết đang dở dang đều đã có điểm không cao hơn giả thuyết hoàn chỉnh tốt nhất, thì không giả thuyết nào trong số chúng có thể vượt lên được nữa — dừng lại là không mất gì. Đây là ý tưởng đằng sau nhận xét rằng dừng sớm có thể bảo đảm tính tối ưu trong một số điều kiện.

5.4.3 Tìm kiếm trực tuyến

Cho đến giờ, trong phần thảo luận chung về các bài toán chuỗi-sang-chuỗi, ta đã giả định rằng mọi từ phía nguồn được quan sát dưới dạng một chuỗi hoàn chỉnh. Tuy nhiên, trong một số ứng dụng thực tế, đầu vào được nhận lần lượt, và ta muốn đưa ra dự đoán dựa trên các quan sát bộ phận. Một ví

dụ là nhận dạng tiếng nói tự động trực tuyến (*online*), trong đó hệ thống liên tục nhận các tín hiệu âm thanh mới và đồng thời xuất ra các đơn vị phiên âm tương ứng.

Theo trực giác, ta có thể xem việc sinh từ đích thứ i là bài toán ánh xạ một tiền tố (*prefix*) của chuỗi phía nguồn sang một từ đích. Ta có thể phát biểu điều này bằng cách đưa vào hàm $g(i)$, biểu thị độ dài tối đa của tiền tố của $\mathbf{x}$ mà ta dùng khi sinh y_i. Như vậy, xác suất của y_i khi biết toàn bộ chuỗi $\mathbf{x}$ và các từ đã sinh trước đó $\mathbf{y}_{<i}$ có thể được xấp xỉ bởi

$$\Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \approx \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}_{\leq g(i)}), \tag{5.88}$$

trong đó $\mathbf{x}_{\leq g(i)}$ ký hiệu chuỗi con $x_1 \dots x_{g(i)}$. Khi đó, log xác suất của chuỗi phía đích $\mathbf{y}$ khi biết chuỗi phía nguồn $\mathbf{x}$ được viết là

$$\begin{aligned}\log \Pr(\mathbf{y}|\mathbf{x}) &= \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \\ &\approx \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}_{\leq g(i)}).\end{aligned} \tag{5.89}$$

Phương trình này phát biểu một bài toán chuỗi-sang-chuỗi thành một bài toán tiền tố-sang-tiền tố (*prefix-to-prefix*), tức là tiền tố $\mathbf{y}_{\leq i}$ chỉ phụ thuộc vào tiền tố $\mathbf{x}_{\leq g(i)}$. Suy luận cho mô hình này rất đơn giản. Với mỗi i, hệ thống tìm kiếm đợi cho đến khi nhận được $g(i)$ từ phía nguồn đầu tiên, rồi mở rộng các giả thuyết như thường lệ. Điều này có thể thực hiện bằng cách dùng lại các thuật toán mô tả ở tiểu mục trước. Ví dụ, ta có thể sửa thuật toán tìm kiếm chùm và thu được thuật toán tìm kiếm trực tuyến sau.

Một ưu điểm của hệ thống này là đầu ra ở bước i có thể được tạo ra ngay khi ta đã thấy $\mathbf{x}_{\leq g(i)}$. Điều này cho ta một hệ thống chuỗi-sang-chuỗi trực tuyến, trong đó các từ đầu vào đến thành một luồng liên tục và dự đoán được sinh ra ngay khi đã quan sát đủ số từ đầu vào.

Dù bài toán tìm kiếm ở đây có vẻ đơn giản, vẫn còn nhiều việc phải làm để định nghĩa $g(i)$. Hiển nhiên, $g(i)$ là một hàm đơn điệu không giảm. Một ví dụ đơn giản là định nghĩa $g(i) = m$ với mọi i. Khi đó, thuật toán trên trở về đúng thuật toán tìm kiếm chùm tiêu chuẩn làm việc với một chuỗi đầu vào hoàn chỉnh. Ngược lại, trong các tác vụ chuỗi-sang-chuỗi trực tuyến, ta muốn $g(i)$ càng nhỏ càng tốt, để có thể bắt đầu tính toán càng

Thuật toán: Một thuật toán tìm kiếm chùm trực tuyến

ONLINEBEAMSEARCH($\mathbf{x}, g(\cdot)$)
// Tìm kiếm trực tuyến, được thực hiện mỗi khi đã nhận đủ số từ đầu vào.
// Ở mỗi bước tìm kiếm, xét một số lượng định trước các ứng viên hứa hẹn.

Tạo một Heap có sức chứa $size_{\text{heap}}$
$Z_0 = \{z_0\}$ với $z_0.tgt = y_0$
$j = 1$ ▷ chỉ số từ nguồn
$i = 1$ ▷ bước sinh phía đích
$input = \emptyset$
while $i \leq n_{\text{max}}$ **do**
    if $j \leq g(i)$ **then** ▷ đọc một từ từ luồng đầu vào
        $input = input \circ x_j$
        j++
    else ▷ dự đoán ở bước i khi đã quan sát được $g(i)$ từ đầu vào (lưu trong $input$)
        Tạo một đống có giới hạn Beam kích thước $size_{\text{beam}}$
        for mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$ **do**
            for mỗi từ $v_k \in V_y$ **do**
                $z_{\text{next}} = \text{Extend}(z_{\text{cur}}, v_k, input)$
                if $input$ bằng **x** và $z_{\text{next}}.tgt$ kết thúc bằng ⟨EOS⟩ **then**
                    Thêm z_{next} vào Heap
                else
                    UpdateBeam(Beam, z_{next})
                end if
                if Heap đã đầy và/hoặc thỏa các tiêu chí dừng khác **then**
                    Thoát khỏi tất cả các vòng lặp
                end if
            end for
        end for
        OutputPartial(Beam)
        Z_i = Beam.PopAll()
        i++
    end if
end while
return Heap.Pop()

OutputPartial(Beam)
// Xuất ra một kết quả bộ phận

Hiển thị giả thuyết tốt nhất trong Beam

sớm càng tốt khi suy luận. Trường hợp đơn giản nhất là khi chuỗi đầu vào và chuỗi đầu ra đồng bộ theo một cách nào đó. Ví dụ, một hệ thống nhận dạng tiếng nói tự động gán cho mỗi khung phổ (*spectral frame*) một đơn vị phiên âm. Trong trường hợp này, ta có một sự tương ứng đơn giản giữa đầu vào và đầu ra: $m = n$ (tức $|\mathbf{x}| = |\mathbf{y}|$), và x_i tương ứng với y_i. Khi đó, ta chỉ cần định nghĩa $g(i) = i$; nói cách khác, mỗi khi một đầu vào mới đến, ta đưa ra một dự đoán.

Một trường hợp phức tạp hơn là các bài toán chuỗi-sang-chuỗi trực tuyến có đảo trật tự (*reordering*), chẳng hạn dịch song song (*simultaneous translation*) (dịch đồng thời), trong đó một từ phía đích có thể phụ thuộc vào các từ phía nguồn ở xa. Một cách đơn giản để giải quyết điều này là trì hoãn các dự đoán thêm một số bước. Ví dụ, phương pháp wait-k (*wait-k*) buộc mỗi dự đoán phải đi chậm hơn đầu vào k từ [Ma và cộng sự, 2019]. Một cách hình thức hơn, phiên bản wait-k của hàm $g(i)$ được định nghĩa là

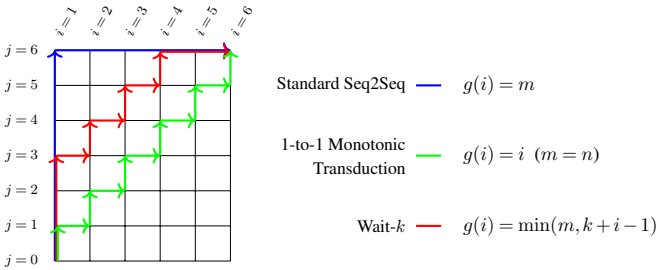
$$g(i) \;=\; \min(m, k + i - 1). \tag{5.90}$$

Ở đây k là một siêu tham số điều khiển độ lớn của ngữ cảnh phía nguồn được xét khi dự đoán các từ phía đích. Khi $k = \infty$, nó giống với các phương pháp tìm kiếm tiêu chuẩn cho suy luận chuỗi-sang-chuỗi. Trong dịch song song và các tác vụ liên quan, kết quả nhìn chung là thỏa đáng khi dùng một giá trị k nhỏ. Hình 5.11 so sánh các hàm $g(i)$ khác nhau.

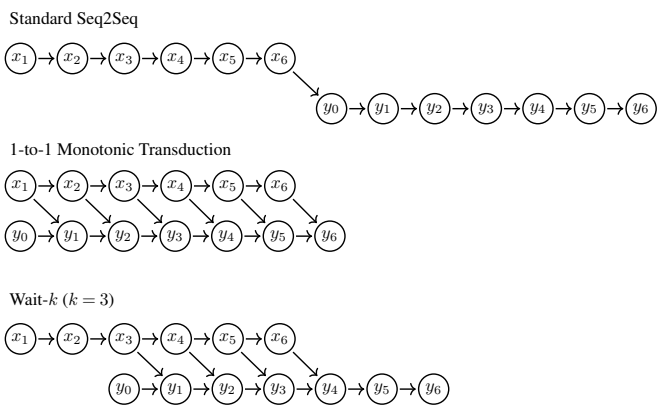
Chú giải của người dịch: Wait-k bằng số cụ thể

Với $m = 6$ từ nguồn và $k = 3$: $g(1) = \min(6, 3) = 3$, $g(2) = 4$, $g(3) = 5$, $g(4) = \min(6, 6) = 6$, $g(5) = \min(6, 7) = 6$, … Nghĩa là hệ thống đọc 3 từ nguồn rồi mới dịch từ đích đầu tiên, sau đó cứ đọc thêm một từ lại dịch thêm một từ; khi đã đọc hết câu nguồn thì dịch nốt phần còn lại. Đây chính là cách một phiên dịch viên "dịch đuổi" làm việc: không đợi diễn giả nói hết câu, nhưng cũng chờ vài từ để nắm ý trước khi nói. k nhỏ thì trễ ít nhưng dễ dịch sai (chưa đủ ngữ cảnh); k lớn thì ngược lại.

Trong một số ứng dụng của bài toán chuỗi-sang-chuỗi trực tuyến, ta có thể biết khi nào cần thực hiện tìm kiếm và khi nào cần đọc đầu vào. Ví dụ, trong dịch máy tương tác (*interactive machine translation*) [Casacuberta và cộng sự, 2009], việc dịch một chuỗi đầu vào bộ phận được kích hoạt trực tiếp bởi hành động của người dùng (ví dụ, nhấn phím). Trong trường hợp này, ta không cần định nghĩa $g(i)$ mà xem nó như một biến đầu vào



(a) Visualization of $g(i)$.



(b) Action sequences.

Hình 5.11: Biểu diễn trực quan (trên) và các chuỗi hành động (dưới) của các hàm $g(i)$ khác nhau cho một cặp chuỗi ($\mathbf{x} = x_1 \dots x_6$, $\mathbf{y} = y_1 \dots y_6$). Trong một chuỗi hành động, x_j được khoanh tròn biểu thị hành động đọc một từ phía nguồn (x_j), và y_i được khoanh tròn biểu thị hành động dự đoán xác suất của y_i khi biết $\mathbf{x}_{\leq g(i)}$ và $\mathbf{y}_{<i}$. Các mũi tên ở đây biểu thị sự phụ thuộc giữa các từ. Vì y_0 ký hiệu ký hiệu bắt đầu $\langle \text{SOS} \rangle$, nó có thể được sinh ra mà không phụ thuộc vào từ nào. (Nhãn trong hình: (a) biểu diễn trực quan của $g(i)$; (b) các chuỗi hành động; *Standard Seq2Seq* = chuỗi-sang-chuỗi tiêu chuẩn; *1-to-1 Monotonic Transduction* = chuyển đổi đơn điệu một-một.)

của mô hình.

Lưu ý rằng mặc dù có thể dùng trực tiếp các mô hình mức chuỗi đã được tiền huấn luyện để suy luận trực tuyến, việc phát triển các hệ thống như vậy thường đòi hỏi thêm công sức huấn luyện. Một cách tiếp cận có cơ sở hơn cho mô hình hóa chuỗi-sang-chuỗi trực tuyến là mô hình hóa phép biến đổi từ $\mathbf{x}$ sang $\mathbf{y}$ như một chuỗi các hành động (*action*) [Grissom II và cộng sự, 2014; Cho và Esipova, 2016; Gu và cộng sự, 2017; Zheng và cộng sự, 2019]. Ví dụ, một hành động có thể là thao tác *dự đoán* (*predict*), thực hiện tìm kiếm ở bước hiện tại, hoặc thao tác *đọc* (*read*), nhận một từ đầu vào mới. Khi đó, ta có thể phát biểu nhiệm vụ thiết kế hàm $g(i)$ thành việc học một chính sách (policy) để quyết định hành động nào được thực hiện khi biết một tiền tố phía nguồn $\mathbf{x}_{\leq j}$ và một tiền tố phía đích $\mathbf{y}_{<i}$. Như vậy, các mô hình chuỗi-sang-chuỗi có thể được huấn luyện trên các trạng thái của những chuỗi hành động này để đưa ra dự đoán tốt hơn khi chỉ dựa trên một phần của đầu vào. Tuy nhiên, việc bàn về huấn luyện các mô hình chuỗi-sang-chuỗi trực tuyến nằm ngoài phạm vi của mục này. Chúng tôi giới thiệu bạn đọc các bài báo trên để biết thêm chi tiết về những phương pháp này.

5.4.4 Tìm kiếm chính xác

Từ góc độ hình thức, lý tưởng nhất là ta muốn xây dựng một hệ thống không có lỗi tìm kiếm. Mặc dù các thuật toán tìm kiếm xấp xỉ (*approximate search*) đã được dùng thành công trong nhiều ứng dụng, điều quan trọng là nghiên cứu lỗi mô hình (*model error*) của các hệ thống này, và vì thế tập trung vào bài toán về mặt nguyên lý chứ không chỉ về mặt thực tiễn. Do đó, phát triển các thuật toán tìm kiếm chính xác cho các mô hình chuỗi-sang-chuỗi từ lâu đã là một chủ đề thú vị trong nghiên cứu NLP. Tuy nhiên, bài toán tìm kiếm cho một hệ thống dịch máy dựa trên từ đơn giản với các mô hình ngôn ngữ n-gram đã được chứng minh là một bài toán NP-khó (*NP-hard*) [Knight, 1999]. Phần lớn các nghiên cứu trước đây đã phát biểu bài toán tìm kiếm thành các bài toán tối ưu tổ hợp (*combinatorial optimization*) kinh điển, như bài toán quy hoạch tuyến tính (*linear programming*) và bài toán người du lịch (*traveling salesman problem*), rồi dùng các bộ giải tương ứng [Germann và cộng sự, 2004; Zaslavskiy và cộng sự, 2009]. Các nỗ lực nghiên cứu khác

đã khám phá các thuật toán tìm kiếm chính xác cho các hệ thống dịch máy thống kê bằng kỹ thuật nới lỏng Lagrange (*Lagrangian relaxation*) [Chang và Collins, 2011; Rush và Collins, 2012] và ôtômat hữu hạn trạng thái (*finite-state automata*) [de Gispert và cộng sự, 2010; Allauzen và cộng sự, 2014].

Chú giải của người dịch: NP-khó nghĩa là gì?

Nói nôm na, một bài toán là *NP-khó* nếu cho tới nay người ta chưa biết thuật toán nào giải *chính xác* nó trong thời gian tăng "vừa phải" (đa thức, như n^2 hay n^3) theo kích thước đầu vào, và nhiều người tin rằng không tồn tại thuật toán như vậy. Bài toán người du lịch (tìm đường ngắn nhất đi qua tất cả các thành phố, mỗi thành phố một lần) là ví dụ nổi tiếng: với 20 thành phố đã có khoảng 10^{17} lộ trình. Vì vậy, người ta thường chấp nhận lời giải xấp xỉ, hoặc tìm cách khai thác cấu trúc đặc biệt của bài toán như ở phần dưới đây.

Khác với các phương pháp này — vốn ít nhiều phụ thuộc vào việc tích hợp mô hình ngôn ngữ n-gram vào mô hình chuỗi-sang-chuỗi — các mô hình được mô tả trong chương này có dạng đơn giản hơn. Ta bắt đầu với một mô hình cơ bản, trong đó hàm tính điểm $\text{Score}(\mathbf{x}, \mathbf{y})$ là log xác suất $\log \Pr(\mathbf{y}|\mathbf{x})$. Phương trình (5.62) cho ta biết $\log \Pr(\mathbf{y}|\mathbf{x})$ có thể viết thành tổng các log xác suất mức từ, và $\log \Pr(\mathbf{y}|\mathbf{x})$ trở nên nhỏ hơn khi sinh thêm nhiều từ phía đích (tức n lớn hơn)[11]. Nói cách khác, $\log \Pr(\mathbf{y}|\mathbf{x})$ là một hàm đơn điệu giảm theo độ dài phía đích n: với mọi i, ta có

$$\begin{aligned} \log \Pr(\mathbf{y}_{\leq i}|\mathbf{x}) &= \log \Pr(\mathbf{y}_{<i}|\mathbf{x}) + \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \\ &\leq \log \Pr(\mathbf{y}_{<i}|\mathbf{x}). \end{aligned} \tag{5.91}$$

Tính chất này còn được gọi là tính đơn điệu (*monotonicity*) của hàm tính điểm.

Khi đó, bằng cách tận dụng tính đơn điệu của điểm mô hình, ta có thể xây dựng một heuristic để loại bỏ những giả thuyết không bao giờ có thể là tốt nhất. Gọi $z_{\text{bestinall}}$ là giả thuyết hoàn chỉnh tốt nhất toàn cục tìm được cho đến lúc này. Nếu một giả thuyết bộ phận mới có điểm mô hình thấp hơn hẳn $z_{\text{bestinall}}.score$, nó không thể được mở rộng để vượt qua $z_{\text{bestinall}}$, nên ta không cần mở rộng nó. Điều này cho phép ta khám phá một vùng

[11] Xét $\log \Pr(\mathbf{y}|\mathbf{x}) = \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x})$. Vì $\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x})$ có giá trị không dương, $\log \Pr(\mathbf{y}|\mathbf{x})$ sẽ nhỏ đi hoặc không đổi khi n tăng.

nhỏ hơn đáng kể so với không gian tìm kiếm ban đầu mà không hề mất tính tối ưu. Lưu ý rằng khi việc tìm kiếm tiến triển, $z_{\text{bestinall}}.score$ liên tục tăng. Việc tìm được một giả thuyết tốt hơn ngày càng khó, dẫn đến cắt tỉa ngày càng mạnh tay hơn khi việc tìm kiếm tiếp diễn. Mã giả dưới đây phác thảo một thuật toán tìm kiếm chính xác cho mô hình chuỗi-sang-chuỗi được định nghĩa bởi Phương trình (5.62).

Thuật toán: Một thuật toán tìm kiếm chính xác

ExactSearch($\mathbf{x}$)
*// Tìm giả thuyết "tốt nhất" bằng cách tận dụng tính đơn điệu của hàm tính điểm (*Score$(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x})$*).*

Tạo một hàng đợi ưu tiên (đống cực đại) Queue
Tạo một giả thuyết z_{best} với $z_{\text{best}}.score = -\infty$
while Queue chưa rỗng **do**
    $z_{\text{cur}} =$ Queue.Pop()
    if $|z_{\text{cur}}.tgt| > n_{\max}$ **then**
        bỏ qua z_{cur} và tiếp tục vòng lặp
    end if
    for mỗi từ $v_k \in V_y$ **do**
        $z_{\text{next}} =$ Extend$(z_{\text{cur}}, v_k, \mathbf{x})$
        $bound = z_{\text{best}}.score$ ▷ một cận dưới của điểm mô hình
        if $bound < z_{\text{next}}.score$ **then** ▷ cắt tỉa chấp nhận được
            if $z_{\text{next}}.tgt$ kết thúc bằng ⟨EOS⟩ **then**
                $z_{\text{best}} = z_{\text{next}}$
                $bound = z_{\text{next}}.score$
            else
                Thêm z_{next} vào Queue
            end if
        end if
    end for
end while
return z_{best}

Đây là một thuật toán tổng quát cho tìm kiếm chính xác, và hiệu quả tìm kiếm của nó chịu ảnh hưởng lớn bởi thiết kế của hàng đợi ưu tiên (*priority queue*) [Meister và cộng sự, 2020]. Ví dụ, ta có thể xem Score$(\mathbf{x}, \mathbf{y})$ là độ ưu tiên của mỗi giả thuyết trong hàng đợi ưu tiên, như

trong một đống cực đại[12]. Khi đó, thuật toán thu được thực hiện một thủ tục tìm kiếm gần giống tìm kiếm theo chiều rộng, vì một giả thuyết có chuỗi phía đích ngắn hơn thường có điểm mô hình cao hơn và dễ đứng đầu hàng đợi ưu tiên hơn. Tuy nhiên, để tìm kiếm hiệu quả, ta muốn tìm ra các giả thuyết hoàn chỉnh càng sớm càng tốt, sao cho nhiều giả thuyết kém hứa hẹn có thể bị loại bỏ ngay ở giai đoạn đầu của việc tìm kiếm. Để làm vậy, ta có thể thiên vị độ ưu tiên của một giả thuyết về phía chuỗi đích dài hơn. Điều này cho ta một thuật toán tìm kiếm theo chiều sâu (*depth-first search*), có nhiều khả năng tìm được giả thuyết hoàn chỉnh trong thời gian ngắn hơn [Stahlberg và Byrne, 2019].

Chú giải của người dịch: Ý tưởng "nhánh và cận"

Thuật toán trên giống trò chơi tìm đường rẻ nhất: nếu bạn đã biết một lộ trình hoàn chỉnh tốn 100 nghìn đồng, thì một lộ trình đang đi dở mà đã tốn 120 nghìn đồng chắc chắn không thể tốt hơn (vì đi tiếp chỉ tốn thêm), nên bỏ nó đi ngay. Ở đây, "chi phí" là $-\log \Pr$, luôn tăng khi thêm từ (Phương trình (5.91)). Biến *bound* đóng vai trò "kỷ lục hiện tại"; kỷ lục càng tốt thì càng nhiều nhánh bị cắt. Vì vậy, tìm sớm một giả thuyết hoàn chỉnh tốt (nhờ tìm kiếm theo chiều sâu) giúp thuật toán chạy nhanh hơn rất nhiều mà vẫn cho kết quả chính xác.

Mặc dù thuật toán tìm kiếm chính xác trở nên rõ ràng khi xét tính đơn điệu của $\Pr(\mathbf{y}|\mathbf{x})$, trong các hệ thống thực tế, như đã bàn ở Mục 5.4.1, $\text{Score}(\mathbf{x}, \mathbf{y})$ thường có dạng phức tạp hơn, bao gồm thưởng độ dài hoặc chuẩn hóa độ dài, nên tính đơn điệu không còn đúng. May mắn là có thể bỏ giả định đơn điệu với cái giá là nới lỏng một chút cận dưới của điểm mô hình dùng để cắt tỉa. Ở đây ta định nghĩa $bound$ là điểm mô hình thấp nhất mà một giả thuyết cần có để trong trường hợp tốt nhất nó có thể được mở rộng thành một giả thuyết tốt ngang z_{best}. Ví dụ, xét mô hình thưởng từ đơn giản mô tả ở Phương trình (5.63): $\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot n$. Với một giả thuyết z_{next}, có tối đa $n_{\max} - |z_{\text{next}}.tgt|$ từ mà ta có thể dự đoán để thu được một giả thuyết hoàn chỉnh. Giả sử tất cả $n_{\max} - |z_{\text{next}}.tgt|$ từ này đều được dự đoán với xác suất 1. Khi đó, điểm mô hình của giả thuyết

[12] Ta có thể hiện thực một hàng đợi ưu tiên bằng một đống cực đại.

thu được (ký hiệu là z_{new}) sẽ là

$$\begin{aligned} z_{\text{new}}.score &= z_{\text{next}}.score + \sum_{i=|z_{\text{next}}.tgt|+1}^{n_{\max}} (\log 1 + \lambda) \\ &= z_{\text{next}}.score + \lambda \cdot (n_{\max} - |z_{\text{next}}.tgt|). \end{aligned} \tag{5.92}$$

Dùng kết quả này, ta có thể định nghĩa $bound$ là

$$bound \;=\; z_{\text{best}}.score - \lambda \cdot (n_{\max} - |z_{\text{next}}.tgt|). \tag{5.93}$$

Một cách khác để suy ra cận dưới là chỉ cần trừ đi phần thưởng từ lớn nhất có thể, cho bởi

$$bound \;=\; z_{\text{best}}.score - \lambda \cdot n_{\max}. \tag{5.94}$$

Đây là một cận dưới lỏng và dẫn tới ít cắt tỉa hơn.

Trong trường hợp chuẩn hóa độ dài, ta có thể làm tương tự. Ví dụ, xét mô hình chuẩn hóa độ dài $\text{Score}(\mathbf{x}, \mathbf{y}) = \frac{\log \Pr(\mathbf{y}|\mathbf{x})}{n}$, như trong Phương trình (5.66–5.67). Một cận dưới cho các điểm mô hình chấp nhận được được cho bởi

$$bound \;=\; \frac{\Pr(z_{\text{next}}.tgt|\mathbf{x})}{n_{\max}}. \tag{5.95}$$

Trong thực tế, một cận dưới như vậy có thể được định nghĩa theo nhiều cách khác nhau để bảo đảm tính tối ưu của việc tìm kiếm, tùy thuộc vào mô hình và chiến lược tìm kiếm được dùng trong hệ thống chuỗi-sang-chuỗi [Huang và cộng sự, 2017b; Stahlberg và Byrne, 2019].

Ta có thể dễ dàng áp dụng các cận dưới này cho thuật toán tìm kiếm chính xác ở trên bằng cách thay dòng 9 bằng Phương trình (5.93) hoặc (5.95). Như một tác dụng phụ, việc tìm kiếm sẽ khám phá nhiều giả thuyết hơn và do đó chậm hơn nhiều.

Chú giải của người dịch: Vì sao phải "nới" cận khi có thưởng độ dài?

Khi có thưởng λ cho mỗi từ, một giả thuyết dở dang hiện đang thua kỷ lục vẫn có thể "lội ngược dòng" nhờ các từ thưởng sau này. Phương trình (5.92) tính kịch bản lạc quan nhất: mọi từ còn lại đều có xác suất 1 (nên $\log 1 = 0$, không mất điểm) và mỗi từ được cộng thêm λ. Ví dụ: $\lambda = 0.5$, $n_{\max} = 10$, giả thuyết đã có 4 từ; nó còn có thể được cộng tối đa $0.5 \cdot 6 = 3$ điểm. Vì

vậy, chỉ khi nó thua kỷ lục *hơn* 3 điểm thì mới chắc chắn bỏ được — đó là Phương trình (5.93).

5.4.5 Tìm kiếm khả vi

Ta đã giải quyết bài toán tìm kiếm bằng cách đưa ra các thuật toán tìm kiếm heuristic, trong đó ta cố gắng cực đại hóa hàm tính điểm trên một tập các chuỗi biến rời rạc. Một cách khác là nới lỏng các biến rời rạc này thành biến liên tục và phát biểu bài toán thành một bài toán **tối ưu liên tục** (*continuous optimization*) [Hoang và cộng sự, 2017; Kumar và cộng sự, 2021]. Mặc dù chúng tôi cố gắng dùng ký hiệu nhất quán trong toàn bộ cuốn sách, ở đây sẽ tiện hơn nếu đưa vào một ký hiệu hơi khác so với các chương trước. Ta sẽ dùng vectơ $\mathbf{y}_i^w \in \{0,1\}^{|V_y|}$ để ký hiệu biểu diễn one-hot (*one-hot representation*) của y_i. Giả sử đầu ra ở bước i là một phân phối trên V_y, ký hiệu là $\Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x})$. Khi đó, ta có thể biểu diễn log xác suất của y_i bằng tích vô hướng của hai vectơ:

$$\begin{aligned}\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) &= \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x}) \\ &= \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}),\end{aligned} \tag{5.96}$$

trong đó $\mathbf{y}_{<i} = y_0 \dots y_{i-1}$ được biểu diễn thành một chuỗi các vectơ one-hot $\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w$. Như đã bàn ở Chương 3, vế phải của phương trình trên tương ứng với việc chọn phần tử y_i của vectơ $\log \Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x})$ (hoặc $\log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x})$).

Với ký hiệu này, ta có thể viết $\log \Pr(\mathbf{y}|\mathbf{x})$ là

$$\begin{aligned}\log \Pr(\mathbf{y}|\mathbf{x}) &= \sum_{i=1}^{n} \log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \\ &= \sum_{i=1}^{n} \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}).\end{aligned} \tag{5.97}$$

Chú giải của người dịch: Vectơ one-hot "chọn" một phần tử

Giả sử bộ từ vựng chỉ có 3 từ {tôi, đi, học} và $\log \Pr(\cdot|\dots) = [-2.3, -0.1, -1.6]$. Nếu y_i = "đi" thì vectơ one-hot là $\mathbf{y}_i^w = [0,1,0]$, và tích vô hướng $[0,1,0] \cdot [-2.3, -0.1, -1.6] = 0 \cdot (-2.3) + 1 \cdot (-0.1) +$

$0 \cdot (-1.6) = -0.1$ đúng bằng log xác suất của "đi". Nếu thay $[0, 1, 0]$ bằng một vectơ "mềm" như $[0.2, 0.7, 0.1]$ (các số không âm, tổng bằng 1), tích vô hướng cho $0.2 \cdot (-2.3) + 0.7 \cdot (-0.1) + 0.1 \cdot (-1.6) = -0.69$: đó là *giá trị trung bình có trọng số* (kỳ vọng) của log xác suất. Đây chính là ý tưởng "nới lỏng" ở phần dưới.

Nếu ta dùng $\log \Pr(\mathbf{y}|\mathbf{x})$ làm hàm mục tiêu (tức $\text{Score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x})$), bài toán tìm kiếm có thể được phát biểu là

$$\hat{\mathbf{y}}_0^w \dots \hat{\mathbf{y}}_n^w = \underset{\mathbf{y}_1^w \dots \mathbf{y}_n^w}{\arg\max} \sum_{i=1}^{n} \mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}). \tag{5.98}$$

Điều này tương đương với dạng tiêu chuẩn của suy luận trong các mô hình chuỗi-sang-chuỗi, cho bởi

$$\begin{aligned} \hat{\mathbf{y}} &= \hat{y}_0 \dots \hat{y}_n \\ &= \underset{y_0 \dots y_n}{\arg\max} \Pr(y_0 \dots y_n|\mathbf{x}). \end{aligned} \tag{5.99}$$

Từ Phương trình (5.98), giờ ta có thể nới lỏng mỗi vectơ one-hot thành một vectơ giá trị thực với ràng buộc rằng tổng mọi phần tử của nó bằng 1, tức là

$$\mathbf{y}_i^w \in \mathbb{R}_+^{|V_y|} \tag{5.100}$$

$$\text{với điều kiện } \|\mathbf{y}_i^w\|_1 = 1. \tag{5.101}$$

Bằng cách này, $\mathbf{y}_i^w$ có thể được xem một cách không chính thức là một biểu diễn $|V_y|$ chiều của y_i, dù nó có nhiều chiều hơn hẳn các vectơ nhúng dày đặc thường dùng trong NLP. Thay vì tương ứng với một từ cụ thể trong bộ từ vựng, $\mathbf{y}_i^w$ giờ mô tả một phân phối liên tục trên bộ từ vựng. Kết quả là số hạng $\mathbf{y}_i^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x})$ tính **log xác suất kỳ vọng** (*expected log-probability*) của dự đoán ở bước i. Để làm cho phân phối có điều kiện $\Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x})$ khả vi theo tiền tố liên tục, Hoang và cộng sự [2017] thay thao tác tra bảng nhúng từ rời rạc tiêu chuẩn ở mỗi bước của bộ giải mã bằng một **vectơ nhúng kỳ vọng** (*expected embedding*), tức là một trung bình có trọng số của ma trận nhúng từ theo vectơ xác suất $\mathbf{y}_{i-1}^w$. Điều thú vị là cách phát biểu này ngụ ý rằng Phương trình (5.98) định nghĩa một tác vụ "mới", trong đó ta cố gắng cực đại hóa một hàm mục tiêu liên tục (tức là tổng của n log xác suất kỳ vọng).

Ta có thể giải Phương trình (5.98) bằng các bộ công cụ tối ưu có sẵn. Vì $\mathbf{y}_i^w$ bị ràng buộc trên một **đơn hình** (*simplex*)[13], việc áp dụng các thuật toán **tối ưu có ràng buộc** (*constrained optimization*) tổng quát là khá trực tiếp. Ngoài ra, có thể dùng các thuật toán được thiết kế riêng để giải bài toán tối ưu với ràng buộc đơn hình.

Chú giải của người dịch: Đơn hình qua hình ảnh quen thuộc

Với $k = 1$, đơn hình là một đoạn thẳng nối $\mathbf{p}_0$ và $\mathbf{p}_1$; với $k = 2$ là một tam giác (kể cả miền trong); với $k = 3$ là một tứ diện. Các điểm $a_0\mathbf{p}_0 + a_1\mathbf{p}_1 + a_2\mathbf{p}_2$ với $a_i \geq 0$, $a_0 + a_1 + a_2 = 1$ chính là mọi điểm trong tam giác — giống công thức trọng tâm khi $a_0 = a_1 = a_2 = \frac{1}{3}$. Một vectơ xác suất trên bộ từ vựng (các số không âm, tổng bằng 1, như ràng buộc (5.100–5.101)) chính là một điểm trên đơn hình có $|V_y|$ đỉnh; mỗi vectơ one-hot là một *đỉnh* của nó. "Nới lỏng" nghĩa là cho phép đi vào bên trong đơn hình thay vì chỉ nhảy giữa các đỉnh, nhờ đó có thể dùng đạo hàm để tối ưu.

Cách tiếp cận thứ ba để giải Phương trình (5.98) là đưa tường minh các ràng buộc vào hàm mục tiêu và dùng các phương pháp hạ gradient để tối ưu hàm này. Ví dụ, Hoang và cộng sự [2017] sửa Phương trình (5.98) và thu được một dạng mới để tối ưu

$$\hat{\mathbf{y}}_0^w \dots \hat{\mathbf{y}}_n^w = \underset{\mathbf{y}_1^w \dots \mathbf{y}_n^w}{\arg\max} \sum_{i=1}^{n} \text{Softmax}(\mathbf{y}_i^w) \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w, \mathbf{x}). \tag{5.105}$$

Ở đây ta bỏ ràng buộc đơn hình trên $\mathbf{y}_i^w$ và thay vào đó áp đặt nó thông qua đầu ra của Softmax.

Khi đã thu được chuỗi tối ưu $\hat{\mathbf{y}}_0^w \dots \hat{\mathbf{y}}_n^w$, ta cần ánh xạ mỗi $\mathbf{y}_i^w$ về một

[13] Đơn hình là một khái niệm hình học. Trong không gian Euclid, một k-đơn hình là một đa diện k chiều được mô tả bởi một tập $k+1$ điểm độc lập $\{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_k\}$. Đa diện này được định nghĩa là tập các điểm

$$P_{k\text{-simplex}} = \{a_0 \cdot \mathbf{p}_0 + a_1 \cdot \mathbf{p}_1 + \dots + a_k \cdot \mathbf{p}_k\}, \tag{5.102}$$

trong đó

$$\sum_{i=0}^{k} a_i = 1, \tag{5.103}$$

$$a_i \geq 0 \quad \text{với mọi } i \in [0, k]. \tag{5.104}$$

từ. Một cách đơn giản là chọn từ tương ứng với phần tử có giá trị lớn nhất của $\mathbf{y}_i^w$. Tuy nhiên, điều này có thể phá vỡ tính tối ưu của lời giải, vì điều kiện $\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w$ bị thay đổi khi các biến này được rời rạc hóa. Một cách thực tế hơn là thực hiện tối ưu để dự đoán từ tiếp theo khi biết một tiền tố; ví dụ, ta cố định $\mathbf{y}_0^w \dots \mathbf{y}_{i-1}^w$ bằng các biểu diễn one-hot của tiền tố tối ưu, và cực đại hóa $\sum_{k=i}^{n} \mathbf{y}_k^w \cdot \log \Pr(\cdot|\mathbf{y}_0^w \dots \mathbf{y}_{k-1}^w, \mathbf{x})$. Sau đó, ta chọn từ tốt nhất ở vị trí i và chuyển sang vị trí tiếp theo.

Cho đến giờ, ta đã giả định rằng mục tiêu tìm kiếm được suy ra từ log xác suất $\log \Pr(\mathbf{y}|\mathbf{x})$ và độ dài đầu ra được xác định trước. Để tìm kiếm trên các chuỗi có độ dài khác nhau, ta có thể lặp lại thủ tục tối ưu trên cho mọi $n \in [1, n_{\max}]$ và chọn chuỗi có điểm cao nhất. Cách làm này cũng giúp dễ dàng tích hợp chuẩn hóa độ dài và thưởng độ dài vào việc tìm kiếm. Cụ thể, ta có thể bỏ qua thiên lệch độ dài trong lúc tối ưu với n cố định và áp dụng các mô hình độ dài sau đó (*post-hoc*). Tức là, ta giữ nguyên mục tiêu tìm kiếm ban đầu và, ở bước cuối cùng, chọn ứng viên tốt nhất trên mọi độ dài dựa trên một $\text{Score}(\mathbf{x}, \mathbf{y})$ đã được chuẩn hóa độ dài.

5.4.6 Tính đa dạng của giả thuyết

Việc sinh ra nhiều đầu ra thường cần thiết khi ta muốn chấm điểm lại (*rescore*) các đầu ra này và/hoặc tương tác với hệ thống. Một trong những phương pháp được dùng rộng rãi nhất là dùng tìm kiếm chùm để sinh ra các giả thuyết xếp hạng đầu. Ví dụ, ta có thể đơn giản xem các phần tử của Heap là k giả thuyết tốt nhất (*k-best*) trong tìm kiếm chùm (xem Mục 5.4.2). Tuy nhiên, cách này thường tạo ra các giả thuyết rất giống nhau, và khó xác định cái nào tốt hơn ngay cả khi có nhiều lựa chọn hơn. Bảng 5.3 cho thấy 4 đầu ra tốt nhất của một hệ thống tóm tắt văn bản (*text summarization*). Ta thấy các văn bản này khá giống nhau. Một lý do của hiện tượng này là các giả thuyết đa dạng, kể cả những giả thuyết cuối cùng có thể đạt điểm mô hình cao khi hoàn chỉnh, sẽ bị cắt tỉa nếu chúng rơi ra khỏi chùm ở bất kỳ giai đoạn trung gian nào của việc tìm kiếm. Từ góc độ mô hình hóa, ta có thể hiểu điều này là một hạn chế cố hữu của các mô hình chuẩn hóa cục bộ (*locally normalized*) được dùng ở đây: mọi dự đoán đều được đưa ra ở một bước trung gian của việc tìm kiếm, và không có cơ chế nào để sửa các quyết định trong quá khứ một khi một tiền tố đã được chốt [Murray và Chiang, 2018].

Bảng 5.3: 4 đầu ra tốt nhất của một hệ thống tóm tắt văn bản trên một mẫu trong bộ dữ liệu CNN/Daily Mail (kích thước chùm $= 4$). Ta thấy các văn bản này chỉ khác nhau vài từ. (Người dịch giữ nguyên văn tiếng Anh và in nghiêng phần khác biệt; bản dịch nghĩa ở dưới.)

Hạng	Đầu ra
1	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker, *91,* showed up to run *his old* show.
2	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker showed up to run *his old* show.
3	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker, *91,* showed up to run *the* show.
4	Manuela Arbelaez accidentally revealed the correct answer to a guessing game for a new Hyundai Sonata. Host Drew Carey couldn't stop laughing. It's been a busy week for "The Price Is Right" when Bob Barker, *91,* showed up to run *his* show.

Dịch nghĩa (hạng 1): Manuela Arbelaez vô tình để lộ đáp án đúng của một trò chơi đoán giá để giành chiếc Hyundai Sonata mới. Người dẫn chương trình Drew Carey không nhịn được cười. Đó là một tuần bận rộn của chương trình "The Price Is Right" khi Bob Barker, 91 tuổi, xuất hiện để dẫn lại chương trình cũ của mình. **Khác biệt:** hạng 2 bỏ "91 tuổi"; hạng 3 viết "dẫn chương trình" thay vì "dẫn chương trình cũ của mình"; hạng 4 viết "dẫn chương trình của mình".

Một cách để cải thiện tính đa dạng của giả thuyết là đưa vào các hình phạt khi các giả thuyết trong chùm thiếu sự khác biệt [Li và Jurafsky, 2016; Vijayakumar và cộng sự, 2018]. Một hàm mục tiêu đơn giản được cho bởi

$$\text{Score}_d(\mathbf{x}, \mathbf{y}) \;=\; \text{Score}(\mathbf{x}, \mathbf{y}) - \lambda \cdot dp. \tag{5.106}$$

Nó kết hợp điểm mô hình gốc $\text{Score}(\mathbf{x}, \mathbf{y})$ với một hình phạt đa dạng (*diversity penalty*) dp. dp có thể được định nghĩa theo nhiều cách. Một cách là phạt các giả thuyết nằm gần nhau trên cây tìm kiếm. Ví dụ, có thể định nghĩa dp là thứ hạng của một giả thuyết trong tập các "anh em" của nó, tức các giả thuyết được mở rộng từ cùng một giả thuyết cha, nhờ đó chùm có thể trải các thành viên của nó trên một vùng rộng hơn của không gian giả thuyết [Li và Jurafsky, 2016]. Một cách khác để đưa vào các thước đo đa dạng là xét sự khác biệt giữa các chuỗi phía đích của các giả thuyết trong chùm. Ví dụ, ta có thể định nghĩa dp là độ tương đồng xâu trung bình giữa một giả thuyết cho trước và các giả thuyết khác trong chùm [Xiao và cộng sự, 2013].

Ý tưởng trên cũng có thể được biểu diễn thành các ràng buộc áp đặt lên thủ tục tìm kiếm. Ví dụ, ta có thể ràng buộc chùm chỉ chứa những giả thuyết bắt nguồn từ các giả thuyết cha khác nhau ở bước trước [Boulanger-Lewandowski và cộng sự, 2013]. Chính xác hơn, với mỗi giả thuyết $z_{\text{cur}} \in Z_{i-1}$, ta chọn giả thuyết tốt nhất cho bước tiếp theo như sau

$$\hat{z}_{\text{next}} \;=\; \underset{z_{\text{next}} \in \text{Extend}(z_{\text{cur}}, V_y)}{\arg\max} \Pr(z_{\text{next}}.tgt|\mathbf{x}). \tag{5.107}$$

Giả thuyết $\hat{z}_{\text{next}}$ sau đó được thêm vào Z_i. Lưu ý rằng về bản chất đây là một phương pháp không gian con (*subspace method*): chia không gian giả thuyết thành các không gian con và thu thập kết quả trên các không gian con này. Ý tưởng trực giác đằng sau phương pháp là các không gian con khác nhau có thể mô tả các khía cạnh khác nhau của bài toán, và nhờ đó ta có được các lời giải đa dạng.

Một cách tiếp cận khác để giải quyết vấn đề đa dạng là nhiễu loạn (*perturb*) tìm kiếm chùm bằng cách đưa các giả thuyết được sinh ngẫu nhiên vào chùm [Holtzman và cộng sự, 2020a; Wiher và cộng sự, 2022]. Một cách phổ biến để làm điều này là chọn một số từ ngẫu nhiên để mở rộng một giả thuyết, rồi thêm các giả thuyết đã mở rộng vào chùm. Nói chung, các từ này có thể được lấy mẫu từ phân phối $\Pr(\cdot|\mathbf{y}_{<i}, \mathbf{x})$ trên toàn

bộ bộ từ vựng hoặc một tập con của nó. Tính ngẫu nhiên cũng có thể được thêm vào đầu vào của hệ thống lúc kiểm tra. Ví dụ, một từ đầu vào có thể được biểu diễn thành một tổ hợp tuyến tính của vectơ nhúng gốc của nó và vectơ nhúng của một từ được lấy mẫu từ một chuỗi huấn luyện chọn ngẫu nhiên [Li và cộng sự, 2021b]. Trong các bài toán có nhiều cực tiểu địa phương (*local minima*), thêm nhiễu ngẫu nhiên vào quá trình tìm kiếm nhìn chung là có ích, vì ta có thể khám phá các giả thuyết đa dạng hơn và tránh để hệ thống bị mắc kẹt ở một số vùng nhất định của không gian tìm kiếm.

Thay vì thực hiện tìm kiếm bằng một hệ thống duy nhất, ta có thể dùng nhiều hệ thống để thu được các giả thuyết đa dạng. Các hệ thống này có thể được xây dựng trên các kiến trúc khác nhau hoặc các cấu trúc/cấu hình ẩn khác nhau [He và cộng sự, 2018; Shen và cộng sự, 2019; Wu và cộng sự, 2020a; Sun và cộng sự, 2020a]. Mặc dù những phương pháp này không thuộc khung tìm kiếm mà ta đang bàn, việc kết hợp kết quả từ nhiều hệ thống nhìn chung là có ích. Tiểu mục sau sẽ bàn về vấn đề này.

Chú giải của người dịch: Vì sao cần đa dạng?

Hãy hình dung bạn nhờ 4 người bạn gợi ý quà sinh nhật, nhưng cả 4 cùng gợi ý "một chiếc áo phông xanh", chỉ khác cỡ áo — có 4 gợi ý mà như chỉ có 1. Bảng 5.3 cho thấy tình huống tương tự: 4 bản tóm tắt chỉ khác nhau vài từ. Hình phạt *dp* trong Phương trình (5.106) trừ điểm những ứng viên "na ná" các ứng viên khác, còn Phương trình (5.107) buộc mỗi giả thuyết cha chỉ được "gửi" một con tốt nhất vào chùm, nên các thành viên của chùm đến từ các nhánh khác nhau của cây tìm kiếm.

5.4.7 Kết hợp nhiều mô hình

Trong học máy, kết hợp mô hình (*ensembling*) (ensemble) chỉ các phương pháp nhằm giải quyết những hạn chế về mô hình hóa chứ không phải các vấn đề về tìm kiếm. Trong tiểu mục này, chúng tôi bàn về các phương pháp này vì việc hiện thực chúng thường đòi hỏi sửa đổi các mô-đun tìm kiếm, và ta có thể hiểu sâu hơn về hệ thống thu được khi nhìn nó từ góc độ tìm kiếm.

Các phương pháp kết hợp mô hình nhằm cải thiện hiệu năng dự đoán bằng cách kết hợp đầu ra của nhiều hệ thống thành phần

(*constituent/component system*). Bài toán kết hợp nhiều mô hình đã được bàn luận rộng rãi kể từ khi các mô hình thống kê xuất hiện trong NLP, nơi các kỹ thuật này thường được gọi là các phương pháp **kết hợp hệ thống** (*system combination*) để nhấn mạnh tính hữu dụng thực tiễn của chúng. Với các mô hình chuỗi-sang-chuỗi được bàn ở đây, một dạng kết hợp hệ thống được dùng rộng rãi là **lấy trung bình dự đoán** (*prediction averaging*) [Sutskever và cộng sự, 2014]. Giả sử ta có một tập hợp K mô hình chuỗi-sang-chuỗi đã huấn luyện. Log xác suất sinh từ phía đích y_i, khi biết ngữ cảnh bên trái $\mathbf{y}_{<i}$ và chuỗi phía nguồn $\mathbf{x}$, có thể được định nghĩa bằng trung bình nhân:

$$\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \;=\; \frac{1}{K}\sum_{k=1}^{K} \log \Pr_k(y_i|\mathbf{y}_{<i}, \mathbf{x}), \tag{5.108}$$

hoặc bằng trung bình cộng

$$\log \Pr(y_i|\mathbf{y}_{<i}, \mathbf{x}) \;=\; \log \frac{1}{K}\sum_{k=1}^{K} \Pr_k(y_i|\mathbf{y}_{<i}, \mathbf{x}), \tag{5.109}$$

trong đó $\Pr_k(y_i|\mathbf{y}_{<i}, \mathbf{x})$ là đầu ra của hệ thống thành phần thứ k. Các dạng này đơn giản đến mức có thể hiện thực cho bất kỳ mô hình chuỗi-sang-chuỗi nào mà không cần sửa đổi đáng kể các hệ thống hiện có, và chúng đã được dùng làm nền tảng cho nhiều hệ thống thành công trong các cuộc thi đánh giá khác nhau [Barrault và cộng sự, 2020; Akhbardeh và cộng sự, 2021].

Chú giải của người dịch: Trung bình nhân và trung bình cộng

Giả sử có $K = 2$ mô hình, cho từ y_i các xác suất 0.9 và 0.1. Trung bình cộng: $\frac{0.9+0.1}{2} = 0.5$. Trung bình nhân: $\sqrt{0.9 \cdot 0.1} = 0.3$ (Phương trình (5.108) chính là log của trung bình nhân, xem Phương trình (5.68–5.69)). Trung bình nhân "khắt khe" hơn: chỉ cần một mô hình cho xác suất rất thấp là kết quả bị kéo xuống mạnh — giống như cần mọi giám khảo đồng thuận. Trung bình cộng "dễ dãi" hơn: một mô hình rất tự tin có thể bù cho mô hình kia.

Một hạn chế của việc lấy trung bình dự đoán là mọi hệ thống thành phần phải tuân theo cùng một cách phát biểu tự hồi quy cơ bản (xem Phương trình (5.62)), và ta cần truy cập trực tiếp vào các xác suất đầu ra theo từng bước $\{\Pr_k(y_i|\mathbf{y}_{<i}, \mathbf{x})\}$ của chúng. Khi làm việc với các hệ thống

hộp đen (*black-box*), ta buộc phải thực hiện kết hợp mức chuỗi (*sequence-level ensembling*). Một chiến lược phổ biến là tổng hợp các chuỗi hoàn chỉnh do các hệ thống thành phần tạo ra. Cách trực tiếp nhất là chọn giả thuyết (*hypothesis selection*) [Hildebrand và Vogel, 2008], trong đó ta đơn giản chọn ra một chuỗi "tốt nhất" từ tập các chuỗi đã sinh dựa trên một tiêu chí định trước. Một cách khác là sinh lại một chuỗi mới, có thể khác với mọi đầu ra riêng lẻ trong tập [Matusov và cộng sự, 2006; Rosti và cộng sự, 2007]. Điều này thường đòi hỏi nén các chuỗi đã sinh thành một cấu trúc thống nhất gọn nhẹ (như một lưới từ (*word lattice*)), tiếp theo là một lượt tìm kiếm bổ sung (ví dụ, giải mã trên lưới và chấm điểm lại) để trích ra chuỗi đồng thuận tối ưu [Deoras và cộng sự, 2011; Stahlberg và cộng sự, 2016; Khayrallah và cộng sự, 2017].

Lưu ý rằng việc kết hợp các mô hình chuỗi-sang-chuỗi có liên quan tới vấn đề đa dạng đã bàn ở tiểu mục trước. Người ta thừa nhận rộng rãi rằng các hệ thống thành phần phải đủ đa dạng thì việc kết hợp mới mang lại lợi ích đáng kể. Vì vậy, các mô hình nên được huấn luyện hoặc xây dựng sao cho nắm bắt các kiểu hành vi khác nhau [Sutskever và cộng sự, 2014; Zhou và cộng sự, 2017]. Một trong những phương pháp phổ biến nhất là kết hợp các điểm lưu (*checkpoint ensembling*). Nó dùng nhiều "ảnh chụp" của cùng một mô hình tại các điểm lưu (*checkpoint*) khác nhau trong quá trình huấn luyện và kết hợp các dự đoán của chúng bằng cách lấy trung bình. Phương pháp này có thể hữu ích để giảm nhẹ vấn đề quá khớp trong thực tế. Ngoài ra, có thể tạo ra các mô hình khác nhau từ một mô hình cơ sở với các thiết lập khác nhau. Ví dụ, ta có thể xây dựng các mô hình có số tham số khác nhau dựa trên một mô hình xương sống (*backbone*) chung. Một cách tổng quát hơn là dùng các mô hình dựa trên những kiến trúc khác nhau, dù điều này phải trả giá bằng nhiều công sức phát triển hơn.

Một cách nhìn khác về kết hợp mức chuỗi là nó tạo ra một sơ đồ tìm kiếm hai lượt. Ở lượt thứ nhất, nhiều hệ thống độc lập thực hiện suy luận. Vì mỗi hệ thống có những thiên kiến quy nạp riêng về mô hình hóa và giải mã, chúng tự nhiên khám phá những vùng khác nhau của không gian tìm kiếm. Một giả thuyết chất lượng cao được một hệ thống phát hiện có thể bị hệ thống khác bỏ lỡ hoàn toàn. Kết quả của lượt thứ nhất là một tập giả thuyết đa dạng, mỗi giả thuyết được xem là "tối ưu" theo một góc nhìn cụ thể. Ở lượt thứ hai, tập đa dạng này xác định một không gian tìm kiếm có

ràng buộc và chất lượng cao, trên đó một mô hình chấm điểm lại tinh vi có thể "điều hướng" để xác định kết quả cuối cùng, tối ưu toàn cục.

5.4.8 Thêm các mục tiêu tìm kiếm khác

Trong tiểu mục này, ta xét thêm các hàm mục tiêu khác có thể áp dụng cho bài toán tìm kiếm.

1. Tìm kiếm với điểm tương lai

Phần lớn các thuật toán mô tả ở trên có thể được xem là các biến thể của thuật toán tìm kiếm tốt nhất trước (*best-first search*) [Meister và cộng sự, 2020]. Tìm kiếm A* (*A* search*) là một ví dụ khác của thuật toán tìm kiếm tốt nhất trước và được coi rộng rãi là một giải pháp tốt cho bài toán tìm kiếm tổng quát. Tìm kiếm A* nguyên bản đòi hỏi sắp xếp mọi trạng thái tìm kiếm ở mỗi bước, điều không khả thi về tính toán trong nhiều bài toán NLP. Do đó, ta tiếp tục tập trung vào tìm kiếm chùm và tìm kiếm tham lam, nhưng dùng một hàm mục tiêu kiểu A*. Cụ thể, cho một trạng thái tìm kiếm $(\mathbf{x}, \mathbf{y}_{\leq i})$, hàm mục tiêu kiểu A* có thể được định nghĩa là:

$$\text{Score}_{\text{A}^*}(\mathbf{x}, \mathbf{y}_{\leq i}) \;=\; g(\mathbf{x}, \mathbf{y}_{\leq i}) + h(\mathbf{x}, \mathbf{y}_{\leq i}). \tag{5.110}$$

Ở đây $g(\mathbf{x}, \mathbf{y}_{\leq i})$ là phần thưởng tích lũy từ trạng thái bắt đầu đến $(\mathbf{x}, \mathbf{y}_{\leq i})$, còn $h(\mathbf{x}, \mathbf{y}_{\leq i})$ là phần thưởng ước lượng của đường đi "tối ưu" từ $(\mathbf{x}, \mathbf{y}_{\leq i})$ đến đích cuối cùng. Vì $g(\mathbf{x}, \mathbf{y}_{\leq i})$ và $h(\mathbf{x}, \mathbf{y}_{\leq i})$ có thể có dạng tùy ý, khung này rất tổng quát. Ví dụ, nếu ta định nghĩa

$$g(\mathbf{x}, \mathbf{y}_{\leq i}) = \text{Score}(\mathbf{x}, \mathbf{y}_{\leq i}), \tag{5.111}$$

$$h(\mathbf{x}, \mathbf{y}_{\leq i}) = 0, \tag{5.112}$$

thì $\text{Score}_{\text{A}^*}(\mathbf{x}, \mathbf{y})$ trở về các hàm mục tiêu đã bàn trước đó.

Để tận dụng đầy đủ cách phát biểu này, việc tìm một hàm ước lượng phần thưởng hay chi phí tương lai là điều tự nhiên. Lý tưởng nhất, ta muốn $h(\mathbf{x}, \mathbf{y}_{\leq i})$ tính chính xác lượng phần thưởng bổ sung có thể thu được khi mở rộng $(\mathbf{x}, \mathbf{y}_{\leq i})$ thành giả thuyết hoàn chỉnh tối ưu. Tuy nhiên, điều này không khả thi vì nó đòi hỏi khám phá mọi giả thuyết có thể mở rộng từ $(\mathbf{x}, \mathbf{y}_{\leq i})$ để tìm ra cái tốt nhất. Trong thực tế, người ta thường dùng một mô hình rẻ hơn về tính toán để xấp xỉ phần thưởng tương lai thực sự. Các cách

tiếp cận truyền thống dựa vào heuristic để định nghĩa $h(\mathbf{x}, \mathbf{y}_{\leq i})$ [Koehn và cộng sự, 2007], chẳng hạn ước lượng đóng góp của các từ có thể được sinh ra. Các heuristic này có thể được điều chỉnh dựa trên thiết kế kiến trúc của mô hình chuỗi-sang-chuỗi [He và cộng sự, 2017; Zheng và cộng sự, 2018]. Một cách tiếp cận có cơ sở hơn là dùng cách phát biểu dựa trên giá trị (*value-based*) [Ren và cộng sự, 2017; Li và cộng sự, 2017a; Leblond và cộng sự, 2021]. Ta có thể xây dựng một chính sách (policy) dự đoán phân phối của y_i khi biết $\mathbf{x}$ và $\mathbf{y}_{<i}$, cùng với một hàm giá trị (*value function*) ước lượng các phần thưởng tương lai. Do đó, Phương trình (5.110) có thể được hiểu là một tổ hợp tuyến tính của điểm chính sách cho $(\mathbf{x}, \mathbf{y}_{\leq i})$ và giá trị tương ứng của nó. Cách xử lý mục tiêu tìm kiếm như vậy thuộc khung tìm kiếm dựa trên giá trị (*value-based search*) và đã được dùng thành công trong học tăng cường [Silver và cộng sự, 2017].

Chú giải của người dịch: Ý tưởng của A*

Hãy nghĩ đến việc tìm đường bằng ứng dụng bản đồ. g là quãng đường bạn đã đi được (đã biết chắc), còn h là ước lượng quãng đường còn lại (ví dụ, khoảng cách đường chim bay đến đích). Chọn bước đi theo $g + h$ thay vì chỉ theo g giúp tránh những ngả đường "khởi đầu đẹp" nhưng dẫn ra xa đích. Trong sinh văn bản, h ước lượng "câu này nếu viết tiếp thì sẽ được thêm/bớt bao nhiêu điểm", giúp không loại bỏ quá sớm những giả thuyết đang có điểm thấp nhưng có triển vọng.

2. Tìm kiếm với mô hình ngôn ngữ

Từ lâu, các mô hình ngôn ngữ đã đóng vai trò trung tâm trong các tác vụ sinh văn bản. Ví dụ, các hệ thống dịch máy thống kê và nhận dạng tiếng nói tự động thường dựa vào các mô hình ngôn ngữ n-gram lớn để tạo ra văn bản trôi chảy. Mặc dù các mô hình chuỗi-sang-chuỗi hiện đại không cần mô hình ngôn ngữ riêng, việc tích hợp chúng trong quá trình tìm kiếm chuỗi-sang-chuỗi vẫn hấp dẫn về mặt trực giác đối với dịch máy và các bài toán liên quan.

Theo truyền thống coi mô hình ngôn ngữ như một đặc trưng của một mô hình log-tuyến tính (hoặc tuyến tính) [Och và Ney, 2002], mục tiêu có bổ sung mô hình ngôn ngữ có thể được định nghĩa là

$$\text{Score}_{\text{lm}}(\mathbf{x}, \mathbf{y}) \;=\; \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda \cdot \log \Pr(\mathbf{y}). \tag{5.113}$$

Cách phát biểu này không bao gồm các số hạng thưởng độ dài và chuẩn hóa độ dài, nhưng cả hai đều có thể dễ dàng được đưa vào như các đặc trưng bổ sung. Nói chung, mô hình ngôn ngữ $\Pr(\mathbf{y})$ chỉ được huấn luyện trên các chuỗi phía đích, cho phép dùng dữ liệu đơn ngữ (*monolingual*) quy mô lớn trong các mô hình chuỗi-sang-chuỗi [Gulcehre và cộng sự, 2017]. Điều thú vị là người ta nhận thấy các mô hình chuỗi-sang-chuỗi hiện nay tự bản thân chúng đã là những mô hình ngôn ngữ mạnh nếu được huấn luyện đủ, và một chiến lược hiệu quả hơn để tận dụng dữ liệu phía đích là tạo ra các kho ngữ liệu tổng hợp, một kỹ thuật được gọi là **tăng cường dữ liệu** (*data augmentation*). Một ví dụ là **dịch ngược** (*back-translation*), trong đó một hệ thống dịch từ phía đích sang phía nguồn dịch các câu đơn ngữ phía đích để sinh ra các câu nguồn tổng hợp. Dữ liệu song ngữ tổng hợp này sau đó được dùng để bổ sung cho tập huấn luyện [Sennrich và cộng sự, 2016a; Edunov và cộng sự, 2018]. Trong nhiều tác vụ, phương pháp đơn giản này có thể cải thiện đáng kể chất lượng bản dịch, nhưng những kết quả này đặt ra câu hỏi về sự cần thiết của việc dùng thêm mô hình ngôn ngữ trong dịch máy nơ-ron.

Lưu ý rằng mô hình ở Phương trình (5.113) phụ thuộc vào việc chọn hệ số λ. Với dịch máy, ta thường quan tâm tới giá trị dương của λ để hệ thống có thể tạo ra đầu ra trôi chảy hơn. Ngược lại, giá trị âm của λ nghĩa là ta muốn các đầu ra ít phổ biến hơn. Ví dụ, nếu $\lambda = -1$, Phương trình (5.113) có thể được viết thành thông tin tương hỗ theo điểm (*point-wise mutual information*) của $\mathbf{x}$ và $\mathbf{y}$

$$\begin{aligned}\text{Score}_{\text{lm}}(\mathbf{x}, \mathbf{y}) &= \log \Pr(\mathbf{y}|\mathbf{x}) - \log \Pr(\mathbf{y}) \\ &= \log \frac{\Pr(\mathbf{x}, \mathbf{y})}{\Pr(\mathbf{x}) \cdot \Pr(\mathbf{y})}. \end{aligned} \tag{5.114}$$

Hàm tính điểm này đã được chứng tỏ là hữu ích để sinh ra các đầu ra đa dạng hơn cho các hệ thống hội thoại nơ-ron [Li và cộng sự, 2016].

Chú giải của người dịch: Vì sao $\lambda = -1$ giúp tránh câu trả lời nhàm chán?

Chatbot thường hay trả lời "Tôi không biết" vì câu này có xác suất cao trong *mọi* ngữ cảnh. Trừ đi $\log \Pr(\mathbf{y})$ nghĩa là phạt những câu vốn phổ biến bất kể câu hỏi là gì. Ví dụ: nếu $\Pr(\mathbf{y}|\mathbf{x}) = 0.2$ và $\Pr(\mathbf{y}) = 0.1$ (câu chung chung), tỉ số là 2; còn một câu cụ thể có $\Pr(\mathbf{y}|\mathbf{x}) = 0.05$ nhưng $\Pr(\mathbf{y}) = 0.001$ thì tỉ

số là 50, nên câu cụ thể được ưu tiên. Dòng thứ hai của (5.114) dùng công thức xác suất có điều kiện $\Pr(\mathbf{y}|\mathbf{x}) = \frac{\Pr(\mathbf{x},\mathbf{y})}{\Pr(\mathbf{x})}$; tỉ số $\frac{\Pr(\mathbf{x},\mathbf{y})}{\Pr(\mathbf{x})\Pr(\mathbf{y})}$ bằng 1 khi $\mathbf{x}$ và $\mathbf{y}$ độc lập, và càng lớn khi chúng càng "gắn bó" với nhau.

3. Tìm kiếm rủi ro Bayes cực tiểu

Cho đến giờ, phần bàn về mục tiêu tìm kiếm của ta tập trung vào việc chọn giả thuyết có điểm cao nhất, một quá trình được gọi là tìm kiếm hậu nghiệm cực đại (*maximum a posteriori, MAP*) (MAP)[14]. Một giả định nền tảng của phương pháp này là xác suất hậu nghiệm $\Pr(\mathbf{y}|\mathbf{x})$ (hay điểm mô hình $\text{Score}(\mathbf{x}, \mathbf{y})$) tương quan với chất lượng của đầu ra. Trong thực tế, giả định này mang lại một số tính chất hữu ích. Chẳng hạn, hệ thống tìm kiếm dễ hiện thực, và mục tiêu tìm kiếm nhất quán với mục tiêu huấn luyện. Tuy nhiên, tìm kiếm MAP có một số nhược điểm, khiến các nhà nghiên cứu tìm kiếm những phương án mạnh hơn. Một vấn đề của tìm kiếm MAP là mục tiêu của nó không khớp với các độ đo đánh giá (*evaluation metric*) thực tế của hệ thống. Các độ đo dùng trong đánh giá đầu-cuối của một hệ thống có thể có dạng khác hẳn $\Pr(\mathbf{y}|\mathbf{x})$. Vấn đề thứ hai là MAP, với tư cách là một ước lượng điểm (*point estimate*) cụ thể trong lý thuyết quyết định

[14]Trong thống kê, MAP là một phương pháp suy ra các tham số của một mô hình thống kê. Giả sử ta có một mô hình mô tả phân phối của một biến x, được tham số hóa bởi θ. MAP tìm giá trị tối ưu của θ bằng cách cực đại hóa xác suất của θ khi biết x, viết là:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \Pr(\theta|x). \tag{5.115}$$

$\hat{\theta}_{\text{MAP}}$ còn được gọi là *mốt* (giá trị có xác suất lớn nhất) của phân phối hậu nghiệm của θ. Với bài toán tìm kiếm MAP ở đây, ta chỉ cần ký hiệu θ là $\mathbf{y}$ và tìm mốt của $\Pr(\mathbf{y}|\mathbf{x})$. Là một phương pháp Bayes, ta có thể viết lại phương trình trên bằng quy tắc Bayes

$$\begin{aligned}\hat{\theta}_{\text{MAP}} &= \arg\max_{\theta} \frac{\Pr(x|\theta) \cdot \Pr(\theta)}{\Pr(x)} \\ &= \arg\max_{\theta} \Pr(x|\theta) \cdot \Pr(\theta),\end{aligned} \tag{5.116}$$

trong đó θ được xem là một biến ngẫu nhiên với phân phối tiên nghiệm $\Pr(\theta)$. Ngược lại, MLE trực tiếp cực đại hóa hàm hợp lý $\Pr(x|\theta)$

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \Pr(x|\theta). \tag{5.117}$$

Do đó, kết quả MAP có thể được xem là một ước lượng của θ xét đến cả khả năng (likelihood) của x khi biết θ lẫn phân phối tiên nghiệm của θ. Lưu ý rằng MAP và MLE là tương đương nếu $\Pr(\theta)$ là phân phối đều.

Bayes (*Bayesian decision theory*), không cung cấp thước đo nào về độ bất định và thường quá tự tin. Trong một số ứng dụng, các mô hình chuỗi-sang-chuỗi trải khối lượng xác suất quá mỏng trên rất nhiều giả thuyết khác nhau [Ott và cộng sự, 2018a], nghĩa là ước lượng MAP có thể không nắm được phần lớn của phân phối thật.

Chú giải của người dịch: MAP, MLE và quy tắc Bayes

Xác suất có điều kiện $\Pr(A|B)$ là xác suất xảy ra A khi biết B đã xảy ra. *Quy tắc Bayes* cho phép "đảo chiều": $\Pr(\theta|x) = \frac{\Pr(x|\theta)\Pr(\theta)}{\Pr(x)}$. Mẫu số $\Pr(x)$ không phụ thuộc θ nên có thể bỏ khi tìm arg max, đó là dòng thứ hai của (5.116). Ví dụ: một đồng xu tung 3 lần đều ra mặt ngửa. MLE kết luận xác suất ra ngửa là 1 (vì 1^3 là lớn nhất); còn MAP, nếu ta có niềm tin ban đầu (tiên nghiệm) rằng đồng xu thường cân đối, sẽ cho một ước lượng "dè dặt" hơn, nằm giữa 0.5 và 1. Khi tiên nghiệm là đều (mọi giá trị như nhau), $\Pr(\theta)$ là hằng số và MAP trùng với MLE.

Ở đây, ta xét tìm kiếm **rủi ro Bayes cực tiểu** (*minimum Bayes risk, MBR*) (**MBR**), một cách có cơ sở để đưa các độ đo đánh giá vào quá trình giải mã và tận dụng toàn bộ phân phối trên các giả thuyết. Phương pháp MBR định nghĩa một hàm rủi ro (*risk function*) trên một cặp chuỗi, ký hiệu là $R(\mathbf{y}, \mathbf{y}_r)$. Hàm này tính chi phí của việc dự đoán $\mathbf{y}$ khi tham chiếu (*reference*) đúng là $\mathbf{y}_r$, được đánh giá theo một độ đo cụ thể. Ví dụ, trong dịch máy, ta có thể định nghĩa điểm rủi ro là $1 - \text{BLEU}$. Tổng rủi ro của một ứng viên $\mathbf{y}$ trên một tập Ω sau đó được định nghĩa là kỳ vọng của $R(\mathbf{y}, \mathbf{y}_r)$ theo phân phối hậu nghiệm $\Pr(\mathbf{y}_r|\mathbf{x})$:

$$\begin{aligned}\text{Risk}(\mathbf{y}) &= \mathbb{E}_{\mathbf{y}_r \sim \Pr(\mathbf{y}_r|\mathbf{x})} R(\mathbf{y}, \mathbf{y}_r) \\ &= \sum_{\mathbf{y}_r \in \Omega} R(\mathbf{y}, \mathbf{y}_r) \cdot \Pr(\mathbf{y}_r|\mathbf{x}). \end{aligned} \tag{5.118}$$

Tuy nhiên, phép lấy tổng trên mọi chuỗi phía đích có thể có là không khả thi về tính toán. Do đó, ta giới hạn Ω trong k đầu ra tốt nhất hoặc các đầu ra được lấy mẫu của một hệ thống [Eikema và Aziz, 2020], ký hiệu là Ω_{system}. Sau đó, ta lấy $\text{Score}(\mathbf{x}, \mathbf{y}) = -\text{Risk}(\mathbf{y})$ và thu được mục tiêu sau cho tìm kiếm MBR

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \; -\text{Risk}(\mathbf{y})$$

$$= \arg\min_{\mathbf{y}} \sum_{\mathbf{y}_r \in \Omega_{\text{system}}} R(\mathbf{y}, \mathbf{y}_r) \cdot \Pr(\mathbf{y}_r|\mathbf{x}). \tag{5.119}$$

Chú giải của người dịch: MBR: chọn ứng viên "được đồng thuận" nhất

Kỳ vọng $\mathbb{E}$ ở (5.118) đơn giản là trung bình có trọng số theo xác suất, giống như điểm trung bình có hệ số. Giả sử hệ thống sinh ra 3 bản dịch A, B, C với xác suất 0.4, 0.35, 0.25, trong đó B và C rất giống nhau còn A khác hẳn. MAP chọn A vì xác suất riêng lẻ cao nhất. MBR thì hỏi: "nếu chọn A, trung bình nó lệch bao nhiêu so với các bản dịch khả dĩ khác?". Vì B và C cộng lại chiếm 0.6 xác suất và giống nhau, chọn B (hoặc C) sẽ có rủi ro trung bình thấp hơn. Như vậy, MBR ưa ứng viên nằm ở "trung tâm" của phân phối, giống như chọn ý kiến được nhiều người đồng tình nhất.

Mô hình này rất tổng quát và áp dụng được cho nhiều bài toán NLP, trong đó cần tìm một giả thuyết tối ưu trong một tập lớn các ứng viên [Goodman, 1996b; Goel và Byrne, 2000; Kumar và Byrne, 2004b]. Nó cho phép các dạng hàm rủi ro linh hoạt, chẳng hạn kết hợp nhiều yếu tố khác nhau khi đánh giá các giả thuyết. Gần đây, tìm kiếm MBR được các nhà nghiên cứu NLP quan tâm vì người ta thấy nó hiệu quả trong việc loại bỏ các thiên lệch do tìm kiếm MAP gây ra [Müller và Sennrich, 2021; Freitag và cộng sự, 2022]. Bên cạnh việc cung cấp một cách phát biểu mục tiêu tìm kiếm, các phương pháp MBR còn có thể được dùng để huấn luyện các mô hình chuỗi-sang-chuỗi, và được xem là một giải pháp hứa hẹn cho sự chênh lệch giữa mục tiêu huấn luyện và mục tiêu đánh giá [Shen và cộng sự, 2016].

5.5 Tóm tắt

Trong chương này, chúng tôi đã trình bày tổng quan về mô hình hóa chuỗi-sang-chuỗi, vốn là nền tảng của nhiều hệ thống NLP hiện đại. Mô hình hóa chuỗi-sang-chuỗi là một lĩnh vực nghiên cứu rất phong phú, và đã được bàn luận rộng rãi trong nhiều ngành khác nhau, thậm chí vượt ra ngoài NLP. Chương này không phải là một bài tổng quan toàn bộ tài liệu về chủ đề này, mà tập trung vào một số phương pháp và ý tưởng cốt lõi. Chúng tôi bắt đầu bằng việc giới thiệu các bài toán chuỗi-sang-chuỗi, cũng như

kiến trúc bộ mã hóa–bộ giải mã, vốn đặt nền móng cho hầu hết các hệ thống chuỗi-sang-chuỗi tiên tiến nhất. Để minh họa ứng dụng của kiến trúc này, chúng tôi đã xét bài toán dịch máy nơ-ron và xây dựng một mô hình dịch máy nơ-ron đơn giản bằng những kiến thức cơ bản đã học cho đến lúc đó.

Chúng tôi cũng đã trình bày cơ chế chú ý và một loạt cải tiến của nó. Nhìn lại vài năm qua, ta thấy rằng khám phá các mô hình chú ý là bước tiếp theo tự nhiên trong việc phát triển các mô hình chuỗi-sang-chuỗi. Mặc dù các mô hình chú ý ban đầu trở nên phổ biến nhờ thành công trong dịch máy, từ đó chúng đã chiếm ưu thế trong cộng đồng NLP. Các mô hình chú ý cũng rất được quan tâm trong một số lĩnh vực con khác của AI, như thị giác máy tính (*computer vision*) [Borji và Itti, 2012; Xu và cộng sự, 2015; Jaderberg và cộng sự, 2015] và xử lý tiếng nói (*speech processing*) [Chorowski và cộng sự, 2015; Chan và cộng sự, 2016; Bahdanau và cộng sự, 2016]. Nhờ vậy, vài năm qua là quãng thời gian đầy hứng khởi đối với những người làm việc trong các lĩnh vực này.

Các mô hình chuỗi-sang-chuỗi thành công đến mức có một xu hướng ngày càng mạnh là thống nhất các bài toán đa dạng vào một khung duy nhất. Không chỉ là ta đã phát triển được các mô hình chuỗi-sang-chuỗi mạnh để giải quyết các bài toán rất tổng quát, mà các xu hướng nghiên cứu hiện nay ngày càng ưa chuộng các cách tiếp cận thống nhất. Một ví dụ là Transformer, một mô hình chuỗi-sang-chuỗi dựa trên tự chú ý, đã trở thành xương sống nền tảng cho các tác vụ trải rộng trên nhiều phương thức dữ liệu (*data modality*) khác nhau, từ văn bản đến hình ảnh và tiếng nói. Nó thậm chí có thể được mở rộng để xử lý các bài toán đa phương thức (*multimodal*), vốn đôi khi khó khăn hơn. Điều này làm mọi thứ trở nên thú vị và hấp dẫn hơn: một cải tiến cho một mô hình có thể được dùng để cải thiện các hệ thống trong nhiều tác vụ khác nhau. Và ta đang chứng kiến một thay đổi đáng kể trong mô thức nghiên cứu (*research paradigm*), trong đó các lĩnh vực NLP và học máy đang hội tụ, và các kết quả nghiên cứu NLP ngày càng có ảnh hưởng hơn. Tuy nhiên, mặt trái của đồng xu là ta đang dành nhiều sự chú ý hơn cho một số bài toán trong khi lơ là những bài toán khác. Trong các hội nghị NLP gần đây, có rất nhiều bài báo bàn về cách huấn luyện các mô hình chuỗi-sang-chuỗi lớn và áp dụng chúng vào các tác vụ sinh văn bản khác nhau, nhưng lại có tương đối ít bài báo về phân tích cú pháp (*parsing*). Đã luôn có những tranh luận về điều này

trong vài thập kỷ qua, chẳng hạn: ta cần những tri thức tiên nghiệm nào, và cần bao nhiêu, để xây dựng một hệ thống NLP? [Church, 2011; See, 2018] Một thảo luận chi tiết về các tranh luận này nằm ngoài phạm vi của chương. May mắn là nghiên cứu NLP hứa hẹn sẽ tiếp tục đa dạng và sôi động, mang lại những hiểu biết giá trị từ cả hai góc nhìn. Ví dụ, có những phát hiện thú vị rằng các mô hình chuỗi nơ-ron có thể học được một số thuộc tính ngôn ngữ học từ dữ liệu, và các cấu trúc ngôn ngữ học có thể giúp ích cho việc thiết kế hệ thống. Ở Chương 6, ta sẽ thấy một vài ví dụ.

Sự "thiên lệch" về trọng tâm nghiên cứu cũng tồn tại ở phía học máy của việc giải quyết vấn đề. Ví dụ, với các bài toán chuỗi-sang-chuỗi bàn ở đây, những năm gần đây đã chứng kiến sự quan tâm tăng vọt đối với thiết kế mô hình và phương pháp huấn luyện, nhưng chỉ một nhóm tương đối nhỏ người bàn về bài toán tìm kiếm. Mặc dù tìm kiếm là một bài toán kinh điển trong AI và đóng vai trò quan trọng trong các hệ thống thực tế [Russell và Norvig, 2010], nó thậm chí không được bàn đến trong các bài hướng dẫn và bài tổng quan gần đây về NLP. Điều này thôi thúc chúng tôi viết một mục về chủ đề này để có được bức tranh đầy đủ hơn về bài toán. Tuy nhiên, phần bàn luận chung của chúng tôi không bao quát mọi khía cạnh của bài toán tìm kiếm. Một chủ đề quan trọng mà chúng tôi bỏ qua là hiệu quả [Birch và cộng sự, 2018; Heafield và cộng sự, 2021]. Dù chương này có một số thảo luận về vấn đề hiệu quả, như các tiêu chí dừng của thuật toán tìm kiếm, các phương pháp hiệu quả là một chủ đề rộng và nhìn chung phụ thuộc vào kiến trúc mô hình. Có thể tìm thấy thảo luận chi tiết hơn về chúng ở Chương 6. Một chủ đề khác mà bạn đọc có thể quan tâm là tìm kiếm có ràng buộc (*constrained search*), trong đó các ràng buộc được áp đặt lên quá trình tìm kiếm [Hokamp và Liu, 2017; Anderson và cộng sự, 2017]. Nói chung, các ràng buộc này đến từ tri thức tiên nghiệm của ta hoặc từ tương tác với người dùng. Ví dụ, tìm kiếm có ràng buộc đã được dùng để áp đặt các ràng buộc dịch thuật ngữ trong dịch máy [Hasler và cộng sự, 2018; Post và Vilar, 2018].

Một lưu ý cuối cùng về các hạn chế của chương này. Cách phát biểu bài toán chuỗi-sang-chuỗi tổng quát mô tả ở đây dựa trên phép phân tích từ trái sang phải của $\Pr(\mathbf{y}|\mathbf{x})$, dẫn đến một mô hình tự hồi quy (*autoregressive model*). Một hạn chế của cách phát biểu này là mỗi dự đoán ở một bước nào đó chỉ phụ thuộc vào các từ đứng trước, nên mô hình không thể truy cập ngữ cảnh bên phải. Để tận dụng ngữ cảnh bên phải của một từ, một

cách đơn giản là xây dựng thêm một mô hình sinh từ phải sang trái. Các mô hình trái-sang-phải và phải-sang-trái sau đó có thể được kết hợp để sinh ra chuỗi đầu ra tốt hơn [Liu và cộng sự, 2016a; Hoang và cộng sự, 2017; Zhang và cộng sự, 2018b; 2020a]. Một cách tiếp cận khác là **sinh phi tự hồi quy** (*non-autoregressive generation*) hay **giải mã phi tự hồi quy** (*non-autoregressive decoding*), trong đó ràng buộc của việc sinh tự hồi quy được loại bỏ và mỗi dự đoán từ được đặt điều kiện trên ngữ cảnh toàn cục [Gu và cộng sự, 2018; Ghazvininejad và cộng sự, 2019; Lee và cộng sự, 2020]. Một ưu điểm then chốt của sinh phi tự hồi quy là khả năng tăng tốc hệ thống, vì mọi từ trong một chuỗi có thể được sinh song song và ta có thể làm điều này hiệu quả bằng GPU.

Chú giải của người dịch: Tự hồi quy và phi tự hồi quy

Tự hồi quy giống như viết câu từng chữ một từ trái sang phải: muốn viết chữ thứ 5 phải đợi viết xong 4 chữ đầu. *Phi tự hồi quy* giống như điền đồng thời mọi ô của một ô chữ khi đã biết trước số ô: các vị trí được dự đoán cùng lúc, nên nhanh hơn nhiều trên GPU (vốn giỏi làm nhiều phép tính song song), nhưng khó bảo đảm các từ ăn khớp với nhau vì chúng không "nhìn thấy" lựa chọn của nhau trong lúc sinh.

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.